cecuro

Audit Report

October 16, 2026

PROJECT

Plether Labs

Audit Overview

Project:	Plether Labs
Repository:	https://github.com/Plether-Fi/plether-core
Audit Date:	October 16, 2026
Commit:	6d6622f2
Scope:	129 files

Findings overview

ADJUSTED BY TEAM

0	0	30	73	7
CRITICAL	HIGH (1)	MEDIUM (32)	LOW (74)	INFO (6)

Disclaimer: The severities in parentheses are the original classifications from Cecuro's automated auditing system. Unlike traditional audits, the AI-based process does not support iterative severity alignment between auditor and development team. The project team has since reclassified 1 finding and marked 3 as invalid based on their own context and judgment. Cecuro does not review or verify adjustments made after delivery.

Findings Overview

ID	SEVERITY	TITLE	STATUS
MED-1	Medium	Owner can release pending claimant reserves to LPs using a mark older than the engine limit	Acknowledged
MED-2	Medium	Owner can miswire settlement tokens and leave traders with unbacked or stuck balances	Acknowledged
MED-3	Medium	Anyone can block governance from rotating the protocol treasury	Acknowledged
MED-4	Medium	Attacker can make later traders miss execution windows by filling the FIFO with stale orders	Acknowledged
MED-5	Medium	Final refund claimant can lock redemption refunds for boot-strap LPs	Acknowledged
MED-6	Medium	Protocol treasury can spend LP withdrawal reserves on fee top-ups	Acknowledged
MED-7	Medium	Keeper can settle traders and LPs on selectively updated basket components	Acknowledged
MED-8	Medium	Trader can be blocked from triggering protection because later L2 blocks reuse the same block number	Acknowledged
MED-9	Medium	Keepers can retry protection closes without the trader's financial limits	Acknowledged

MED-10	Medium	Liquidator can liquidate short traders at prices below the configured cap-adjusted mark	Acknowledged
MED-11	Medium	Senior redeemer can leave remaining senior principal un-protected	Acknowledged
MED-12	Medium	Mixed recovery inflows can strand LP revenue in ownerless pool assets	Acknowledged
MED-13	Medium	Governance can retarget armed stop-losses after oracle rotation	Acknowledged
MED-14	Medium	Trader can be blocked from queueing an open behind their closing order	Acknowledged
MED-15	Medium	Triggered SHORT protection can get stuck and leave a user unprotected	Acknowledged
MED-16	Medium	Traders can enqueue invalid opens when the cached mark is stale	Acknowledged
MED-17	Medium	Oracle lag can block traders from triggering protections	Acknowledged
MED-18	Medium	Keeper can rewrite the engine mark at the same timestamp and change liquidation and LP settlement prices	Acknowledged
MED-19	Medium	Senior depositor can block others by cycling refundable reservations	Acknowledged
MED-20	Medium	Frozen close orders can be blocked by live publish-time divergence limits	Acknowledged
MED-21	Medium	Admin can rotate in a zero-weight feed that still bricks the oracle	Acknowledged
MED-22	Medium	Trader can avoid paying LP carry by checkpointing early	Acknowledged
MED-23	Medium	Mixed-time basket can block trader protections during valid mark updates	Acknowledged
MED-24	Medium	LP can delay other redeemers with zero-payout refund epochs	Acknowledged
MED-25	Medium	Arbitrum block numbering can delay every later trader in the FIFO queue	Acknowledged
MED-26	Medium	Owner can activate trading against ownerless pool assets	Acknowledged
MED-27	Medium	Owner can change senior rate while junior LP accounting is stale	Acknowledged
MED-28	Medium	Trader can arm a new protection from a stale engine mark after oracle state already crossed it	Acknowledged
MED-29	Medium	Small redeemers can lose funded withdrawal value to the pool and seed account	Acknowledged
MED-30	Medium	Trader can lose stop-loss automation during routine frozen markets	Acknowledged
MED-31	Medium	Coordinated zaps transfer value between BEAR and BULL DOVs	Invalid
MED-32	Medium	Settled BULL collateral regains borrowing power below CAP	Invalid
LOW-1	Low	Indexer can misattribute tranche deposit lifecycle to the wrong account	Acknowledged

LOW-2	Low	Unset mark can block valid first opens	Acknowledged
LOW-3	Low	LP can have smaller share claims revert on a funded deposit	Acknowledged
LOW-4	Low	LP can have smaller withdrawal claims revert on a funded redeem	Acknowledged
LOW-5	Low	LP can have small deposit claims revert below maxDeposit	Acknowledged
LOW-6	Low	User can permanently strand tokens in MarginClearinghouse	Acknowledged
LOW-7	Low	Integrator can misclassify zero add-margin attempts	Acknowledged
LOW-8	Low	Integrators can underquote required open collateral for small positions	Acknowledged
LOW-9	Low	Integrators can misread immediate close cash from previews	Acknowledged
LOW-10	Low	ABI consumers can miss the public self-redeem-cancel selector advertised by the vault id	Acknowledged
LOW-11	Low	Integrator can misread risk-off cutoff and misclassify unaffected close orders	Acknowledged
LOW-12	Low	Operator can treat a stale cached mark as an execution blocker on the atomic settlement route	Acknowledged
LOW-13	Low	LP claimants can lose async rounding dust to other tranche holders	Acknowledged
LOW-14	Low	Integrators can overestimate request capacity because maxRequestDeposit ignores controller	Acknowledged
LOW-15	Low	Product consumers can miss router risk-off in protocol status	Acknowledged
LOW-16	Low	lp integrators can overestimate withdrawal readiness after an epoch rolls	Acknowledged
LOW-17	Low	Integrators can overquote redeem dust that the vault will still reject	Acknowledged
LOW-18	Low	Keeper API docs can hide protection bounty forfeiture from liquidation callers	Acknowledged
LOW-19	Low	Integrator can overstate trader settlement after liquidation	Acknowledged
LOW-20	Low	monitor callers can see preview accounting as available while terminal deficit data is invalid	Acknowledged
LOW-21	Low	Monitor can block complete observations for cached settlement when the oracle view is unreadable	Acknowledged
LOW-22	Low	Governance cannot use the documented non-dilutive recapitalization path	Acknowledged
LOW-23	Low	Integrators can misread senior withdrawal capacity during settlement holds	Acknowledged
LOW-24	Low	Pauser can brick emergency pause when no order has ever been committed	Acknowledged
LOW-25	Low	Receipt verifiers can reject expired and config-mismatch evidence	Acknowledged

LOW-26	Low	Trader can be blocked from opening by underfunded protocol fee top-ups	Acknowledged
LOW-27	Low	Anyone can force zero-share junior fee accrual to permanently skip completed hours	Acknowledged
LOW-28	Low	Liquidation preview callers can misclassify risk-off-refunded accounts	Acknowledged
LOW-29	Low	Keeper tooling can misclassify stale trigger data as same-block protection failure	Acknowledged
LOW-30	Low	Lens caller can receive stale post-VPI accrual in invalid open previews	Acknowledged
LOW-31	Low	Trader can lose a capped bounty on undersized opens before the first mark update	Acknowledged
LOW-32	Low	Preview callers can misread partial close failures	Acknowledged
LOW-33	Low	Deployer can brick the router by binding a non-contract engine lens or house pool	Acknowledged
LOW-34	Low	Integrators can overstate close carry because the evaluator reports pending carry after collection	Acknowledged
LOW-35	Low	Integrator can misread trader account equity from the public view interface	Acknowledged
LOW-36	Low	Trader or keeper can use inverted feeds to bypass the basket confidence cap	Acknowledged
LOW-37	Low	Integrator can receive wrong side-margin totals from planner	Acknowledged
LOW-38	Low	Owner can temporarily disable oracle-backed operations by rotating to an oracle with a non-contract Pyth endpoint	Acknowledged
LOW-39	Low	Front-runner can block third-party vault share claims by dusting the receiver	Acknowledged
LOW-40	Low	Controller can force a zero-balance receiver into LP cooldown	Acknowledged
LOW-41	Low	Integrator can see live status before engine wiring is complete	Acknowledged
LOW-42	Low	Governance can misclassify protocol-driven close failures as user faults in receipts	Acknowledged
LOW-43	Low	Integrator can query wrong async vault limits through IERC7575	Acknowledged
LOW-44	Low	Governance can set a minimum order size that partial closes ignore	Acknowledged
LOW-45	Low	Expiring keeper can finalize receipts with a zero observed config hash	Acknowledged
LOW-46	Low	Trader can have orders stuck until expiry when terminal book is missing	Acknowledged
LOW-47	Low	Keepers can lose mark context from mismatch receipts	Acknowledged
LOW-48	Low	LP receipts can report gross pool depth instead of net economic depth	Acknowledged

LOW-49	Low	Trader can open with equity exactly at the initial-margin threshold	Acknowledged
LOW-50	Low	Reservation mismatch cleanup can strip a surviving position's protection	Acknowledged
LOW-51	Low	Integrator can understate active maintenance during FAD windows	Acknowledged
LOW-52	Low	Keeper can underfund liquidation batch item calls	Acknowledged
LOW-53	Low	Trader previews can overstate committable margin	Acknowledged
LOW-54	Low	Senior redeemers can leave dust high-water mark by splitting withdrawals	Acknowledged
LOW-55	Low	Integrator can be misled by close preview accepting uncommittable client ids	Acknowledged
LOW-56	Low	Integrators can mis-handle rejected deposits because the lens says they stay pending	Acknowledged
LOW-57	Low	Trader can underpay open-order keeper bounty by committing against a stale mark	Acknowledged
LOW-58	Low	Governance can make withdrawable-margin views revert for traders	Acknowledged
LOW-59	Low	Owner can leave emergency recovery ownerless	Acknowledged
LOW-60	Low	Lens callers can crash max-open quotes with oversized margin inputs	Acknowledged
LOW-61	Low	PoolReconcile reads can enforce the order skew limit instead of the pool policy	Acknowledged
LOW-62	Low	Contract caller can permanently lock deferred ETH refunds	Acknowledged
LOW-63	Low	Partial LP settlement can round dust away from redeemers	Acknowledged
LOW-64	Low	Integrator can mis-handle settlement debits because `settleUsdc` documents the wrong limit	Acknowledged
LOW-65	Low	Owner can misroute ownerless pool assets to the junior seed receiver	Acknowledged
LOW-66	Low	Integrator can misread live position margin from engine views	Acknowledged
LOW-67	Low	Same-window redeemer can burn a small LP's shares for no assets	Acknowledged
LOW-68	Low	Broken oracle payloads can be misreported as unavailable history	Acknowledged
LOW-69	Low	Integrators can misread applyOpenCost return value and overestimate pledged margin	Acknowledged
LOW-70	Low	Keeper cannot use single-order execution to preserve progress on unavailable history	Acknowledged
LOW-71	Low	Protocol lens can understate pool assets by subtracting treasury credits that never leave the pool	Acknowledged
LOW-72	Low	Pending depositor can receive impossible share quotes for a wiped tranche	Acknowledged

LOW-73	Low	Trader cannot add new stop-loss protection while degraded mode is active	Acknowledged
LOW-74	Low	Exercise can pay collateral again after the 90-day sweep	Invalid
HIGH-1	Info	Trader can close positions at already-known oracle ticks against LPs	Acknowledged
INFO-1	Info	Caller can strand its own ETH refund by using an un-claimable recipient	Open
INFO-2	Info	Owner can cancel pool config without an event	Open
INFO-3	Info	Router can return a misleading oracle rotation error	Acknowledged
INFO-4	Info	Integrator can misread settlement buffer checks from stale evaluator docs	Acknowledged
INFO-5	Info	Integrator can misclassify open-order failures in planner policy tooling	Acknowledged
INFO-6	Info	Operators can misread recovery-sidecar failures as execution-sidecar failures	Acknowledged

Security audit completed.

Audit Scope

The following 129 files were included in this security audit:

- packages/perps/src/CfdEngine.sol
- packages/perps/src/CfdEngineAdmin.sol
- packages/perps/src/CfdEngineLens.sol
- packages/perps/src/CfdEngineOpenQuoter.sol
- packages/perps/src/CfdEnginePlanTypes.sol
- packages/perps/src/CfdEnginePlanner.sol
- packages/perps/src/CfdEngineProtocolLens.sol
- packages/perps/src/CfdEngineSettlementSidecar.sol
- packages/perps/src/CfdMath.sol
- packages/perps/src/CfdOrderPolicyEvaluator.sol
- packages/perps/src/CfdOrderPolicyEvaluatorBase.sol
- packages/perps/src/CfdTypes.sol
- packages/perps/src/EmergencyPauseCoordinator.sol
- packages/perps/src/HousePool.sol
- packages/perps/src/HousePoolRedemptionMathSidecar.sol
- packages/perps/src/MarginClearinghouse.sol
- packages/perps/src/OrderLifecycleBook.sol
- packages/perps/src/OrderRecoverySidecar.sol
- packages/perps/src/OrderRouter.sol
- packages/perps/src/OrderRouterAdmin.sol
- packages/perps/src/OrderRouterExecutionSidecar.sol
- packages/perps/src/OrderRouterLiquidationBatchSidecar.sol
- packages/perps/src/OrderV3Types.sol
- packages/perps/src/PletherOracle.sol
- packages/perps/src/PositionProtectionBook.sol
- packages/perps/src/TerminalNavBookV2.sol
- packages/perps/src/TrancheVault.sol
- packages/perps/src/interfaces/AccountLensViewTypes.sol
- packages/perps/src/interfaces/CfdEngineSettlementTypes.sol
- packages/perps/src/interfaces/ClaimEngineViewTypes.sol
- packages/perps/src/interfaces/EngineStatusViewTypes.sol
- packages/perps/src/interfaces/HousePoolEngineViewTypes.sol
- packages/perps/src/interfaces/IAsyncTrancheVault.sol
- packages/perps/src/interfaces/IPerpsLPViews.sol
- packages/perps/src/interfaces/IPerpsTrader.sol
- packages/perps/src/interfaces/IPerpsTraderActions.sol
- packages/perps/src/interfaces/IPerpsTraderViews.sol
- packages/perps/src/interfaces/IPletherOracle.sol
- packages/perps/src/interfaces/IPositionProtectionActions.sol
- packages/perps/src/interfaces/IPositionProtectionBook.sol
- packages/perps/src/interfaces/IPositionProtectionViews.sol
- packages/perps/src/interfaces/IProtocolViews.sol
- packages/perps/src/interfaces/ISettlementMonitorLens.sol
- packages/perps/src/interfaces/ITerminalNavBookV2.sol
- packages/perps/src/interfaces/ITrancheVaultBootstrap.sol
- packages/perps/src/interfaces/IWithdrawGuard.sol
- packages/perps/src/interfaces/PerpsViewTypes.sol
- packages/perps/src/interfaces/PositionProtectionTypes.sol
- packages/perps/src/interfaces/ProtocolLensViewTypes.sol
- packages/perps/src/interfaces/SettlementMonitorViewTypes.sol
- packages/perps/src/libraries/CashPriorityLib.sol
- packages/perps/src/libraries/CfdEngineCollateralSnapshotLib.sol
- packages/perps/src/libraries/CfdEnginePlanLib.sol
- packages/perps/src/libraries/CloseAccountingLib.sol
- packages/perps/src/libraries/CloseCommitmentLib.sol
- packages/perps/src/libraries/HousePoolAccountingLib.sol
- packages/perps/src/libraries/HousePoolFreshnessLib.sol
- packages/perps/src/libraries/HousePoolPendingPreviewLib.sol
- packages/perps/src/libraries/HousePoolReconcilePlanLib.sol
- packages/perps/src/libraries/HousePoolRedemptionMathLib.sol

packages/perps/src/interfaces/IAsyncTrancheVaultClaimableRedeem.sol

packages/perps/src/interfaces/ICfdEngine.sol

packages/perps/src/interfaces/ICfdEngineAccountLens.sol

packages/perps/src/interfaces/ICfdEngineAdminHost.sol

packages/perps/src/interfaces/ICfdEngineCore.sol

packages/perps/src/interfaces/ICfdEngineLens.sol

packages/perps/src/interfaces/ICfdEnginePlanner.sol

packages/perps/src/interfaces/ICfdEngineProtocolLens.sol

packages/perps/src/interfaces/ICfdEngineRiskParamsView.sol

packages/perps/src/interfaces/ICfdEngineSettlementHost.sol

packages/perps/src/interfaces/ICfdEngineSettlementSidecar.sol

packages/perps/src/interfaces/ICfdEngineTypes.sol

packages/perps/src/interfaces/ICfdOrderPolicyEvaluator.sol

packages/perps/src/interfaces/IERC7540.sol

packages/perps/src/interfaces/IERC7575.sol

packages/perps/src/interfaces/IHousePool.sol

packages/perps/src/interfaces/IHousePoolEmergencyAdmin.sol

packages/perps/src/interfaces/IHousePoolRedemptionMathSidecar.sol

packages/perps/src/interfaces/IMarginAccount.sol

packages/perps/src/interfaces/IMarginClearinghouse.sol

packages/perps/src/interfaces/IOrderLifecycleBook.sol

packages/perps/src/interfaces/IOrderRouter.sol

packages/perps/src/interfaces/IOrderRouterAccounting.sol

packages/perps/src/interfaces/IOrderRouterAdminHost.sol

packages/perps/src/interfaces/IOrderRouterEmergencyAdmin.sol

packages/perps/src/interfaces/IOrderRouterErrors.sol

packages/perps/src/interfaces/IOrderRouterExecutionHost.sol

packages/perps/src/interfaces/IOrderRouterLiquidationBatchHost.sol

packages/perps/src/interfaces/IPerpsAdmin.sol

packages/perps/src/interfaces/IPerpsKeeper.sol

packages/perps/src/interfaces/IPerpsLP.sol

packages/perps/src/interfaces/IPerpsLPActions.sol

packages/perps/src/libraries/HousePoolSeedLifecycleLib.sol

packages/perps/src/libraries/HousePoolSeniorCapacityLib.sol

packages/perps/src/libraries/HousePoolTrancheGateLib.sol

packages/perps/src/libraries/HousePoolWaterfallAccountingLib.sol

packages/perps/src/libraries/HousePoolWithdrawalPreviewLib.sol

packages/perps/src/libraries/JuniorMaintenanceFeeMathLib.sol

packages/perps/src/libraries/LiquidationAccountingLib.sol

packages/perps/src/libraries/MarginClearinghouseAccountingLib.sol

packages/perps/src/libraries/MarketCalendarLib.sol

packages/perps/src/libraries/OpenAccountingLib.sol

packages/perps/src/libraries/OracleFreshnessPolicyLib.sol

packages/perps/src/libraries/OrderFailurePolicyLib.sol

packages/perps/src/libraries/OrderSettlementFloorLib.sol

packages/perps/src/libraries/OrderValidationLib.sol

packages/perps/src/libraries/PositionRiskAccountingLib.sol

packages/perps/src/libraries/SolvencyAccountingLib.sol

packages/perps/src/router/OrderBountyAccounting.sol

packages/perps/src/router/OrderCommitHandler.sol

packages/perps/src/router/OrderExecutionHandler.sol

packages/perps/src/router/OrderExecutionOrchestrator.sol

packages/perps/src/router/OrderExecutionSettlement.sol

packages/perps/src/router/OrderHandler.sol

packages/perps/src/router/OrderLiquidationBatchLogic.sol

packages/perps/src/router/OrderLiquidationHandler.sol

packages/perps/src/router/OrderOracleExecution.sol

packages/perps/src/router/OrderQueueBook.sol

packages/perps/src/router/OrderReservationAccounting.sol

packages/perps/src/router/OrderRouterBase.sol

packages/perps/src/router/OrderValidation.sol

packages/perps/src/router/PositionProtectionHandler.sol

packages/options/src/DOVZapRouter.sol

packages/options/src/PletherDOV.sol

packages/spot/src/oracles/MorphoOracle.sol

packages/spot/src/core/SyntheticSplitter.sol

packages/spot/src/oracles/StakedOracle.sol

script/DeployToMainnet.s.sol

packages/options/src/MarginEngine.sol

Findings

Medium ACKNOWLEDGED

Owner can release pending claimant reserves to LPs using a mark older than the engine limit

ID: MED-1

Comment

MED-1 — Addressed. Senior-rate finalization now checks the current pool freshness limit and the effective PoolReconcile policy before reconciling or consuming pending claimant buckets. The existing coupon rate is checkpointed before the new rate becomes active. A mark that passes the pool limit but fails the tighter engine limit reverts without releasing reserves or changing configuration.

Fix:
https://github.com/Plether-Fi/plether-core/commit/21f06ac69b6e3c6b8d5068959194575c209dcdd5

Locations:

- packages/perps/src/HousePool.sol:319-330
- packages/perps/src/HousePool.sol:1438-1445
- packages/perps/src/HousePool.sol:1667-1683
- packages/perps/src/HousePool.sol:1700-1707
- packages/perps/src/HousePool.sol:1802-1806
- packages/perps/src/CfdEngineProtocolLens.sol:77-91
- packages/perps/src/CfdEngineAdmin.sol:201-226

A senior-rate finalization can consume pending claimant buckets even when the cached oracle mark is already stale under the protocol's real PoolReconcile policy. The pool-wide freshness policy is the tighter of `engineMarkStalenessLimit` and `pool.markStalenessLimit`, and `CfdEngineProtocolLens.getHousePoolInputSnapshot()` builds HousePool snapshots with that tighter value. But `HousePool.finalizePoolConfig()` gates rate changes with `_requireRateChangeMarkFresh()`, which checks only `poolConfig.markStalenessLimit`. If governance has tightened the engine live limit below the pool limit, a mark that is 30 seconds old with `(engine=10s, pool=60s)` passes the finalization gate even though normal HousePool reconcile and withdrawal logic would treat it as stale. That mismatch matters because `_checkpointSeniorCouponBeforeRateChange()` still builds `reconcileSnapshot` from the stale terminal NAV and calls `_applyPendingClaimantBucketsLive()` on the portion of `pendingRecapitalizationUsdc` and `pendingTradingRevenueUsdc` that appears settleable under that stale snapshot. Those buckets are otherwise explicitly reserved from withdrawals by `_buildWithdrawalSnapshot()`. In a concrete trace, assume physical assets are 1,000, reserved liabilities are 900, and there is `pendingTradingRevenueUsdc = 100`. With the correct fresh mark, distributable value is 0, so the 100 USDC should stay reserved. If the 30-second-old mark still reports distributable at least 100, `finalizePoolConfig()` consumes the bucket, moves it into tranche principals, and clears the reservation. `getFreeUSDC()` then rises from 0 to 100 and LP redemptions can pull out value that should have remained quarantined until a truly fresh reconcile.

Preconditions

- Open positions exist so HousePool freshness is active and `markFreshnessRequired` is true (`packages/perps/src/CfdEngineProtocolLens.sol:77-91`).
- Governance has configured `engineMarkStalenessLimit` lower than `pool.markStalenessLimit`, and the cached mark age falls between those two bounds (`packages/perps/src/CfdEngineAdmin.sol:201-226`, `packages/perps/src/HousePool.sol:1438-1445`).
- A pending senior-rate change is finalized while `pendingRecapitalizationUsdc` or `pendingTradingRevenueUsdc` is nonzero (`packages/perps/src/HousePool.sol:319-330`, `packages/perps/src/HousePool.sol:1751-1775`).

Impact

Pending claimant assets that should stay reserved until a fresh reconcile can be converted into tranche principal and become withdrawable. That breaks the protocol's oracle-freshness boundary for LP accounting and can misprice LP state or leak reserved pool value to redeemers.

Remediation

Use the same effective PoolReconcile freshness check in `finalizePoolConfig()` that HousePool uses everywhere else. Replace `_requireRateChangeMarkFresh()` with `_requireFreshMark(_getHousePoolInputSnapshot(), _getHousePoolStatusSnapshot())`, or otherwise enforce the tighter engine-and-pool staleness limit before any claimant bucket settlement runs.

Medium ACKNOWLEDGED

Owner can miswire settlement tokens and leave traders with unbacked or stuck balances

ID: MED-2

Comment

MED-2 — Addressed. CfdEngine construction now requires a deployed clearinghouse whose settlementAsset() matches the Engine's USDC. The one-time setPool() setter requires deployed code, the same USDC token, and ENGINE() equal to this Engine. Incompatible bindings are rejected before acceptance.

These checks validate reported bindings, not complete contract identity, token decimals, or arbitrary ERC-20 behavior; a malicious implementation can lie about its getters.

Fix:
https://github.com/Plether-Fi/plether-core/commit/97674a9197ad9469b334a9e1ae8541e5acbbf0f3

Locations:

```
packages/perps/src/CfdEngine.sol:226-230
packages/perps/src/CfdEngine.sol:276-319
packages/perps/src/CfdEngine.sol:366-376
packages/perps/src/CfdEngine.sol:1391-1395
packages/perps/src/HousePool.sol:581-590
packages/perps/src/MarginClearinghouse.sol:978-986
packages/perps/src/MarginClearinghouse.sol:997-1007
packages/perps/src/MarginClearinghouse.sol:1076-1110
```

A one-time deployment mistake can break the protocol's cash plumbing permanently. `CfdEngine` stores its own `USDC` token in the constructor and later accepts any nonzero `HousePool` address in `setPool`, but it never verifies that the clearinghouse custody token and the pool token are the same asset or that the pool is actually bound to this engine. Normal settlement paths then assume those three contracts share one settlement token. For example, trader-claim and rebate flows call `pool.payOut(clearinghouse, amount)` and immediately credit internal clearinghouse balances with `settleUsdc` or `creditPnlPledge`; if `HousePool.USDC()` differs from `MarginClearinghouse.settlementAsset()`, the clearinghouse books withdrawable balances without receiving the token it pays out on withdrawal. The opposite direction is also broken: carry collection makes the clearinghouse transfer its settlement token to the engine, then the engine forwards its own `USDC` to the pool, so a clearinghouse/engine token mismatch makes carry realization revert and blocks deposits, withdrawals, order settlement, and bounty credits that checkpoint carry.

Preconditions

- `CfdEngine` is deployed with `_usdc` that does not match `MarginClearinghouse.settlementAsset()`, or `setPool()` is called with a pool whose `USDC`/`ENGINE` binding does not match this engine (`packages/perps/src/CfdEngine.sol:276-319`, `packages/perps/src/CfdEngine.sol:366-376`).
- Any ordinary protocol action reaches a settlement path that moves cash between the pool, engine, and clearinghouse: claim settlement, action rebate payout, open-order rebate payout, or carry realization (`packages/perps/src/CfdEngine.sol:226-230`, `packages/perps/src/CfdEngine.sol:1391-1395`).
- The deployment is immutable, so the bad wiring cannot be corrected in place (documented fresh-stack / no-upgrade model).

Impact

Depending on which pair is mismatched, traders either receive clearinghouse credits that are not backed by the withdrawal token, or every carry/payout path starts reverting. That can leave the clearinghouse insolvent, strand the wrong token inside protocol contracts with no recovery path, and permanently lock trader and LP funds behind an unfixable deployment error.

Remediation

Validate settlement-asset coherence during wiring. In the engine constructor require `_usdc == MarginClearinghouse(_clearinghouse).settlementAsset()`, and in `setPool()` require the candidate pool to report the same `USDC` token and this engine as its immutable `ENGINE` binding before accepting it.

Medium ACKNOWLEDGED

Anyone can block governance from rotating the protocol treasury

ID: MED-3

Comment

MED-3 — Addressed. setProtocolTreasury() no longer requires the previous treasury's clearinghouse balance to be zero. A third-party deposit or dust balance therefore cannot block rotation. Only future protocol-fee credits use the new treasury; existing credit remains owned and withdrawable by the old account.

Rotation intentionally does not migrate historical funds or recover balances from an inaccessible old treasury.

Fix:
https://github.com/Plether-Fi/plether-core/commit/67f5bad4a7ff165de52eefc54dffdb5ba4e0eae4

Locations:

`packages/perps/src/CfdEngine.sol:418-437`

`packages/perps/src/MarginClearinghouse.sol:364-445`

`packages/perps/src/MarginClearinghouse.sol:1267-1287`

Any user can stop governance from changing the protocol fee recipient by sending a dust deposit to the current treasury account. `MarginClearinghouse.depositFor()` lets an arbitrary caller credit settlement balance to any nonzero account, including `CfdEngine.protocolTreasury()`. `CfdEngine.setProtocolTreasury()` then refuses to install a new treasury while `clearinghouse.balanceUsdc(currentTreasury) != 0`, and the only normal withdrawal paths require the credited treasury address itself to call `withdraw` or `withdrawMargin`.

For example, after the old treasury withdraws to prepare a rotation, Alice calls `depositFor(oldTreasury, 1)`. The owner's `setProtocolTreasury(newTreasury)` call reverts with `CfdEngine__ProtocolTreasuryBalanceNotEmpty()`. Alice can repeat this around every retry; if the old treasury key is lost, compromised, or a contract that cannot withdraw from the clearinghouse, governance has no on-chain way to clear the forced balance, so future protocol fees and forfeited bounties stay routed to the old treasury.

Preconditions

- The owner calls `setProtocolTreasury()` with a nonzero address that cannot later call `MarginClearinghouse.withdraw`, because it is the wrong address or a contract with no withdrawal entrypoint (`packages/perps/src/CfdEngine.sol:422-436`, `packages/perps/src/MarginClearinghouse.sol:408-445`).

- At least one order or liquidation credits protocol fees to `protocolTreasury`, either from trader cash or from a pool-funded top-up (`packages/perps/src/MarginClearinghouse.sol:1267-1287`; protocol fee credits are used throughout engine settlement).
- The old treasury account then has a nonzero clearinghouse balance, which makes later `setProtocolTreasury()` calls revert (`packages/perps/src/CfdEngine.sol:432-434`).

Impact

Protocol fee assets become permanently stuck inside the clearinghouse under an unreachable account. The treasury cannot withdraw them and governance cannot rotate to a usable treasury until that unreachable balance is somehow emptied, which the code does not provide.

Remediation

Require `protocolTreasury` to be a known withdraw-capable account type before accepting it, or add an owner-governed migration path that can transfer an old treasury balance to a new treasury during rotation.

Medium ACKNOWLEDGED

Attacker can make later traders miss execution windows by filling the FIFO with stale orders

ID: MED-4

Comment

MED-4 — Addressed with bounded permissionless cleanup. clearInvalidOrders(uint64[]) can remove expired or configuration-invalid orders anywhere in the global queue, including behind a live head, without oracle updates. Each item uses the existing reservation, refund, bounty, protection and receipt rules. Ineligible or already terminal IDs are skipped; an item failure stops the call while preserving earlier completed cleanup. The list is capped at min(64, maxPruneOrdersPerCall). Global FIFO execution is retained. Cleanup still costs gas, may require multiple calls, and does not prevent fresh invalid submissions or guarantee a profitable keeper incentive. Fix: https://github.com/Plether-Fi/plether-core/commit/b90670eb8e65a988b86c61e5e2a02ffe18129d20

Locations:

`packages/perps/src/OrderRouterExecutionSidecar.sol:148-188`

`packages/perps/src/OrderRouterExecutionSidecar.sol:247-285`

`packages/perps/src/OrderRouterExecutionSidecar.sol:883-905`

`packages/perps/src/router/OrderCommitHandler.sol:56-57`

`packages/perps/src/OrderRouterAdmin.sol:27-28`

`packages/perps/src/OrderRouterAdmin.sol:366-387`

A permissionless trader can force later orders to spend multiple transactions on cleanup before they are even eligible for oracle execution. `executeOrder()` and `executeOrderBatch()` classify pre-oracle heads as `RiskOff`, `Expire`, or `ConfigMismatch`, but they stop once they hit the fixed cleanup caps and return `PendingReason.CleanupLimit` instead of continuing to the requested later order. For ordinary terminal heads the cap is `maxPruneOrdersPerCall`, which defaults to 64 and is globally limited to 256. There is no global queue bound, only a 32-order cap per account, so an attacker can spread stale orders across many accounts and build an arbitrarily long expired/config-mismatched prefix.

In practice the attacker can open many minimal positions, queue zero-bounty close orders, let them expire, then allow other users to commit behind that expired prefix. When a keeper tries to execute a later victim order, the first call only prunes up to the cap and returns without touching the victim order or oracle path. The keeper must send more transactions just to reach the live order. With the deployed default 60 second execution window and 64-order prune cap, a backlog of a few thousand stale heads is enough to require dozens of extra transactions, which can make later orders expire unexecuted. `expireOrder(orderId)` does not remove this griefing vector because it clears only one expired order per transaction, so the attacker still converts one victim execution into linear keeper work.

Preconditions

- The router only prunes a bounded number of pre-oracle terminal heads per call (-`MAX_RISK_OFF_REFUNDS_PER_CALL = 64` and `maxPruneOrdersPerCall` capped at 256) in `OrderRouterExecutionSidecar.sol:148-188` and `247-285`.
- The attacker can commit many orders because the only admission limit is per-account (`pendingOrderCounts[account] <= maxPendingOrders`, with `maxPendingOrders <= 32`) in `OrderCommitHandler.sol:56-57` and `OrderRouterAdmin.sol:366-373`.
- Some attacker-controlled prefix orders become pre-oracle terminal before a victim order reaches the head, for example by expiring after commit or by being invalidated by a later config change recognized in `OrderRouterExecutionSidecar.sol:897-903`.

Impact

Later users can lose order liveness. Their orders remain stuck behind attacker-created stale heads until keepers spend extra transactions on cleanup, and if the backlog is large enough those later orders can miss their execution windows and expire without ever reaching the oracle or engine path.

Remediation

Add a permissionless multi-order stale cleanup path that can remove arbitrary expired or config-mismatched orders in one transaction without requiring the victim execution call to walk the prefix under a small fixed cap.

Medium ACKNOWLEDGED

Final refund claimant can lock redemption refunds for bootstrap LPs

ID: MED-5

Comment

MED-5 — Addressed. Both wallet-share and direct claimable-deposit redemption requests now require that tranche's seed receiver to be configured. The shared enqueue guard prevents an unseeded redemption epoch from reaching a terminal refund that needs a receiver for share dust; both maximum-request views return zero before seeding. The guard is tranche-local and does not wait for the other tranche's seed or trading activation. Pre-seed asset assignment remains supported. This prevents new affected requests rather than changing the dust allocation policy.

Fix:
https://github.com/Plether-Fi/plether-core/commit/efcf629aed56cbbf11b8cf5a717b89eb66a5fa2f

Locations:

`packages/perps/src/HousePool.sol:630-634`

`packages/perps/src/HousePool.sol:678-680`

`packages/perps/src/TrancheVault.sol:899-929`

`packages/perps/src/TrancheVault.sol:1248-1263`

`packages/perps/src/TrancheVault.sol:1527-1549`

A tranche that has bootstrap shares but no configured seed receiver can deadlock its terminal redeem-refund path. `HousePool.assignUnassignedAssets()` mints tranche shares through `bootstrapMint()` but explicitly does not initialize the seed lifecycle, so `TrancheVault.seedReceiver` can remain zero even while users hold redeemable shares. Later, if a redeem epoch is rejected into `refundEnabled` and floor rounding leaves refundable share dust, the final `claimRedeemRefund()` call reaches `_sweepRedeemRefundDust()`, which unconditionally reverts with `InvalidSeedPosition` when `seedReceiver == address(0)`.

A concrete trace is: governance assigns unassigned junior assets to Alice before any junior seed is initialized, Alice transfers some of those shares to Bob after cooldown, and both redeem into the same epoch. If settlement rejects the remainder with `refundableShares = 2` against `epoch.shares = 3`, Bob's pro-rata refund is 1 share and 1 share remains as dust. When Bob makes the final `claimRedeemRefund()` call, `_sweepRedeemRefundDust()` runs before the transfer, sees `seedReceiver == 0`, and reverts the whole claim. Bob cannot receive his 1 owed share, Alice's prior acknowledgement is rolled back from a system-wide completion perspective, and the epoch stays stuck until governance later initializes a seed receiver. If governance never seeds that tranche, the refund path is permanently blocked.

Preconditions

- Governance used `assignUnassignedAssets()` to mint tranche shares before any `initializeSeedPosition()` configured a `seedReceiver`, so the vault has live user shares while `seedReceiver == address(0)` (`packages/perps/src/HousePool.sol:630-634`, `packages/perps/src/TrancheVault.sol:1527-1549`).
- At least two controllers participate in the same redeem epoch and settlement converts the head into the refundable path, leaving positive refund dust after pro-rata floor rounding (`packages/perps/src/TrancheVault.sol:1483-1501`, `packages/perps/src/TrancheVault.sol:1248-1263`).
- A controller makes the final `claimRedeemRefund()` call that causes `epoch.refundBasisClaimed == epoch.shares` (`packages/perps/src/TrancheVault.sol:899-929`).

Impact

The last refund claimant cannot recover their shares, and the epoch's remaining refundable shares stay trapped in vault escrow until owner intervention configures a seed receiver. In the affected bootstrap configuration, LP redemption refunds can be blocked indefinitely and become permanently stuck if the tranche is never seeded.

Remediation

Do not let `_sweepRedeemRefundDust()` depend on a seed receiver that may not exist. Either configure a dust receiver during `assignUnassignedAssets()` or change the refund-dust sweep to use a dedicated always-set sink instead of reverting when the tranche has bootstrap shares but no seed position.

Medium ACKNOWLEDGED

Protocol treasury can spend LP withdrawal reserves on fee top-ups

ID: MED-6

> **Comment**
> MED-6 — Addressed. Protocol-fee top-ups are capped by HousePool.getSettlementFreeUSDC(), which protects directional liabilities, the settlement buffer, trader claims, pending recapitalization/revenue, and unassigned assets. The view uses settlement-safe inputs and the same reserve arithmetic as LP withdrawal accounting. Close previews project the corresponding post-close reserves, including newly queued revenue, so preview and live fee collection remain consistent. Unfunded top-ups do not create a new treasury claim; trader payout and rebate priority is unchanged.
>
> Fix:
> https://github.com/Plether-Fi/plether-core/commit/6f05ea8987e33881f0df859bf48c55a1f5928d14

Locations:

`packages/perps/src/CfdEngineSettlementSidecar.sol:887-909`

`packages/perps/src/libraries/CashPriorityLib.sol:23-31`

`packages/perps/src/HousePool.sol:1210-1215`

`packages/perps/src/HousePool.sol:1700-1707`

`packages/perps/src/HousePool.sol:581-589`

Protocol-fee top-ups can pull USDC from HousePool balances that the pool itself still treats as reserved. `_settleProtocolFeeTopUp()` computes available cash with `reserveFreshPayouts(pool.totalAssets(), totalTraderClaimBalanceUsdc)`, which subtracts only outstanding trader claims. It does not subtract the reserves that HousePool withdrawal accounting protects: max-liability headroom, supplemental reserve, pending claimant buckets, and `unassignedAssets`. For example, if HousePool holds 1,000 USDC with 200 USDC of pending claimant inflows and 300 USDC of unassigned assets, HousePool withdrawal accounting exposes only 500 USDC as free. A skew-reducing open or undercollateralized close that leaves a 700 USDC execution-fee shortfall will still let `_settleProtocolFeeTopUp()` pay 700 USDC to treasury because trader claims are zero. `HousePool.payOut()` reduces `accountedAssets` immediately, so the treasury margin is created by consuming assets that were supposed to remain reserved for LP withdrawals or later bootstrap assignment. The later HousePool flows then discover a cash shortfall even though the funds left through a normal settlement path.

Preconditions

- A settlement path needs a pool-funded protocol-fee shortfall, so `_settleProtocolFeeTopUp` is reached with `amountUsdc > clearinghouseCreditedUsdc` (`packages/perps/src/CfdEngineSettlementSidecar.sol:887-899`).

- HousePool currently has non-trader-claim reservations such as supplemental liability reserve, pending claimant inflows, or `unassignedAssets`, which HousePool withdrawal accounting subtracts from free cash (`packages/perps/src/HousePool.sol:1210-1215`, `packages/perps/src/HousePool.sol:1700-1707`).
- The deployment uses the documented treasury-margin model where treasury balances must not consume HousePool cash reserved for LP withdrawals (`packages/perps/ACCOUNTING_SPEC.md:759-760`, `packages/perps/ACCOUNTING_SPEC.md:999`).

Impact

Normal trade settlement can convert LP-reserved pool cash into withdrawable treasury margin. That misroutes custody, leaving LP redemptions, pending claimant distribution, or unassigned-asset assignment underfunded later.

Remediation

Base protocol-fee top-ups on HousePool's full withdrawal-reserve view, not just `totalAssets - traderClaims`; for example, cap top-ups by the pool's canonical free-withdrawable cash after all HousePool reservations.

Medium ACKNOWLEDGED

Keeper can settle traders and LPs on selectively updated basket components

ID: MED-7

Comment

MED-7 — Partially mitigated; residual limitation acknowledged.

Live baskets are built from complete signed payloads rather than mutable Pyth storage. Every component must satisfy the freshness window, and accepted component timestamps cannot move backward.

A caller can still choose an older valid signed component within the allowed window. The contract cannot prove that a newer report exists off-chain. Requiring simultaneous feed timestamps would impair availability, while tighter windows reduce but do not eliminate this choice. A sole keeper already has execution-timing discretion.

Different accepted marks were demonstrated, but no wrongful liquidation, different LP payout, or profitable exploit from this additional choice was established. The residual component-selection risk is accepted.

Mitigations:
https://github.com/Plether-Fi/plether-core/commit/2fca7941
https://github.com/Plether-Fi/plether-core/commit/7de2f742
Acknowledgment:
https://github.com/Plether-Fi/plether-core/commit/c23add5a
Documented in packages/perps/SECURITY.md under 'Known limitation: live basket component selection'.

Locations:

`packages/perps/src/PletherOracle.sol:489-497`

`packages/perps/src/PletherOracle.sol:621-639`

`packages/perps/src/PletherOracle.sol:641-691`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:534-536`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:575-589`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:678-711`

A permissionless caller can execute current-basket paths against a basket assembled from only the feeds they chose to refresh. `PletherOracle` first forwards arbitrary `pythUpdateData` to `pyth.updatePriceFeeds`, then immediately reads every configured component from Pyth storage with `getPriceUnsafe`. It never checks that the submitted payload actually advanced every configured feed, or even advanced any specific feed. As long as each untouched component is still within the mode's age limit and the oldest/newest component timestamps remain inside the configured divergence bound, `_computeBasketPriceFromFeeds` accepts the mixed basket. A keeper can therefore submit updates for only the basket legs that move execution in their favor, leave the other legs at older stored values, and still obtain a valid mark for liquidation, frozen mark refresh, frozen execution, or LP epoch settlement. For example, in a 50/50 two-feed basket, if feed A is refreshed from 1.00 to 1.20 while feed B is intentionally left at an older stored 1.00 instead of its current 0.80, the accepted basket becomes 1.10 even though the coherent current basket is 1.00.

Preconditions

- The caller uses a current-basket path that reads Pyth storage after `updatePriceFeeds`, such as pool reconcile, mark refresh, frozen execution, or liquidation (`packages/perps/src/PletherOracle.sol:509-514`, `packages/perps/src/PletherOracle.sol:443-453-`, `packages/perps/src/PletherOracle.sol:432-440`).
- A permissionless keeper supplies `pythUpdateData` that advances only a favorable subset of the configured feeds, while the remaining feeds stay in storage with publish times still inside the configured age and divergence limits (`packages/perps/src/PletherOracle.sol:489-497`, `packages/perps/src/PletherOracle.sol:621-691`).
- The deployment keeps nonzero staleness and divergence windows, such as the defaults `orderExecutionStalenessLimit = 60` and `liquidationStalenessLimit = 15` (`packages/perps/src/PletherOracle.sol:64-69`).

Impact

Liquidations, frozen closes, position-protection triggers, mark refreshes, and LP epoch settlement can all execute on a basket price that is not a coherent market snapshot. This lets permissionless executors skew settlement toward favorable component timing and transfer value between traders, keepers, and LPs without needing to compromise Pyth itself.

Remediation

After `updatePriceFeeds`, require every configured feed to have advanced to at least a caller-independent target publish time before using a current basket, or switch these paths to a historical basket primitive that proves a coherent timestamp set.

Medium ACKNOWLEDGED

Trader can be blocked from triggering protection because later L2 blocks reuse the same block number

ID: MED-8

Comment

MED-8 — Addressed. Protection arming/trigger checks and ordinary order commit/execution checks now use the execution-chain block number. On Arbitrum One, Nova and Sepolia this comes from ArbSys.arbBlockNumber(), so both gates advance with L2 blocks even while the L1 block number is unchanged. Same-L2-block triggering and execution remain prohibited; oracle publication-time checks remain independent. A missing ArbSys response on those chains reverts rather than falling back to the L1 clock.

Fix:
https://github.com/Plether-Fi/plether-core/commit/8a2853f80bce067656c1fd0fcf2dadc5892e2aee

Locations:

`packages/perps/src/PositionProtectionBook.sol:465-467`

`packages/perps/src/PositionProtectionBook.sol:960-965`

`packages/perps/src/interfaces/PositionProtectionTypes.sol:14-15`

On the deployed Arbitrum stack, a protection that should become triggerable on a later block can remain blocked across multiple later L2 blocks because the contract snapshots and compares `block.number`. When a protection arms, `_arm()` stores `armedBlock = uint64(block.number)` (`PositionProtectionBook.sol:960-965`), and `activate()` later reverts unless `block.number > armedBlock` (`PositionProtectionBook.sol:465-467`). The documented requirement is only a valid later-block oracle publication (`PositionProtectionTypes.sol:14-15`), but Arbitrum `block.number` reflects the ancestor chain context rather than the native L2 block height, so several consecutive L2 blocks can share the same value. On the protocol's stated deployment chain, an honestly later protection-trigger transaction can therefore be misclassified as the same block and rejected until the ancestor block advances.

Preconditions

- A protection is armed and stores the current `block.number` into `armedBlock` (`packages/perps/src/PositionProtectionBook.sol:960-965`).
- Any keeper or user calls `triggerPositionProtection` in a later Arbitrum L2 block whose ancestor-chain `block.number` is unchanged, so `block.number <= armedBlock` still holds (`packages/perps/src/PositionProtectionBook.sol:313-321`, `packages/perps/src/PositionProtectionBook.sol:465-467`).

- The deployment uses Arbitrum semantics where `block.number` does not monotonically identify each L2 block (`.guard/research/dependencies.md:7-9`).

Impact

Take-profit and stop-loss triggers can stay unavailable for extra L2 blocks on the live deployment target even though the oracle publication and the trigger transaction are later in real execution order. That delays protective closes precisely during fast markets, when traders expect the later-block guard to be minimal.

Remediation

Do not use `block.number` as the later-block discriminator on Arbitrum. Compare only an oracle-publish-time floor that matches the intended rule, or store and compare an execution-order signal that actually advances every L2 block on the target chain.

Medium ACKNOWLEDGED

Keepers can retry protection closes without the trader's financial limits

ID: MED-9

Comment

MED-9 — Addressed. The protection retains trader-selected ceilings for execution notional, gross account debit, action charges and explicit fees, and copies them into both the initial close and every permissionless retry. Only the owner may amend the retained ceilings while protection is Latched; keepers cannot relax them. Limits use six-decimal USDC: zero means zero allowance and uint256.max explicitly opts into unlimited allowance. A triggered close remains a market close, so these financial ceilings do not guarantee a fill at the trigger price. Tight limits may prevent execution until conditions improve or the owner amends them.

Fix:
https://github.com/Plether-Fi/plether-core/commit/8a2853f80bce067656c1fd0fcf2dadc5892e2aee

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:178-192`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:580-620`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:623-645`

`packages/perps/src/OrderLifecycleBook.sol:377-389`

`packages/perps/README.md:118-121`

A permissionless keeper can trigger or retry a protection close that has no effective financial caps, so the protected trader can be closed after carry, fees, or frozen spread exceed the limits they could set on an ordinary close. The router constructs these close attempts internally in _protectionCloseRequest() with expectedConfigHash set to zero and the financial maxima set to type(uint256).max, then registers the request as a normal pending V3 intent. The normal policy evaluator later enforces the stored bounds, but those bounds are maxed out for protection attempts. Since PositionProtectionBook only lets the trader choose take-profit and stop-loss trigger prices, neither the initial trigger close nor a later permissionless retry can carry trader-selected caps for gross account debit, action charge, explicit fees, or execution notional.

Preconditions

- An account has an armed or latched position protection, so the router can create an internal trigger close or retry close through `_protectionCloseRequest()` (packages/perps/src/router/OrderLiquidationBatchLogic.sol:178-192; 580-620; 623-645).
- A keeper or other external caller triggers the protection or calls `retryPositionProtectionClose`, and execution happens later under worse carry, fee, or frozen-spread conditions than when the user originally set the protection.

Impact

A protection close can consume arbitrarily more fees and gross account debit than the trader would be allowed to authorize through a normal bounded V3 order. This can turn a supposed safety mechanism into an unbounded forced close that drains extra settlement or settles under materially different economics than the user consented to.

Remediation

Encode explicit financial bounds into internally generated protection-close requests, preferably by snapshotting the user's chosen limits when the protection is created and reusing them for trigger and retry orders.

Medium ACKNOWLEDGED

Liquidator can liquidate short traders at prices below the configured cap-adjusted mark

ID: MED-10

Comment

MED-10 — Addressed. Single-account and batch liquidation pricing now apply the directional confidence adjustment to the raw, uncapped basket, then clamp the resulting liquidation price to CAP. The neutral Engine mark remains capped separately. This removes the artificial below-cap SHORT liquidation price caused by subtracting confidence from an already capped mark. Confidence adjustments remain part of liquidation pricing; the fix changes their ordering, not the policy of using adverse prices.

Fix:
https://github.com/Plether-Fi/plether-core/commit/006c818d364282943e2063eab52ab0c89181304c

Locations:

`packages/perps/src/PletherOracle.sol:275-283`

`packages/perps/src/PletherOracle.sol:305-307`

`packages/perps/src/PletherOracle.sol:694-705`

`packages/perps/src/PletherOracle.sol:742-770`

Short positions can be liquidated at a lower price than the oracle policy defines when the neutral basket is above `CAP_PRICE`. `updateLiquidationPrice` and `updateLiquidationBatchPrice` first clamp the neutral basket inside `_snapshotFromBasket`, then apply the adverse confidence shift to that already-capped number. The documented liquidation rule is the opposite order: shift the neutral basket by the confidence interval and then cap at `CAP_PRICE`. With `CAP_PRICE = 200_000_000`, raw basket `200_020_000`, aggregate confidence `200_000`, and `adverseConfidenceMultiplierBps = 2_500`, the intended SHORT liquidation price is `min(200_020_000 - 50_000, 200_000_000) = 199_970_000`, but the implementation returns `200_000_000 - 50_000 = 199_950_000`. The engine uses that lower price directly for liquidation planning, so short traders near the boundary can be liquidated earlier than intended or lose extra value during liquidation settlement.

Preconditions

- The neutral basket computed from Pyth feeds is greater than `engine.CAP_PRICE()` (`packages/perps/src/PletherOracle.sol:703-704-`, `packages/perps/src/PletherOracle.sol:798-799`).
- The liquidated account has a nonzero SHORT position, and a keeper or liquidator calls the permissionless liquidation path that consumes `updateLiquidationPrice` or `updateLiquidationBatchPrice`.

- Impact scales with `CAP_PRICE` and the configured `adverseConfidenceMultiplierBps` (`packages/perps/src/PletherOracle.sol:798-799`, `packages/perps/src/PletherOracle.sol:759-762`).

Impact

Short accounts can be falsely liquidated or settled at harsher-than-specified liquidation prices whenever the neutral basket is above the cap. The error is small per lot under default settings but grows with position size and can change liquidation eligibility at the threshold.

Remediation

Apply the liquidation confidence shift to the pre-cap basket value and clamp only once afterwards. In practice, compute adverse liquidation prices from `basket.price`, keep `markPrice` as the neutral capped value, and avoid feeding `snapshot.price` back into the liquidation shift helpers.

Medium ACKNOWLEDGED

Senior redeemer can leave remaining senior principal unprotected

ID: MED-11

Comment

MED-11 — Addressed. Senior withdrawals now reduce the high-water mark by the same economic fraction as principal, using the actual net USDC funded: P' = P - A and H' = H - floor(H * A / P). Retained exit fees therefore remain covered by Senior recovery protection. Healthy withdrawals preserve H' = P'; impaired withdrawals preserve proportional protection with conservative rounding. Zero funding leaves protection unchanged and full principal funding clears it. The remaining impaired-rounding behavior is accepted under LOW-54.

Fix:
https://github.com/Plether-Fi/plether-core/commit/9ab65f4175cb90805eabb434908ab4832a124d56

Locations:

`packages/perps/src/libraries/HousePoolWaterfallAccountingLib.sol:121-131`

`packages/perps/src/HousePool.sol:833-840`

`packages/perps/src/TrancheVault.sol:1832-1845`

A senior redemption can leave part of the remaining senior principal outside the high-water-mark protection, so later losses can cause future recovery revenue to be routed to junior too early. `HousePool._settleLpEpoch()` funds a senior redemption using the ERC-4626 virtual-offset quote, then `scaleSeniorOnWithdraw()` reduces `seniorHighWaterMark` by `fundedShares / preBurnSupply` while reducing `seniorPrincipal` by the funded USDC amount. Those are different economic fractions whenever senior principal has moved without proportional share issuance, such as after junior-paid senior coupon or retained exit-fee value. For example, if seniorPrincipal and seniorHighWaterMark are 2,000 USDC while the senior vault still has 1,000,000 shares from an original 1,000 USDC deposit, funding a 500,000-share redemption pays 999 USDC but reduces the high-water mark by 1,000 USDC. The remaining senior holders have 1,001 USDC of principal but only a 1,000 USDC restoration target. If the pool then takes a loss before a positive-revenue reconcile ratchets the high-water mark back to current principal, later revenue restores senior only to the understated target and the excess can accrue to junior.

Preconditions

- A senior redemption is funded through `HousePool._settleLpEpoch()` so `scaleSeniorOnWithdraw()` runs with `fundedAssets` taken from the vault redemption quote and `fundedShares` burned from the senior vault (`packages/perps/src/HousePool.sol:821-840`).

- The senior vault uses its live ERC-4626 conversion with `VIRTUAL_SHARES = 1000`, so redeemed assets are not proportional to `fundedShares / preBurnSupply` when supply is not very large (`packages/perps/src/TrancheVault.sol:1832-1845`).
- Before a later positive-revenue reconcile path can ratchet `seniorHighWaterMark` back up, the pool experiences a mark-fresh reconcile with zero or negative delta, which preserves the understated watermark (`packages/perps/src/libraries/HousePoolWaterfallAccountingLib.sol:141-160`, `packages/perps/src/libraries/HousePoolWaterfallAccountingLib.sol:168-185`).

Impact

Remaining senior LPs can lose restoration priority after another loss event. Revenue that should first refill their unimpaired entitlement can be diverted to junior LPs, and senior-protection checks that rely on `max(seniorPrincipal, seniorHighWaterMark)` can temporarily understate protected senior exposure.

Remediation

Reduce `seniorHighWaterMark` by the same economic fraction removed from senior principal, not by raw burned-share ratio. Using the funded-asset debit against pre-withdraw principal, or otherwise mirroring the vault's virtual-share pricing when scaling HWM, keeps remaining senior entitlement aligned with remaining ownership.

Medium ACKNOWLEDGED

Mixed recovery inflows can strand LP revenue in ownerless pool assets

ID: MED-12

Comment

MED-12 — Addressed. The shared pending-accounting helper recomputes revenue continuation after applying recapitalization. When recapitalization restores principal and LP shares survive, revenue in the same batch follows the restored claimant ownership instead of being quarantined as unassigned. The reported 40,000 USDC recapitalization plus 15,000 USDC revenue case now produces senior principal of 40,000, junior principal of 15,000, and zero unassigned assets. Preview and live reconciliation use the same helper. Recapitalization remains senior-first; partial-settlement residuals stay reserved, and surplus without eligible owners remains unassigned.

Fix:
https://github.com/Plether-Fi/plether-core/commit/f852f969352f14406bef582d52cab4b024b32db7

Locations:

`packages/perps/src/HousePool.sol:1521-1552`

`packages/perps/src/HousePool.sol:1577-1628`

`packages/perps/src/libraries/HousePoolPendingPreviewLib.sol:40-53`

`packages/perps/src/libraries/HousePoolPendingPreviewLib.sol:154-183`

Already-funded LP revenue can lose its claimant path during wipeout recovery and get parked in `unassignedAssets` even when seeded LP ownership still exists. The pending-accounting flow decides `allowRevenueContinuation` from the pre-recapitalization principal state (`seniorPrincipal + juniorPrincipal != 0`), then applies recapitalization before revenue. After a wipeout, storage principal can be zero while Senior and Junior share supply still exist. In that state, a batch containing both recapitalization and revenue is processed as follows: `applyClaimantRecapitalizationIntent()` restores senior principal first, but `applyRevenueIntent(..., false)` then sees nonzero claimed principal and sends the entire revenue amount to `unassignedAssets` instead of continuing through the surviving claimant waterfall. For example, after a total loss leaves `seniorPrincipal = 0`, `juniorPrincipal = 0`, `seniorSupply > 0`, and `juniorSupply > 0`, settling a 40,000 USDC recapitalization plus 15,000 USDC of realized revenue produces `seniorPrincipal = 40,000`, `seniorHighWaterMark = 40,000`, and `unassignedAssets = 15,000`. That 15,000 USDC is then excluded from normal LP ownership, blocks new tranche deposits, and can reach LPs only through the owner-only `assignUnassignedAssets()` path. This contradicts the protocol's own claimant-continuity rule that value should use seeded ownership paths when available and only truly ownerless value should enter `unassignedAssets`.

Preconditions

- A fresh or settleable reconcile occurs while storage principal has been wiped to zero, but surviving tranche share supply remains (`packages/perps/src/HousePool.sol:1784-1788`).
- The engine records both recapitalization and revenue claimant inflows before the same pending-accounting pass (`packages/perps/src/HousePool.sol:601-628`, `packages/perps/src/HousePool.sol:1518-1525`).
- No prior principal exists when `allowRevenueContinuation` is computed from `seniorPrincipal + juniorPrincipal != 0` (`packages/perps/src/HousePool.sol:1521`, `packages/perps/src/HousePool.sol:1577`).

Impact

LP-owned revenue is misclassified as ownerless cash. The funds stop following normal senior/junior ownership, new deposits remain blocked because `unassignedAssets > 0`, and recovering the value requires an owner-only reassignment that can arbitrarily choose the receiver.

Remediation

Recompute the revenue-continuation decision after recapitalization has updated the pending waterfall, or make `applyPendingClaimantBucketsPreview()` treat a recapitalization-restored claimant state as eligible for immediate revenue continuation when surviving share supply exists.

Medium ACKNOWLEDGED

Governance can retarget armed stop-losses after oracle rotation

ID: MED-13

Comment

MED-13 — Accepted design; no code change. Armed take-profit/stop-loss protection intentionally retains its trader-selected thresholds across oracle rotation and evaluates them using the active protocol oracle. Rotating oracle infrastructure should not cancel protection or require traders to re-arm it.

This assumes the replacement oracle represents the same underlying market. Rotation validates wiring and preserves the configured policy, but does not prove semantic equivalence of the replacement oracle's feed composition, weights, inversions or normalization. A rotation that changes those semantics can change when existing thresholds trigger. That governance risk is accepted; automatic protection invalidation or binding to an oracle/configuration hash is not adopted.

Disposition: https://github.com/Plether-Fi/plether-core/pull/124

Locations:

`packages/perps/src/PositionProtectionBook.sol:130-145`

`packages/perps/src/PositionProtectionBook.sol:435-482`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:575-584`

`packages/perps/src/OrderLifecycleBook.sol:171-201`

`packages/perps/src/OrderRouterAdmin.sol:188-195`

Users can lose the meaning of an armed take-profit or stop-loss after governance rotates the router to a new oracle. `PositionProtectionBook` stores only the raw trigger prices, side, and size, then `triggerPositionProtection` asks the router's current `pletherOracle` for a fresh mark and calls `activate` with that mark. No protection record stores or checks the execution-config hash or oracle identity, even though `OrderLifecycleBook.currentExecutionConfigHash()` explicitly includes the oracle address. As a result, an already-armed protection that was configured against oracle A will keep using the same numeric thresholds after `OrderRouterAdmin.finalizeOracleConfig()` installs oracle B. If the new oracle changes feed ids, weights, inversions, base prices, or Pyth endpoint semantics, the old thresholds are now interpreted against a different market definition. The protection can trigger immediately, stop triggering entirely, or trigger only after a much larger move, and the user has no on-chain signal that their stored stop-loss was silently retargeted instead of being invalidated like an ordinary pinned order.

Preconditions

- A protection is already armed and later trigger evaluation reaches `activate`, which only checks status, timing, pending orders, and position equality before accepting the supplied mark price (`packages/perps/src/PositionProtectionBook.sol:452-482`).
- The owner finalizes an oracle rotation through `OrderRouterAdmin.finalizeOracleConfig()` before the trigger occurs (`packages/perps/src/OrderRouterAdmin.sol:188-195`).
- A permissionless keeper later calls `triggerPositionProtection`, causing the router sidecar to fetch a mark from the current oracle and pass it into `activate` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:575-584`).

Impact

Existing stop-loss and take-profit protections can fire against the wrong oracle or fail to fire when users expect. That breaks the integrity of protective orders and can leave traders exposed to avoidable losses or close them unexpectedly under a different market definition.

Remediation

Bind each protection to the current execution configuration when it is created or armed, and refuse to evaluate or keep it armed after the config hash changes. The simplest fix is to store `currentExecutionConfigHash()` on the protection and require it to match during activation, falling back to terminal failure or explicit re-arming after an oracle migration.

Medium ACKNOWLEDGED

Trader can be blocked from queueing an open behind their closing order

ID: MED-14

> **Comment**
> MED-14 — Addressed. Open admission projects the live position through earlier queued orders for the same account before checking the requested side. An opposing open can be queued after a projected full close; a partial close that leaves opposing exposure still blocks it. The live-position preview's MUST_CLOSE_OPPOSING result is ignored only after the queued-position side check passes. This is an admission projection, not a guarantee that the preceding close will execute: actual position, margin and risk checks run again at execution, so a failed earlier close can still make the later open fail. Fix: https://github.com/Plether-Fi/plether-core/commit/5029cde91b87d24865fc8d6c8683686989ef45ac

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-418`

`packages/perps/src/CfdEngineLens.sol:131-138`

`packages/perps/src/CfdEngineLens.sol:224-232`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:312-314`

A trader with an opposite-side live position cannot reliably queue the next open that should follow an earlier queued close. `OrderLiquidationBatchLogic._validatedOpenBounty()` asks `engineLens.previewOpenRevertCode()` to classify the new open using the current engine position only, then immediately rejects every `CommitTimeRejectable` code while the cached mark is fresh. The lens planner returns `MUST_CLOSE_OPPOSING` whenever the live position is on the other side, and it does not project the account's earlier queued close from the router queue. In a normal sequence where Alice is LONG, first commits a full LONG close, and then wants to queue a SHORT open behind it, the second commit is rejected as `OrderRouter__PredictableOpenInvalid` whenever `lastMarkTime` is fresh, even though the queue would be valid after the earlier close executes. The same router docs explicitly say valid close/open/close histories are supported, so this precheck contradicts the intended FIFO semantics and makes admission depend on mark freshness rather than queued exposure.

Preconditions

- The account has a live position on one side, and an earlier queued close would remove or reverse that exposure before the later open executes (`packages/perps/src/libraries/CfdEnginePlanLib.sol:312-314`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:361-379`).
- The trader submits a new open on the opposite side while the engine's cached mark is nonzero and still fresh under the router commit policy (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-418`).

- The deployment relies on the documented support for overlapping close/open/close queue histories (`packages/perps/README.md:1147-1148-`, `packages/perps/DEPOSIT_FREE_CLOSE.md:22-23`).

Impact

Routine multi-step trading sequences become unavailable during normal fresh-mark operation. Users cannot pre-queue the next open behind a closing order, so they must wait for the close to execute or for the cached mark to become stale, which breaks the documented FIFO workflow and can leave accounts unable to stage timely exposure changes.

Remediation

Base commit-time predictable-open rejection on the router's queued-position projection instead of the live engine position, or skip the `MUST_CLOSE_OPPOSING` pre-reject when earlier queued orders can remove the opposite-side exposure before this open reaches the head.

Medium ACKNOWLEDGED

Triggered SHORT protection can get stuck and leave a user unprotected

ID: MED-15

Comment

MED-15 — Addressed by preventing zero-price execution rather than treating a zero adjusted quote as a valid close. Confidence configuration is checked at proposal and application: maxConfidenceRatioBps <= 10,000, adverseMultiplierBps <= 30,000, and their product must be strictly below 100,000,000. Order execution and historical-cache reuse reject zero prices, including when the adverse adjustment exhausts the capped price. Rejection rolls back execution and preserves the pending protection attempt; expiry can relatch an unchanged position for retry. The accepted limitation is temporary execution unavailability when no positive valid quote exists. The fix does not fabricate a minimum price or guarantee execution under every accepted raw-basket configuration.

Fix:
https://github.com/Plether-Fi/plether-core/commit/44a2598e544b6b0d0a3b2737a6b8e822bcb0cd6f

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:625-645`

`packages/perps/src/OrderRouter.sol:428-439`

`packages/perps/src/libraries/OrderValidationLib.sol:64-69`

A triggered or retried SHORT take-profit or stop-loss can never execute at an oracle execution price of zero. The router synthesizes every protection close with `targetPrice = 1` for `side == SHORT` in `OrderLiquidationBatchLogic._protectionCloseRequest()` and `OrderRouter.executePositionProtectionTriggerItem()`, but the normal slippage rule for close SHORT orders is `executionPrice >= targetPrice`. The oracle price domain is `[0, CAP_PRICE]`, and `PletherOracle` can legitimately return `0` after an adverse downward confidence shift. In that state, every triggered SHORT protection attempt terminalizes as `Slippage`, `handleFailedProtectionAttempt()` relatches the protection, and each retry recreates the same `targetPrice = 1` order, so the protection loop cannot close the position until price/configuration changes.

Preconditions

- A position protection close is generated for a SHORT position, either on trigger or on retry, so the synthesized close request uses `targetPrice = 1` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:625-645`, `packages/perps/src/OrderRouter.sol:428-439`).
- A keeper or the trader triggers or retries the protection while the oracle produces `executionPrice == 0` for that close attempt. This is reachable because order execution prices are allowed to sit at the bottom of the documented `[0, CAP_PRICE]` domain and the adverse-confidence shift can round a downward-adjusted SHORT close price to zero.

- The configured adverse pricing policy permits a downward shift at least as large as the raw price, for example via `adverseConfidenceMultiplierBps` and `basketMaxConfidenceRatioBps` values that are both accepted by router admin validation (`packages/perps/src/OrderRouterAdmin.sol:336-344`).

Impact

The protection mechanism is unavailable exactly when the position needs to exit. The triggered close repeatedly fails with `Slippage`, the protection stays latched instead of executing, and the user remains exposed to further adverse price moves until someone changes conditions or closes manually.

Remediation

Use a true no-limit sentinel for generated SHORT closes. The simplest fix is to special-case protocol-generated protection attempts in slippage validation, or encode a sentinel that explicitly bypasses the lower-bound check instead of using `1`.

Medium ACKNOWLEDGED

Traders can enqueue invalid opens when the cached mark is stale

ID: MED-16

Comment
MED-16 — Addressed for price-independent opposing-side invalidity. Admission now checks the position projected through earlier queued account orders even when the cached mark is stale, rejecting an open if opposing exposure would remain. Numeric-domain rejection also remains unconditional. A projected full close can legitimately remove that exposure, so the live-position MUST_CLOSE_OPPOSING preview is not blindly enforced. Price-dependent checks remain freshness-gated and are revalidated at execution; this does not guarantee that every admitted order will fill. Stale nonzero-mark admission is deliberately retained, with its bounty limitation accepted under LOW-57. Fix: https://github.com/Plether-Fi/plether-core/commit/5029cde91b87d24865fc8d6c8683686989ef45ac

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-418`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:82-93`

`packages/perps/src/OrderRouterExecutionSidecar.sol:592-603`

A trader can commit an open order that the router can already prove is invalid whenever the engine's cached mark is stale. `_validatedOpenBounty()` calls `previewOpenRevertCode(...)`, but it only converts `CommitTimeRejectable` planner failures into `OrderRouter__PredictableOpenInvalid` while `engine.lastMarkTime()` is fresh under the order execution policy. Once the cached mark is stale, the router admits failures that do not require a fresh price to classify, such as opening against an existing opposite-side position, too-small positions, insufficient initial margin, or underfunded dedicated reserves. The order is added to the FIFO with an execution bounty reservation, and execution later re-runs the same planner checks and finalizes the order as `PlannerRejected`, paying the reserved bounty to the executor. This lets users accidentally burn bounty on impossible orders and lets attackers cheaply stuff the FIFO with underfunded opens that keepers must clear one by one.

Preconditions

- The engine's cached mark is stale at commit time, so `_validatedOpenBounty()` skips `OrderFailurePolicyLib.isPredictablyInvalidOpen(...)` rejection (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-418`).
- An unprivileged trader submits an open order whose invalidity comes from state or inputs that do not become valid just because a fresh oracle price later arrives, such as opposite-side exposure or unfundable reserves (`packages/perps/src/libraries/CfdEnginePlanLib.sol:292-315`, `packages/perps/src/libraries/CfdEnginePlanLib.sol:566-573`).

- Deployment keeps delayed FIFO order execution enabled and enforces the documented `maxPendingOrders = 5` queue cap per account (`.guard/research/protocol.md:42-43`).

Impact

Attackers can populate the queue with orders that are already known to be impossible, then external keepers must pay oracle fees and execution gas to clear them as terminal failures. This degrades queue liveness and keeper economics during stale-mark periods and can delay honest orders behind predictable junk.

Remediation

Do not gate all predictable-invalid screening on mark freshness. Continue rejecting stale-price-dependent categories when the mark is stale, but still reject freshness-independent `CommitTimeRejectable` codes such as size-quantum, opposing-side, and reserve-funding failures.

Medium ACKNOWLEDGED

Oracle lag can block traders from triggering protections

ID: MED-17

Comment

MED-17 — Addressed. Protections retain a separate armedPublishTime and compare trigger publication time against that oracle baseline, rather than the wall-clock armedAt timestamp. Creation and replacement use the accepted current oracle observation; protection attached to a successfully filled open records the Engine oracle timestamp at arming. A qualifying newer publication can therefore trigger even if it still predates the wall-clock arming time. Same-execution-block and oracle-freshness checks remain enforced. A publication that has not advanced beyond the recorded oracle baseline is still intentionally rejected.

Fix:
https://github.com/Plether-Fi/plether-core/commit/8f6cfd3d84d1289aa1209e3b08a06e9bfdf38a0d

Locations:

`packages/perps/src/PositionProtectionBook.sol:465-467`

`packages/perps/src/PositionProtectionBook.sol:797-809`

`packages/perps/src/PositionProtectionBook.sol:960-966`

An armed protection can stay untriggerable even after a later oracle update crosses its threshold, because the Book compares the new oracle publish time to `block.timestamp` from when the protection was armed instead of to the previously armed oracle time. `_arm()` stores `armedAt = block.timestamp`, but `_prepareActivation()` requires `publishTime > protection.armedAt`. Protection creation and replacement only require the cached engine mark to be fresh, not current, so a protection can be armed while `ENGINE.lastMarkTime()` still lags wall clock. For example, if the cached mark was published at `1000`, the sequencer timestamp is `1050`, and the trader arms protection then, a later oracle update at publish time `1005` that crosses the stop-loss still reverts with `SameBlockTrigger` because `1005 <= 1050`. The protection remains `Armed`, no close is queued, and the trader must wait for oracle publish time to catch up to chain time even though a strictly newer oracle tick already exists.

Preconditions

- The trader creates, replaces, or activates a parent-open protection while the cached engine mark is fresh enough for `_freshCachedProtectionMark()` but its `lastMarkTime` is behind current `block.timestamp` (`packages/perps/src/PositionProtectionBook.sol:797-809`).
- A later `triggerPositionProtection()` call supplies a valid mark-refresh snapshot whose `publishTime` is newer than the armed mark but still not greater than the stored `armedAt` wall-clock timestamp.

- The protected threshold is crossed during that interval, so the trader relies on the protection to queue the closing order.

Impact

Stop-loss and take-profit protections can be delayed for as long as oracle publish time trails wall clock. During that window the account stays exposed and can exit at a materially worse price or be liquidated before the protection becomes triggerable.

Remediation

Store the oracle publish time that the protection was armed against, and require the trigger snapshot to be newer than that oracle time instead of newer than `block.timestamp`.

Medium ACKNOWLEDGED

Keeper can rewrite the engine mark at the same timestamp and change liquidation and LP settlement prices

ID: MED-18

Comment

MED-18 — Addressed.

The Engine rejects replacing its cached mark with a different clamped price at the same oracle timestamp. The Oracle also enforces equal-time consistency against the Engine and rejects changed prices at the accepted live basket's existing timestamp.

Historical execution updates the cached mark only when its timestamp advances, so an older historical fill cannot rewind a newer mark.

Fixes:
https://github.com/Plether-Fi/plether-core/commit/2fca7941
https://github.com/Plether-Fi/plether-core/commit/7de2f742

Locations:

`packages/perps/src/CfdEngine.sol:1046-1056`

`packages/perps/src/CfdEngine.sol:1423-1431`

`packages/perps/src/OrderRouterExecutionSidecar.sol:1012-1019`

`packages/perps/src/PletherOracle.sol:651-691`

`packages/perps/src/PletherOracle.sol:694-710`

`packages/perps/src/HousePool.sol:773-795`

Any keeper who can call the router's permissionless mark-refresh, liquidation, or LP-settlement paths can replace `lastMarkPrice` with a different price that has the same `publishTime` as the cached mark. `PletherOracle` builds basket timestamps from the minimum component publish time, so two different validated basket compositions can legitimately share the same `publishTime` while producing different prices. The router accepts those snapshots because the oracle only rejects `publishTime < engine.lastMarkTime()`, the execution sidecar forwards marks when `publishTime >= engine.lastMarkTime()`, and `CfdEngine.updateMarkPrice()` rejects only strictly older times. Inside `_applyCaryAndMark()` an equal timestamp still overwrites `lastMarkPrice`. A keeper can therefore wait until one basket with publish time `T` sets the mark, then submit a second basket with the same `T` but a different basket value. The engine will keep `lastMarkTime = T` but replace `lastMarkPrice`, which then changes downstream behavior that trusts the cached mark, including `HousePool.settleLpEpoch()` authorization and terminal NAV accounting. This lets a permissionless keeper choose between multiple same-timestamp valid marks and force later liquidations, protection triggers, bounty checkpointing, or LP epoch settlement to use the more favorable one even though no newer oracle time has arrived.

Preconditions

- The engine already has a cached mark with publish time `T` (`packages/perps/src/CfdEngine.sol:1423-1431`).
- A permissionless keeper can obtain another oracle basket whose validated `publishTime` is also `T` but whose computed basket price differs because basket publish time is the minimum component time, not a unique basket identifier (`packages/perps/src/PletherOracle.sol:651-691`).
- The keeper can call any router path that forwards the new snapshot into `engine.updateMarkPrice(...)`, such as order execution, liquidation, mark refresh, or LP settlement preparation (`packages/perps/src/OrderRouterExecutionSidecar.sol:1012-1019`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:536-579`).

Impact

The cached engine mark is mutable within one oracle timestamp. A keeper can bias mark-dependent accounting and authorization toward the more favorable same-timestamp price, affecting liquidation eligibility, protection triggers, bounty recipient carry checkpointing, terminal NAV, and the exact mark HousePool accepts for LP epoch settlement.

Remediation

Treat equal-timestamp marks as immutable by rejecting `publishTime <= lastMarkTime` in router/oracle mark-update paths, or bind the cached mark to a stronger monotonic identifier than the minimum component publish time.

Medium ACKNOWLEDGED

Senior depositor can block others by cycling refundable reservations

ID: MED-19

Comment

MED-19 — Addressed by making an accepted Senior reservation a firm commitment. Healthy accepted deposits cannot be cancelled, and activation honors booked capacity despite subsequent coupon accrual or cap reductions. Pending Senior commitments also count in the Junior withdrawal backing covenant, preventing Junior funding from consuming their backing. Refunds remain available on epoch rejection, projected terminal wipe, Senior impairment, or an explicit LP settlement hold; the hold also closes new Senior admission. Accepted trade-offs: booked exposure may exceed a later reduced cap, and pending commitments can constrain Junior withdrawals.

Fix:
https://github.com/Plether-Fi/plether-core/commit/86e3c54fe57e65c7f1afd9c3c0e9c09cc9bc1a88

Locations:

`packages/perps/src/TrancheVault.sol:571-598`

`packages/perps/src/TrancheVault.sol:544-550`

`packages/perps/src/TrancheVault.sol:819-855`

`packages/perps/src/HousePool.sol:747-756`

Anyone with enough USDC can keep the senior tranche closed to later depositors by reserving the remaining senior capacity and cancelling before the request activates. `TrancheVault._requestDeposit()` calls `HousePool.reserveSeniorDeposit()` immediately for senior deposits, before the request is settled into active pool capital. `HousePool.reserveSeniorDeposit()` adds the amount to `reservedSeniorDepositAssetsUsdc`, and `HousePool.getSeniorDepositCapacity()` subtracts that reservation from the capacity returned through `TrancheVault.maxRequestDeposit()`. A request for the remaining headroom therefore makes later senior deposit requests fail with `ExceededMaxRequestDeposit` or `DepositsUnavailable`.

The reservation is not economically binding. Until the request epoch matures, `cancelPendingDeposit()` removes the queued position, calls `releaseSeniorDepositReservation()`, and transfers the same USDC back to the controller without a fee or slashing. By submitting during the five-minute cutoff window, the attacker can target the next eligible epoch, hold the capacity for about 60 to 65 minutes, then cancel and submit a new request in one transaction from an attacker-controlled contract. This blocks honest senior LP entry while the attacker avoids active senior tranche NAV exposure.

Preconditions

- The senior tranche still has requestable headroom, so `POOL.getSeniorDepositCapacity()` returns a positive amount that `maxRequestDeposit()` exposes to new callers (`packages/perps/src/TrancheVault.sol:544-550`, `packages/perps/src/HousePool.sol:750-756`).
- An unprivileged LP submits a senior request after the active epoch's cutoff, so `_requestEpochWindow()` assigns it to `currentEpoch + 2`, and honest senior LPs attempt to deposit before that delayed request is cancelled or settled (`packages/perps/src/TrancheVault.sol:544-550`, `packages/perps/src/TrancheVault.sol:571-598`).
- The attacker is willing to leave the reserved USDC in vault escrow until cancellation; cancelling the unmatured request returns the full amount and releases the reservation with no fee or slashing (`packages/perps/src/TrancheVault.sol:819-855`, `packages/perps/src/HousePool.sol:761-767`).

Impact

A single LP can create a repeating denial of service against senior-tranche admission. Victims cannot enter the senior vault during the blocked windows even though the attacker never contributes active senior capital.

Remediation

Do not consume global senior capacity at request time for post-cutoff requests. Reserve only the next matured epoch's capacity, or charge a non-refundable cancellation penalty that makes repeated capacity parking uneconomical.

Medium ACKNOWLEDGED

Frozen close orders can be blocked by live publish-time divergence limits

ID: MED-20

> **Comment**
> MED-20 — Addressed. Frozen close execution uses the frozen close policy's maxStaleness as the basket component publish-time divergence limit, rather than the live order limit. Single and batch closes can therefore use component reports that satisfy the frozen policy even when their timestamps differ beyond the live limit. Opens retain the live divergence rule. Accepted trade-off: a frozen basket can combine reports published at different times within the configured frozen allowance (three days with the current setting); it is not a simultaneous market snapshot. Component freshness and confidence validation remain enforced. Fix: https://github.com/Plether-Fi/plether-core/commit/cf0d750b8289c285628d56e0466c9773ff75d2a6

Locations:

`packages/perps/src/PletherOracle.sol:570-578`

`packages/perps/src/PletherOracle.sol:789-793`

`packages/perps/src/libraries/OracleFreshnessPolicyLib.sol:59-67`

Close orders in a frozen-oracle window can still fail because `PletherOracle` applies the live `orderExecutionStalenessLimit` to component publish-time divergence instead of the frozen `fadMaxStaleness` policy it uses for component age. `OracleFreshnessPolicyLib.getPolicy` correctly switches frozen order execution to `fadMaxStaleness` for age validation (`packages/perps/src/libraries/OracleFreshnessPolicyLib.sol:59-67`), but `_policyPublishTimeDivergence` special-cases only non-order-execution modes and otherwise falls back to `_maxPublishTimeDivergence`, which returns `orderExecutionStalenessLimit` for order execution (`packages/perps/src/PletherOracle.sol:570-578, 789-793`). As a result, a frozen close can reject a basket whose components are all individually fresh enough under frozen policy but differ by more than the live 60-second divergence bound. I confirmed this with a scratch Foundry test by forcing `engine.isOracleFrozen()` true and feeding two current prices at `t=1000` and `t=1061`: `updateOrderExecutionPrice` reverts with `PletherOracle__PublishTimeDivergence` even though both components are far within the default frozen 3-day staleness window.

Preconditions

- The engine is in an oracle-frozen window, so frozen close execution is expected to use `fadMaxStaleness` (`packages/perps/src/libraries/OracleFreshnessPolicyLib.sol:59-67`).

- A keeper executes a close order through `updateOrderExecutionPrice` or batch execution using current frozen prices whose per-component ages are all within `engine.fadMaxStaleness()` but whose publish times differ by more than `orderExecutionStalenessLimit` (`packages/perps/src/PletherOracle.sol:509-512, 570-578`).

Impact

Users can be unable to close during a frozen market even though the data satisfies the documented frozen staleness policy. This is a liveness failure on a protective path that the trust model explicitly intends to keep live during containment windows.

Remediation

Use `policy.maxStaleness` as the divergence limit for frozen order execution as well, or otherwise derive frozen-order divergence from `fadMaxStaleness` instead of `orderExecutionStalenessLimit`.

Medium ACKNOWLEDGED

Admin can rotate in a zero-weight feed that still bricks the oracle

ID: MED-21

Comment

MED-21 — Addressed.

Zero-weight basket components are rejected during Oracle construction. An unused feed must be removed from the basket instead of retained with zero weight, preventing an economically irrelevant feed from blocking oracle-backed operations.

Zero and duplicate feed IDs are also rejected. Basket validation requires nonzero component weights and base prices, with weights summing to 1e18.

Zero-weight fix: https://github.com/Plether-Fi/plether-core/commit/2c63f117f21abc-cac638497490d3a9e752514847
Feed-ID validation: https://github.com/Plether-Fi/plether-core/commit/2fca7941

Locations:

`packages/perps/src/PletherOracle.sol:88-92`

`packages/perps/src/PletherOracle.sol:128-137`

`packages/perps/src/PletherOracle.sol:621-689`

`packages/perps/src/router/OrderOracleExecution.sol:46-73`

A `PletherOracle` can be deployed and rotated into the router with zero feed weights or zero feed IDs, but those components still participate in every runtime validation path and can make all price modes revert. The constructor explicitly allows individual feed IDs and weights to be zero and checks only that the total weights sum to `1e18` (`packages/perps/src/PletherOracle.sol:88-92, 128-137`). At runtime, however, both `_computeLiveBasketPrice` and `_computeBasketPriceFromFeeds` iterate over every configured component regardless of weight, fetch its Pyth price, enforce staleness, publish-time divergence, and `p.price > 0`, and only then apply the weight (`packages/perps/src/PletherOracle.sol:621-689`). That means a supposedly disabled zero-weight component is still load-bearing: if its feed is unset, stale, or divergent, `updatePrice`, delayed execution, liquidation pricing, and pool reconciliation all revert even though the component contributes zero economic weight. Router oracle validation does not inspect feed composition, so such an oracle passes rotation as long as its engine, pool, and Pyth getters look correct (`packages/perps/src/router/OrderOracleExecution.sol:46-73`).

Preconditions

- Governance or the router admin deploys or rotates to a `PletherOracle` that includes a zero-weight or zero-ID basket component, which the constructor accepts (`packages/perps/src/PletherOracle.sol:88-92, 128-137`).
- That component later returns missing, stale, zero, or time-divergent data from Pyth while any oracle-backed path builds a basket (`packages/perps/src/PletherOracle.sol:621-689`).

Impact

The active oracle can fail across all runtime modes even though the misconfigured component was meant to be economically inert. Delayed executions, liquidations, mark refreshes, and LP settlement reads can all be denied until governance rotates to a corrected oracle.

Remediation

Reject zero-weight and zero-ID components at construction, or skip zero-weight components entirely in all runtime validation and basket-building loops.

Medium ACKNOWLEDGED

Trader can avoid paying LP carry by checkpointing early

ID: MED-22

Comment

MED-22 — Accepted rounding limitation. Position carry is rounded down to six-decimal USDC, and the checkpoint advances without retaining the fractional remainder, even when the charge rounds to zero. Each conversion can discard less than one USDC atom (0.000001 USDC); repeated checkpoints can cumulatively reduce LP carry revenue. This includes checkpoints triggered by zero-amount withdrawals. Practical savings depend on position borrow size, utilization, checkpoint frequency, and transaction costs. The behavior is accepted; fractional carry persistence is not being added.

Documented in SECURITY.md, 'Accepted rounding behavior': https://github.com/Plether-Fi/plether-core/commit/f6e0f156bf5b170aa383c1df533eff9cb27ec157

Locations:

`packages/perps/src/libraries/PositionRiskAccountingLib.sol:77-124`

`packages/perps/src/MarginClearinghouse.sol:387-405`

`packages/perps/src/MarginClearinghouse.sol:1289-1316`

`packages/perps/src/CfdEngine.sol:528-546`

`packages/perps/src/CfdEngine.sol:1353-1364`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:178-196`

A trader can repeatedly checkpoint an open position before one USDC atom of carry accrues and permanently discard the fractional carry owed to LPs. Carry is accrued in a side index, but account-level collection converts the index delta to six-decimal USDC with floor division and `_checkpointPositionCarry` then overwrites `pos.lastCarryIndex` even when the rounded carry is zero. Margin-changing entrypoints call this checkpoint path, including deposits after crediting settlement, and zero-amount withdrawals also run the same checkpoint before the withdraw guard. By calling one of these paths often enough that each interval rounds to zero, the trader resets the carry basis without paying, so the fractions never accumulate into a later charge.

Preconditions

- The trader has an open position with nonzero `borrowBaseUsdc`, so carry accrues from side index changes (`packages/perps/src/CfdEngine.sol:932-936`, `1358-1364`).

- The trader can call a clearinghouse path that changes margin, such as `depositMargin`, which credits settlement and then forces `realizeCarryBeforeMarginChange(account)` (`packages/perps/src/MarginClearinghouse.sol:322-405`).
- Each checkpoint interval is short enough that `computeIndexedCarryUsdc(borrowBaseUsdc, endIndex - startIndex)` rounds down to zero even though the true fractional carry is positive (`packages/perps/src/libraries/PositionRiskAccountingLib.sol:107-123`).

Impact

LPs lose carry revenue. A trader who keeps checkpointing before 1 USDC accumulates can suppress carry charges for as long as they keep the cadence, so leverage that should pay recurring borrow cost becomes effectively free.

Remediation

Retain sub-USDC carry instead of discarding it. Store an unscaled per-position carry remainder or accumulate carry in higher precision, and only advance `lastCarryIndex` after adding the fractional remainder forward rather than dropping it on zero-collection checkpoints.

Medium ACKNOWLEDGED

Mixed-time basket can block trader protections during valid mark updates

ID: MED-23

Comment

MED-23 — Addressed.

Protections record the Engine's oracle timestamp when armed and require subsequent oracle progress against that value, rather than comparing oracle time with the wall-clock arming time.

The live basket's progress marker uses the newest component timestamp, so a newer component can advance it even when another component has not updated. A changed price at an already accepted marker is rejected. Component freshness and nonregression checks remain enforced.

The separate component-selection limitation is acknowledged under MED-7.

Fixes:
https://github.com/Plether-Fi/plether-core/commit/2fca7941
https://github.com/Plether-Fi/plether-core/commit/7de2f742b56f58393f4284159d88e10d2abb752e

Locations:

`packages/perps/src/PletherOracle.sol:651-691`

`packages/perps/src/CfdEngine.sol:1419-1431`

`packages/perps/src/PositionProtectionBook.sol:462-466`

A valid mixed-time oracle basket can move the engine mark and liquidation price while still preventing an armed take-profit or stop-loss from triggering. `PletherOracle` accepts live baskets whose component publish times differ by up to the configured divergence, but it records the basket `publishTime` as the earliest component time (`packages/perps/src/PletherOracle.sol:651-691`). The router uses that snapshot to refresh the engine mark, and the engine accepts an equal mark timestamp (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:575-583`, `packages/perps/src/CfdEngine.sol:1418-1431`). `PositionProtectionBook.activate()` then rejects unless `publishTime > armedAt` (`packages/perps/src/PositionProtectionBook.sol:447-476`).

If Alice arms protection at time `1000`, and later five feeds advance to `1005` while one feed remains at `1000`, the basket can still pass the divergence check and cross Alice's trigger price. The engine mark, and liquidation pricing, now reflect the moved basket, but protection activation still reverts because the basket timestamp remains `1000`. This can happen during honest mixed-time Pyth updates, and a caller can also force it by refreshing only a subset of feeds. Traders can therefore miss intended stop-loss or take-profit exits and may be liquidated before the lagging feed advances or a later caller includes fresher data for it.

Preconditions

- An account has an armed protection whose separation checks rely on `armedAt` and `armedBlock` (`packages/perps/src/PositionProtectionBook.sol:963-977`).
- At least one configured Pyth feed still has the old publish time while other basket components advance enough to cross the trigger price, and the resulting component spread remains within the active divergence limit (`packages/perps/src/PletherOracle.sol:621-691`).
- Any permissionless keeper or liquidator submits selective Pyth updates or simply acts while one component lags in storage.

Impact

Stop-loss and take-profit protections can be unavailable during real basket moves, exposing traders to avoidable liquidation or missed exits while the engine and liquidation paths continue using the moved mark.

Remediation

Use a basket freshness marker that advances whenever any component used in pricing advances, preferably `maxPublishTime` or an explicit basket version/hash, and apply the same marker consistently in both engine mark updates and protection trigger gating.

Medium ACKNOWLEDGED

LP can delay other redeemers with zero-payout refund epochs

ID: MED-24

Comment

MED-24 — Accepted bounded-work limitation; no code change. Zero-payout redemption epochs can consume settlement work and require additional calls before later epochs are funded. The limit is 16 epochs per tranche phase, not 16 requests or wallets: requests in the same epoch aggregate. New epochs append behind existing requests within that tranche, so they cannot continually insert ahead of an already queued withdrawal. We retain the bounded queue design and accept the extra-call behavior; this does not guarantee Junior funding under insufficient liquidity or persistent Senior priority.

Disposition: https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/HousePool.sol:932-967`

`packages/perps/src/HousePool.sol:177-177`

`packages/perps/src/TrancheVault.sol:941-967`

`packages/perps/src/TrancheVault.sol:1483-1501`

Anyone who can hold dust shares can force LP settlement to spend its bounded redemption work on zero-payout refunds instead of funding real withdrawals. In `HousePool._processRedemptionPhase`, a head redeem epoch whose quote rounds to zero calls `refundRedeemEpochRemainder(...)` and still increments `phase.processedEpochs` (`packages/perps/src/HousePool.sol:941-967`). Because each settlement call visits at most `MAX_LP_EPOCHS_PER_PHASE = 16` epochs, an attacker can split one funded position into many 1-share accounts, submit one full-balance redeem from each account in consecutive epochs, and consume the whole per-call budget on refund-only work before a later victim epoch is reached. The scratch test `TrancheVaultZeroPayoutWorkCapDoSTest` confirms that 16 such epochs produce `juniorProcessedEpochs == 16` while `juniorFundedShares == 0`, leaving a later large redeem completely unfunded in the same settlement call.

This is a silent assumption mismatch between the settlement progress guard and the actual economic work done. The bounded processor assumes that visiting an epoch is enough progress to justify ending the call without reverting, but zero-payout refund epochs do not move any assets or burn any funded shares. A reachable LP can therefore prefill the queue with dust epochs and make keepers or other users pay for many extra settlement calls before ordinary redeemers receive funds.

Preconditions

- The redeem phase uses the bounded processor with `MAX_LP_EPOCHS_PER_PHASE = 16`, and zero-output heads take the `refundRedeemEpochRemainder` branch that still increments `phase.processedEpochs` (`packages/perps/src/HousePool.sol:177`, `packages/perps/src/HousePool.sol:941-967`).
- An LP attacker can obtain many 1-share accounts by splitting a funded position, then submit one full-balance redeem from each account in different epochs so each request is its own zero-payout head. Settlement remains permissionless, so the victim depends on repeated third-party or self-funded calls to clear the dust backlog.
- The queued dust redeems must price to zero assets under the current tranche state, which is reachable for 1-share full exits and is exercised in `packages/perps/test/perps/spec/lp/AsyncLpEpochSettlement.t.sol:648-685`.

Impact

Victim redemptions can be delayed for as many settlement calls as the attacker prepays in dust epochs. The queue still makes eventual progress, but keepers and withdrawing LPs must burn extra gas on refund-only passes before economically meaningful redeems are funded.

Remediation

Do not count zero-payout refund epochs against the bounded redemption work cap. Preferably keep a separate counter for asset-funding work and only increment the capped `processedEpochs` value when the call actually funds or burns a redeem epoch; zero-payout refund cleanup can still be bounded by a distinct gas-safe limit if needed.

Medium ACKNOWLEDGED

Arbitrum block numbering can delay every later trader in the FIFO queue

ID: MED-25

> **Comment**
> MED-25 — Addressed. Order commitment and same-block execution guards now use the execution-chain block number. On Arbitrum One, Nova and Sepolia this is ArbSys.arbBlockNumber(), so a later L2 block can execute the FIFO head even while Solidity block.number still reports the same L1 block. Same-L2-block execution remains prohibited, and historical publication-time requirements remain independent. On those Arbitrum chains an unavailable ArbSys response reverts rather than falling back to the L1 clock. Fix: https://github.com/Plether-Fi/plether-core/commit/8a2853f80bce067656c1fd0fcf2dadc5892e2aee

Locations:

`packages/perps/src/OrderRouter.sol:149-160`

`packages/perps/src/OrderRouterExecutionSidecar.sol:432-439`

On Arbitrum, one queue head can hold later users behind it for multiple sequenced L2 blocks because the router uses ancestor-chain `block.number` as its same-block MEV fence. `executeCommitOrderItem` records `commitBlock = uint64(block.number)` when the order is created, and `_executePrepared` returns `PendingReason.SameBlock` whenever a later execution still sees `block.number == order.commitBlock`, even if the keeper is already in a later L2 block with a post-commit oracle publication. Since `executeOrder` and `executeOrderBatch` only advance from `nextExecuteId` and stop when the head remains pending, every later queued order stays blocked until the ancestor block changes. This widens the intended one-block lockout into a multi-block availability failure on Arbitrum and can consume a meaningful part of the caller-chosen execution window or cause short-window orders to expire before they ever become executable.

Preconditions

- A pending order is at the global FIFO head, with `commitBlock` recorded from `block.number` at commit time (`packages/perps/src/OrderRouter.sol:149-160`).
- A trader or griefer submits that head order, and a keeper attempts execution in a later sequenced L2 block that still reports the same EVM `block.number` as the commit block.
- The deployment runs on Arbitrum or another chain where multiple sequenced blocks can share one ancestor-chain `block.number` (`.guard/research/dependencies.md:8`).

Impact

Any user can temporarily DoS the protocol's shared delayed-order queue. Later traders cannot get their own orders executed while the poisoned head order keeps returning `SameBlock`, so protective closes and ordinary user exits can be delayed during volatile periods.

Remediation

Use an L2-local separation check instead of `block.number == order.commitBlock`. On Arbitrum, rely on the timestamp and oracle publish-time MEV boundary already recorded in the order, or store and compare an execution-specific monotonic value that actually changes every L2 block on the target chain.

Medium ACKNOWLEDGED

Owner can activate trading against ownerless pool assets

ID: MED-26

Comment

MED-26 — Addressed for initial activation. HousePool.activateTrading() now rejects the false-to-true transition while stored unassigned assets, projected unassigned assets, or residual pending claimant assets are nonzero. Existing seed and senior-capacity checks remain enforced.

This is a bootstrap boundary: repeat activation remains allowed, and ownerless value arising after activation does not introduce a new continuous trading or queued-order gate. Raw, unaccounted USDC donations remain outside canonical accounting until accountExcess() admits them. This guard does not add an oracle-freshness check.

Fix:
https://github.com/Plether-Fi/plether-core/commit/00baf069a93f34f3727c9f55508d6269fc56ead5

Locations:

`packages/perps/src/HousePool.sol:514-530`

`packages/perps/src/HousePool.sol:507-512`

`packages/perps/src/libraries/HousePoolSeedLifecycleLib.sol:43-55`

`packages/perps/src/CfdEngine.sol:1098-1101`

New trader risk can be opened while `unassignedAssets` still exists. `activateTrading()` only checks that both seed flags are set, senior limits are finite, and reserved Senior commitments fit those limits. It does not check the same ownerless-asset condition that blocks deposit activation, even though the accounting spec says `unassignedAssets > 0` must block activation. After `activateTrading()` sets `isTradingActive = true`, `canIncreaseRisk()` becomes true because it is just the seed-and-flag predicate, so Router admission can accept new opens against pool backing that still has no claimant path. Those ownerless assets are included in `totalAssets()` and therefore in Engine depth/solvency calculations, so trader profits and losses can consume or depend on value governance has not yet assigned to any tranche owner.

Preconditions

- The pool has completed both seed initializations, but `unassignedAssets` is nonzero because reconcile routed value with no safe claimant path into storage (`packages/perps/src/HousePool.sol:1538-1549`, `packages/perps/src/HousePool.sol:650-680`).

- The owner calls `activateTrading()` before assigning those assets with `assignUnassignedAssets()` (`packages/perps/src/HousePool.sol:517-529-`). Any trader can then submit risk-increasing orders through the normal Router path, which checks `pool.canIncreaseRisk()` for the lifecycle gate (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:317-322`; same predicate is used protocol-wide).
- Deployment intends the documented activation invariant from the accounting spec that `unassignedAssets > 0` blocks activation (`packages/perps/ACCOUNTING_SPEC.md:333-337`).

Impact

The protocol can start underwriting trader positions with assets that still have no LP owner. Losses can consume value that governance has not yet restored to a tranche, and profits can rely on that same ownerless cash for solvency and carry depth. This breaks the documented seed/activation lifecycle and can misallocate later-assigned LP value.

Remediation

Make `activateTrading()` reuse the same projected entry gate used for deposit activation, or at minimum revert while current or projected `unassignedAssets` is nonzero and while pending claimant reservations would leave activation blocked.

Medium ACKNOWLEDGED

Owner can change senior rate while junior LP accounting is stale

ID: MED-27

Comment

MED-27 — Addressed. A Senior coupon-rate change must satisfy both the current Pool freshness limit and the Engine's reconciliation freshness policy. HousePool performs full reconciliation under the old rate before activating the new rate, rather than taking the stale-mark coupon-only fallback. Rejection leaves the pending proposal and accounting unchanged. The accepted limitation is that a rate change, including other configuration changes bundled with it, waits until a qualifying fresh mark is available.

Fix:
https://github.com/Plether-Fi/plether-core/commit/9dbaf0c0d984e113c85616efb6542c43fa852774

Locations:

`packages/perps/src/HousePool.sol:328-331`

`packages/perps/src/HousePool.sol:1438-1445`

`packages/perps/src/HousePool.sol:1667-1683`

`packages/perps/src/CfdEngineProtocolLens.sol:79-91`

The owner can finalize a new `seniorRateBps` even when the mark is already stale under HousePool's actual reconcile policy. `finalizePoolConfig()` gates rate changes with `_requireRateChangeMarkFresh(_getHousePoolStatusSnapshot())`, but `_requireRateChangeMarkFresh` only compares `lastMarkTime` against `poolConfig.markStalenessLimit`. The real reconcile policy is tighter when the engine's live limit is lower, because `_markIsFreshForReconcile` uses the engine lens and `OracleFreshnessPolicyLib` to select the minimum of the pool and engine limits. In that state `finalizePoolConfig()` succeeds, then `_checkpointSeniorCouponBeforeRateChange()` falls into its stale-mark branch instead of reconciling current LP PnL. That lets governance switch the coupon rate and checkpoint coupon on stale senior principal, so junior and senior balances continue from an out-of-date base until a later fresh reconcile. If the fresh mark would have reduced senior principal, junior LPs are overcharged and the inflated senior high-water mark persists into later revenue allocation.

Preconditions

- The owner finalizes a pool config whose `seniorRateBps` differs from the current value (`packages/perps/src/HousePool.sol:328-331`).
- Open positions exist so mark freshness is required for reconcile, and the engine's effective live limit is tighter than the pool limit because `engineMarkStalenessLimit < poolConfig.markStalenessLimit` (`packages/perps/src/CfdEngineProtocolLens.sol:79-91`).

- `ENGINE.lastMarkTime()` is newer than `block.timestamp - poolConfig.markStalenessLimit` but older than `block.timestamp - effectiveReconcileLimit`, so `_requireRateChangeMarkFresh` passes while `_markIsFreshForReconcile` fails (`packages/perps/src/HousePool.sol:1438-1445`, `packages/perps/src/HousePool.sol:1667-1683`).

Impact

A routine governance rate change can misallocate LP value. Senior coupon checkpointing and the new coupon rate start from stale principal instead of current mark-to-market principal, which can overcharge junior LPs, underpay or overpay senior LPs, and ratchet an incorrect senior high-water mark that affects future revenue distribution.

Remediation

Use the same freshness predicate for the rate-change gate and the coupon checkpoint path. `finalizePoolConfig()` should require `_markIsFreshForReconcile` against the full engine snapshot, not a standalone comparison to `poolConfig.markStalenessLimit`.

Medium ACKNOWLEDGED

Trader can arm a new protection from a stale engine mark after oracle state already crossed it

ID: MED-28

Comment

MED-28 — Addressed. New protection, replacement and attached-open staging validate trigger geometry against the current accepted oracle observation, rather than only the cached Engine mark. The observation supplies the arming publication baseline; post-lock collateral validation still uses the Engine accounting mark. The integrated synchronization guard also rejects arming when the accepted oracle mark is ahead of, or inconsistent with, the Engine mark. A qualifying mark must be installed before arming can proceed. This checks accepted on-chain oracle information; it cannot establish whether a newer report exists off-chain.

Protection validation: https://github.com/Plether-Fi/plether-core/commit/8f6cfd3d84d1289aa1209e3b08a06e9bfdf38a0d
Oracle/Engine synchronization: https://github.com/Plether-Fi/plether-core/commit/819192182833597641504b38cce7751bbcacecb8

Locations:

`packages/perps/src/PositionProtectionBook.sol:221-223`

`packages/perps/src/PositionProtectionBook.sol:285-286`

`packages/perps/src/PositionProtectionBook.sol:797-810`

`packages/perps/src/PositionProtectionBook.sol:462-466`

`packages/perps/src/PletherOracle.sol:145-159`

A trader can create or replace a take-profit or stop-loss after the on-chain oracle state has already crossed that threshold, and the Book still arms the protection as if the trigger were not met. `createPositionProtection()` and `replacePositionProtection()` call `_freshCachedProtectionMark()`, which only reads `ENGINE.lastMarkPrice()` and `ENGINE.lastMarkTime()` and checks age against the current oracle policy. It never reads the oracle's current basket price. That matters because `PletherOracle.updatePrice()` is permissionless and updates Pyth storage without updating the engine mark, so the oracle can already hold a newer crossed price while the engine cache remains within staleness limits. Once the protection is armed, `_prepareActivation()` rejects any trigger snapshot whose `publishTime <= armedAt`, so the already-published crossed tick cannot be used retroactively. Alice can therefore observe the current oracle price below her LONG take-profit, have that price written on-chain via `PletherOracle.updatePrice()`, then create or replace protection while the engine cache still says the mark is above the trigger. If price rebounds before a newer publish time arrives, the protection never closes even though the current on-chain oracle had already crossed the configured threshold when it was armed.

Preconditions

- The engine's cached mark is still within the active staleness window, so `_freshCachedProtectionMark()` accepts `ENGINE.lastMarkPrice()` and `ENGINE.lastMarkTime()` (`packages/perps/src/PositionProtectionBook.sol:797-810`).
- Before the protection is created or replaced, anyone submits fresher Pyth data to `PletherOracle.updatePrice()`, which updates oracle storage but does not update the engine mark (`packages/perps/src/PletherOracle.sol:145-159`).
- The fresher oracle publish time already satisfies the trader's take-profit or stop-loss, but no later publish time greater than the Book's new `armedAt` arrives before the market moves back across the threshold (`packages/perps/src/PositionProtectionBook.sol:462-466`).

Impact

New and replacement protections can miss an already-available exit that is visible in on-chain oracle state. Traders and integrators can believe a stop-loss or take-profit is armed when, in practice, the first crossed observation has already been made unusable and the position will remain open unless the threshold is crossed again on a later publish time.

Remediation

Validate create and replace operations against the oracle's current basket price, not just the engine cache. The simplest fix is to read `pletherOracle.getLatestPrice(PriceMode.MarkRefresh)` or an equivalent current snapshot during protection creation and replacement, and reject or immediately trigger when that snapshot already meets a stored threshold.

Medium ACKNOWLEDGED

Small redeemers can lose funded withdrawal value to the pool and seed account

ID: MED-29

Comment

MED-29 — Accepted tiny-share allocation limitation; no code change. In a partially funded then refundable epoch, a controller's funded-share entitlement can floor to zero even when its proportional funded-asset amount is positive. The zero-share gate suppresses that asset claim; terminal asset residue returns to the pool and refundable-share residue goes to the seed receiver. We accept this edge case and retain the existing entitlement accounting. Its value depends on share price and epoch allocation, so it is not universally bounded below one USDC base unit.

Disposition: https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:771-785`

`packages/perps/src/TrancheVault.sol:1214-1244`

`packages/perps/src/TrancheVault.sol:1248-1263`

A redeem request that is partially funded can pay a controller zero funded shares even when that controller's pro-rata funded assets are still positive. `claimableRedeemAssets()` first computes `entitledShares = floor(position.shares * epoch.fundedShares / epoch.shares)` and then hard-zeros assets whenever `entitledShares == 0` (`packages/perps/src/TrancheVault.sol:780-785`). The same coupling is repeated in `_maybeProcessRedeemFunding()`: if `entitledShares == 0`, `entitledAssets` is forced to zero, and once every controller has processed funding or refund basis the remaining `epoch.fundedAssets - epoch.claimedAssets` is swept back to `HousePool` as `assetDust` (`packages/perps/src/TrancheVault.sol:1224-1244-`). Refund share dust is likewise transferred to the permanent seed account (`packages/perps/src/TrancheVault.sol:1248-1263`). For example, if two controllers each redeem 1 share unit, the pool funds only 1 share for 2 asset units, and the remainder becomes refundable, then each controller gets `floor(1 * 1 / 2) = 0` funded shares even though each has `floor(1 * 2 / 2) = 1` funded asset unit. Both users see zero claimable assets, both can only call the refund path, and after the last refund claim the full 2 funded asset units are returned to the pool while the remaining refundable share dust is sent to the seed receiver. The redeemers receive neither the funded assets nor the residual share.

Preconditions

- A redeem epoch is partially funded and then marked refundable for the remainder (`packages/perps/src/TrancheVault.sol:1456-1501`).

- For at least one controller, `floor(position.shares * epoch.fundedShares / epoch.shares) == 0` while `floor(position.shares * epoch.fundedAssets / epoch.shares) > 0`, which can happen for very small full-balance redeemers when the epoch has sparse funded shares but non-trivial asset value (`packages/perps/src/TrancheVault.sol:780-785`, `packages/perps/src/TrancheVault.sol:1224-1227`).
- That controller or its operator claims the redeem refund, allowing `_mayBeProcessRedeemFunding()` and the terminal dust sweeps to run (`packages/perps/src/TrancheVault.sol:899-924`).

Impact

Affected redeemers permanently lose both the funded asset portion of their withdrawal and the refundable share remainder. The funded assets are reclassified as pool excess and the share dust is transferred to the seed account, so the loss falls on the small redeemers rather than remaining claimable.

Remediation

Do not gate funded-asset entitlement on `entitledShares != 0`. Track and pay `floor(position.shares * epoch.fundedAssets / epoch.shares)` independently, and only sweep true residual dust after both funded-share and funded-asset entitlements for every controller have been settled.

Medium ACKNOWLEDGED

Trader can lose stop-loss automation during routine frozen markets

ID: MED-30

Comment

MED-30 — Addressed with an atomic manual exit; frozen-market automatic triggering remains intentionally unavailable. cancelPositionProtectionAndCommitClose() lets the owner cancel an Armed protection and enqueue a caller-bounded full-position close in one transaction, including during frozen markets, Router pause and degraded mode. It requires the matching live position, an empty account queue and a fresh client order ID. Cancellation and the unused trigger-bounty refund roll back if close commitment fails; commitment carry is collected and checked against the close limits.

The new close follows ordinary FIFO, oracle, execution-mode and financial bounds. A successful commit does not guarantee execution. Later failure or expiry does not rearm the cancelled protection. Pending-open, Triggered and Latched records cannot use this action; new automatic triggers still require the live-oracle policy.

Fix:
https://github.com/Plether-Fi/plether-core/commit/2f53523c00fa75c093a4e470e7af87c5216b68a2

Locations:

`packages/perps/src/PositionProtectionBook.sol:462-463`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:295-296`

An armed protection stops working whenever the oracle enters its frozen mode, even though ordinary close execution still exists. `PositionProtectionBook._prepareActivation()` unconditionally reverts with `OrderRouter__ConditionalTriggerFrozen()` when `pletherOracle.isOracleFrozen()` is true, so `triggerPositionProtection()` cannot queue the protective close during those periods. At the same time the router rejects any new manual close while a protection is active, because public order commit paths enforce `ProtectionActive`. The user's only escape hatch is to first cancel the protection and then submit a separate close order. During a routine frozen window this turns a single safety mechanism into a two-transaction manual workflow, which can cause missed exits or larger losses if price moves before both transactions are mined and executed.

Preconditions

- The account has an active armed protection (`packages/perps/src/PositionProtectionBook.sol:456-460`).
- The protocol is in an oracle-frozen window, so `pletherOracle.isOracleFrozen()` returns true (`packages/perps/src/PositionProtectionBook.sol:462-463`).

- The trader or a keeper tries to use the protection, or the trader tries to submit a replacement close order while the protection is still active (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:295-296`).

Impact

The TP/SL feature is unavailable during a normal operating mode, and it also blocks one-step manual close submission while it is active. Traders must cancel first and then re-commit a close, which can miss intended stop-loss or take-profit exits.

Remediation

Allow frozen-mode protection triggers to queue the same frozen-policy close path used by ordinary closes, or automatically ignore the protection lock for user-initiated close commits while the protection cannot trigger.

Medium | Marked invalid by client

Coordinated zaps transfer value between BEAR and BULL DOVs

ID: MED-31

MARKED INVALID BY CLIENT

Out of scope (outside of packages/perps)

Locations:

`packages/options/src/DOVZapRouter.sol:100-110`

`packages/options/src/PletherDOV.sol:593-627`

A depositor can queue funds in the DOV for the more valuable synthetic before a routine `coordinatedZapAndStartEpochs()` call. The matched leg charges both DOVs the same USDC amount and deposits the same number of BEAR and BULL tokens, even when those tokens have different prices. With CAP=$2, BEAR at $1.20, BULL at $0.80, and $100 contributed by each DOV, the route allocates $120 of BEAR to one vault and $80 of BULL to the other. `PletherDOV` then attributes the received staked tokens to each vault's own depositors, so the value difference is not reconciled between shareholders.

Preconditions

- The supplied `DOVZapRouter` is configured as both vaults' zap keeper and performs its normal coordinated conversion (`packages/options/src/DOVZapRouter.sol:84-106`).
- Both DOVs have USDC to match, and the synthetic prices differ from CAP/2 (`packages/options/src/DOVZapRouter.sol:100-110`).
- A public depositor chooses the DOV receiving the more valuable synthetic (`packages/options/src/PletherDOV.sol:166-185`).

Impact

Shareholders in one DOV subsidize shareholders in the other through a routine matched mint, causing contributed capital to be allocated unequally. The amount ultimately realized depends on the vault's covered-call epoch and subsequent market movements.

Remediation

Calculate the matched pair from each side's price, charge each vault its corresponding value, and route residual balances separately. Enforce a minimum received value for each DOV within explicit fees and slippage.

Medium | Marked invalid by client

Settled BULL collateral regains borrowing power below CAP

ID: MED-32

MARKED INVALID BY CLIENT

Out of scope (outside of packages/perps)

Locations:

`packages/spot/src/oracles/MorphoOracle.sol:78-87`

`packages/spot/src/core/SyntheticSplitter.sol:382-384`

`packages/spot/src/core/SyntheticSplitter.sol:418-420`

`packages/spot/src/oracles/StakedOracle.sol:56-65`

`script/DeployToMainnet.s.sol:268-274`

After `SyntheticSplitter` enters liquidation, `emergencyRedeem()` pays the full CAP collateral for BEAR and leaves BULL without redemption value. The liquidation state is irreversible, but `MorphoOracle.price()` continues to derive BULL's value from the live basket. If the basket later falls below CAP, the oracle again reports a positive BULL price. A holder can supply staked BULL directly to the configured Morpho market and borrow against that reported value; the leverage router's ACTIVE checks do not gate Morpho's direct lending entrypoints.

Preconditions

- The splitter is liquidated and BEAR is redeemed for the full CAP collateral, leaving BULL with no redemption value (`packages/spot/src/core/SyntheticSplitter.sol:382-384`, `packages/spot/src/core/SyntheticSplitter.sol:391-420`).
- The live basket later falls below CAP, causing the inverse oracle to report positive value for BULL (`packages/spot/src/oracles/MorphoOracle.sol:78-87`).
- The configured Morpho BULL market has lending liquidity and accepts BULL collateral (`script/DeployToMainnet.s.sol:268-274`).
- A borrower stakes BULL and uses Morpho's direct collateral and borrow functions outside the router lifecycle checks (`test/fork/MorphoBorrowInvariantFork.t.sol:207-227`).

Impact

Collateral whose redemption value was extinguished by settlement can regain borrowing power. Borrowing against that unsupported value can leave Morpho lenders with bad debt if the borrower does not repay. The realized amount depends on the deployed market, available liquidity, and oracle configuration.

Remediation

Make the collateral oracle settlement-aware so BULL remains valued at zero after liquidation, and verify the direct lending paths against the terminal state rather than relying on router-only lifecycle checks.

Low ACKNOWLEDGED

Indexer can misattribute tranche deposit lifecycle to the wrong account

ID: LOW-1

> **Comment**
> LOW-1 — Addressed. Deposit lifecycle event ABI argument labels now identify the request controller rather than an owner. The controller owns the asynchronous request/claim and can differ from the funding account and final receiver. Event topics and encoded values are unchanged; the correction removes the misleading labels. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/TrancheVault.sol:190-202`

`packages/perps/src/TrancheVault.sol:596-597`

`packages/perps/src/TrancheVault.sol:856-857`

`packages/perps/src/TrancheVault.sol:1100-1102`

The vault's legacy deposit-side events do not emit the actor their ABI says they emit. `DepositRequested`, `DepositRequestCancelled`, and `DepositSharesClaimed` all declare an indexed `owner` field, but the implementation passes `controller` instead. In a real third-party-funded flow, Alice can fund `requestDeposit(25_000e6, Bob, Alice)`, the canonical `DepositRequest` event will correctly record `owner = Alice` and `controller = Bob`, but the extra `DepositRequested` event will publish Bob in the slot labelled `owner`. The same mismatch happens on cancellation and claim. Any indexer, frontend, or off-chain accounting job that consumes these legacy events by their documented field names will attribute requests and claim activity to the wrong address.

Preconditions

- An off-chain consumer uses `DepositRequested`, `DepositRequestCancelled`, or `DepositSharesClaimed` as their source of truth for the address labelled `owner` (`packages/perps/src/TrancheVault.sol:190-202`).
- The request is controlled by an address different from the asset owner or final receiver, which is a supported flow through `requestDeposit(assets, controller, owner)` and the async claim surface (`packages/perps/src/TrancheVault.sol:563-568`, `packages/perps/src/TrancheVault.sol:951-958`).

Impact

Off-chain systems can record the wrong LP owner/controller for deposit requests, cancellations, and claims. That can break portfolio attribution, compliance reporting, automation keyed by the owner field, and frontend request histories even though on-chain balances remain correct.

Remediation

Change these legacy events so their indexed field names and emitted values match. The simplest fix is to rename the affected fields to `controller`, or emit the actual owner/receiver semantics instead of the controller value.

Low ACKNOWLEDGED

Unset mark can block valid first opens

ID: LOW-2

> **Comment**
> LOW-2 — Addressed through explicit mark initialization. Admission no longer substitutes a synthetic 1e8 price when the Engine mark is zero; it reverts with MarkPriceUnavailable before pricing notional or reserving a bounty. Bootstrap refreshes the Engine mark from Pyth before trading activation, including an already-active bootstrap rerun. Close preview applies the same zero-mark guard where a price is required. The accepted requirement is a real initial oracle mark before price-dependent admission; subsequent nonzero marks need not be fresh at commitment, as acknowledged under LOW-57. Fix: https://github.com/Plether-Fi/plether-core/commit/d7b64e164d3572f0af12504a4c86f2eea0e4de18

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:393-401`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:338-344`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:430-438`

The router can reject valid first open orders before any oracle mark has been cached. _commitReferencePrice returns the hardcoded value 1e8 when engine.lastMarkPrice() is zero, and _validatedOpenBounty uses that value for the opening notional check and bounty calculation. If trading has been activated but no one has refreshed the mark yet, an order that exceeds minOpenNotionalUsdc at the real oracle price can be treated as too small at the dummy $1 price and revert with OrderRouter__CommitValidation(11).

This does not let an attacker steal funds or permanently block trading, and the reported partial-close impact is not reachable in the ordinary lifecycle because positions are opened through oracle-backed execution that updates the cached mark. It is still observable wrong admission behavior for the initial pre-mark state.

Preconditions

- The engine cached mark is unset, so `engine.lastMarkPrice() == 0` and `_commitReferencePrice` substitutes `1e8` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:430-438`).
- A user commits an open or close that is near the router's minimum-notional or minimum-partial-close threshold (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:338-344`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:393-401`).

Impact

Valid initial open orders can be rejected, and the opening execution bounty can be calculated from a fabricated price, until a caller refreshes the cached mark.

Remediation

Do not substitute 1e8 for an unset cached mark in commit-time admission. Require a nonzero cached mark before open admission or refresh the mark as part of the activation/commit path.

Low ACKNOWLEDGED

LP can have smaller share claims revert on a funded deposit

ID: LOW-3

Comment
LOW-3 — Accepted partial-claim rounding behavior; no code change. A partial share-denominated deposit claim can round its asset cost up to the entire remaining asset entitlement while leaving shares unclaimed, so the guard reverts. Claiming the full remaining entitlement avoids this specific rounding case. A failed partial claim preserves the entitlement. We retain the guard rather than permit one side of the asset/share entitlement to be exhausted while the other is stranded.

Disposition:
https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:1043-1064`

`packages/perps/src/TrancheVault.sol:1067-1081`

`packages/perps/src/TrancheVault.sol:1277-1282`

A user who has a funded async deposit cannot necessarily claim a smaller share amount even when `maxMint()` says those shares are available. `maxMint()` returns the full remaining share entitlement for the head deposit request, but `_claimMint()` converts a requested share amount back into assets with `Math.Rounding.Ceil` and then reverts whenever that rounded asset amount is greater than or equal to the remaining asset basis. With a finalized request worth 1e6 assets and 1e9 shares, requesting 999,999,999 shares rounds up to the full 1e6 assets and reverts at `_depositAssetsForShares()`, while requesting the full 1e9 shares succeeds. Any integrator that splits claims, shaves one share for slippage, or assumes ERC-4626-style monotonicity from `maxMint()` will hit unexpected reverts on otherwise funded claims.

Preconditions

- A deposit request has been finalized so the controller has nonzero `claimableDepositShares` and `maxMint(controller)` returns that full remaining share entitlement (`packages/perps/src/TrancheVault.sol:1053-1060`, `packages/perps/src/TrancheVault.sol:1277-1282`).
- The claimant calls `mint(shares, receiver)` or `claimDeposit(...assets...)` for a partial amount close enough to the remaining entitlement that `ceil(shares * epoch.assets / epoch.shares)` equals the full remaining assets (`packages/perps/src/TrancheVault.sol:1067-1081`).
- No special deployment parameters are required.

Impact

Funded deposit claims become non-monotonic: values below the advertised maximum can revert while the full claim succeeds. This breaks chunked claim flows and any integration that relies on `maxMint()` being a usable upper bound for smaller share claims.

Remediation

Make partial share claims saturate to the remaining asset basis instead of reverting when the rounded asset amount reaches the final tranche, or expose a separate minimum-partial-claim threshold so `maxMint()` does not imply unsupported intermediate amounts.

Low ACKNOWLEDGED

LP can have smaller withdrawal claims revert on a funded redeem

ID: LOW-4

Comment
LOW-4 — Accepted partial-claim rounding behavior; no code change. A partial asset-denominated withdrawal can round its share cost up to all remaining funded shares while leaving assets unclaimed. The transaction reverts without consuming the entitlement. Claiming the full funded remainder avoids this specific case. The guard is retained to prevent mismatched exhaustion of the asset and share entitlement.

Disposition:
https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:1154-1177`

`packages/perps/src/TrancheVault.sol:1284-1289`

A user who has a funded async redemption cannot necessarily claim a smaller asset amount even when `maxWithdraw()` advertises that amount as withdrawable. `_claimWithdraw()` converts the requested assets back into shares with `Math.Rounding.Ceil` and reverts when that rounded share amount reaches the full remaining share entitlement. In the tested shape where a request is funded for `remainingShares + 1` assets, withdrawing `remainingAssets - 1` still rounds up to `remainingShares` and reverts, while withdrawing the full `remainingAssets` succeeds. This makes funded withdrawal claims non-monotonic and breaks integrations that split redemptions or trim the requested assets below the maximum for safety.

Preconditions

- A redemption request has already been funded so the controller has nonzero `claimableRedeemAssets` and `maxWithdraw(controller)` returns that full remaining asset entitlement (`packages/perps/src/TrancheVault.sol:1164-1167`, `packages/perps/src/TrancheVault.sol:1284-1289`).
- The claimant calls `withdraw(assets, receiver, controller)` for a partial amount close enough to the funded total that `ceil(assets * remainingShares / remainingAssets)` equals the full remaining shares (`packages/perps/src/TrancheVault.sol:1169-1175`).
- No special deployment parameters are required.

Impact

Values below the advertised `maxWithdraw()` can still revert, so claim processing is not monotonic. Wallets and routers that assume they can safely withdraw a little less than the maximum can fail on already funded claims.

Remediation

When a rounded partial withdraw would consume the full remaining share basis, either treat it as a full claim or expose claim granularity explicitly so `maxWithdraw()` does not imply unsupported intermediate amounts.

Low ACKNOWLEDGED

LP can have small deposit claims revert below maxDeposit

ID: LOW-5

Comment
LOW-5 — Accepted minimum claim-size limitation; no code change. An asset-denominated deposit claim can round down to zero shares and revert even below maxDeposit. The entitlement is preserved on failure; claiming the full remainder avoids this specific case. maxDeposit is an upper bound on claimable assets, not a guarantee that every smaller positive amount produces a nonzero share allocation. No fractional-share accounting is being introduced.

Disposition:
https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:1006-1041`

`packages/perps/src/TrancheVault.sol:1270-1275`

A controller can have small asset-denominated deposit claims revert even though they are below maxDeposit(). TrancheVault.maxDeposit() returns the full remaining finalized deposit basis from claimableDepositRequest(), but _claimDeposit() prices partial asset claims with floor rounding and rejects them with TrancheVault__ClaimSharesZero when the rounded share amount is zero. For example, if a finalized epoch allocated 1 share unit for 1,000,000 asset units, maxDeposit(controller) returns 1,000,000, claiming the full amount succeeds, but claiming 1 asset unit reverts because it maps to zero shares. The funds remain claimable through a full remainder claim or share-denominated claim, so the impact is failed integrations or frontends that assume every positive amount up to maxDeposit() is a valid asset claim chunk.

Preconditions

- A deposit request has been finalized so the controller has nonzero `claimableDepositRequest` and `maxDeposit(controller)` returns the full remaining asset basis (`packages/perps/src/TrancheVault.sol:1016-1025`, `packages/perps/src/TrancheVault.sol:1270-1275`).
- The claimant requests a nonzero partial asset amount whose pro-rata share allocation rounds down to zero under `Math.mulDiv(assets, epoch.shares, epoch.assets, Floor)` (`packages/perps/src/TrancheVault.sol:1026-1035`).
- No special deployment parameters are required.

Impact

Values below the advertised asset maximum can still revert. Frontends and routers that split funded deposit claims into smaller asset chunks can fail unexpectedly on live positions.

Remediation

Either enforce and expose a minimum partial asset claim size derived from the epoch exchange rate, or treat any positive partial claim that rounds to zero shares as unsupported in the public limit surface instead of advertising the full remaining assets through `maxDeposit()`.

Low ACKNOWLEDGED

User can permanently strand tokens in MarginClearinghouse

ID: LOW-6

Comment

LOW-6 — Non-settlement tokens addressed; USDC limitation accepted. MarginClearinghouse.rescueToken(token, recipient) lets the owner transfer the full balance of an unrelated ERC-20 to a nonzero recipient. The function rejects settlement USDC, uses SafeERC20 and the existing reentrancy guard, and leaves account balances and margin buckets unchanged. Transfer-tax tokens may deliver less than the requested amount reported by the event.

Direct, uncredited USDC transfers remain unrecoverable through current APIs, including by the owner, and are not allocated to LPs. The clearinghouse has no aggregate credited-balance counter that safely separates surplus from account backing. Adding USDC recovery and aggregate surplus accounting is explicitly out of scope; this remaining limitation is accepted.

Token rescue: https://github.com/Plether-Fi/plether-core/commit/f6e0f156bf5b170aa383c1df533eff9cb27ec157
USDC limitation: https://github.com/Plether-Fi/plether-core/commit/0f0a4e2e7227aac56e8cd172cf7e724a6d34ffb9

Locations:

`packages/perps/src/MarginClearinghouse.sol:290-304`

`packages/perps/src/MarginClearinghouse.sol:315-445`

`packages/perps/src/MarginClearinghouse.sol:1690-1793`

Tokens sent directly to MarginClearinghouse can be lost permanently. The contract only transfers assets out when it debits an existing internal clearinghouse balance, but it has no owner sweep, excess-accounting, or generic ERC20 rescue path. If Alice sends 1,000 USDC to the clearinghouse address with a raw ERC20 transfer instead of calling `depositMargin`, the contract's raw balance increases but `settlementBalances[Alice]` does not, so neither Alice nor governance has any code path to recover those funds. The same applies to any non-settlement ERC20 accidentally sent to the contract.

Preconditions

- Settlement USDC or another ERC20 is transferred to `MarginClearinghouse` without going through a code path that credits an internal balance, such as `depositMargin`, `depositFor`, or an operator settlement credit (`packages/perps/src/MarginClearinghouse.sol:315-445`, `packages/perps/src/MarginClearinghouse.sol:978-1007`, `packages/perps/src/MarginClearinghouse.sol:1698-1728`).
- Any external sender can trigger this by using a raw token `transfer` to the clearinghouse address instead of the intended entrypoint.

- No deployment parameter mitigates it because the contract exposes no rescue or excess-handling function.

Impact

Accidentally transferred tokens become unrecoverable and are removed from circulation for the sender. For settlement USDC, the protocol also has no operational way to admit or sweep the excess balance if an exchange, bridge, or integrator sends funds to the clearinghouse outside the intended flow.

Remediation

Add an explicit recovery path for unsolicited assets. For the settlement token, prefer a HousePool-style excess accounting or owner sweep flow that cannot touch credited user balances. For other ERC20s, add an owner-gated rescue function.

Low ACKNOWLEDGED

Integrator can misclassify zero add-margin attempts

ID: LOW-7

> **Comment**
> LOW-7 — Addressed. addMargin(account, 0) now reverts with CfdEngine__ZeroAmount instead of CfdEngine__PositionTooSmall. Caller ownership is still checked first; a zero amount is rejected independently of position existence. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/CfdEngine.sol:496-507`

`packages/perps/src/interfaces/ICfdEngineTypes.sol:38-43`

`addMargin` rejects `amount == 0`, but it reverts with `CfdEngine__PositionTooSmall()` instead of the dedicated zero-amount error the engine exposes elsewhere. The function NatSpec says the argument must be a nonzero amount, and the shared error set includes `CfdEngine__ZeroAmount()`, so downstream callers and monitors have a reasonable basis to interpret `PositionTooSmall` as a size-domain failure rather than an empty-input failure. This is a semantic mismatch in the public API surface, not just a naming nit, because receipt pipelines and UIs often key behavior off exact selectors.

Preconditions

- A trader or integrator calls `addMargin(account, 0)` on the engine (`packages/perps/src/CfdEngine.sol:498-507`).

Impact

Off-chain components can report the wrong failure reason or group zero-amount mistakes with genuine minimum-position-size failures. That breaks selector-based UX, monitoring, and client error handling around margin management.

Remediation

Make `addMargin` revert with `CfdEngine__ZeroAmount()` when `amount == 0`, and reserve `CfdEngine__PositionTooSmall()` for actual position-size conditions.

Low ACKNOWLEDGED

Integrators can underquote required open collateral for small positions

ID: LOW-8

> **Comment**
> LOW-8 — Not retained as a current documentation defect. CfdTypes.RiskParams already documents minBountyUsdc as both the minimum liquidation charge/reserve and the initial-equity floor; OpenAccountingLib also documents the floor on initialMarginRequirementUsdc. The coupling is intentional: integrators must use the floored planner requirement, not only the percentage of notional. No parameter split or economic change is made. Existing documentation: https://github.com/Plether-Fi/plether-core/commit/023412d0d Reviewed formula: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/libraries/OpenAccountingLib.sol#L136-L140

Locations:

`packages/perps/src/libraries/OpenAccountingLib.sol:137-139`

`packages/perps/ACCOUNTING_SPEC.md:787-794`

`packages/perps/README.md:1473-1483`

`packages/perps/DEPLOYMENT.md:316-324`

The published protocol docs describe `minBountyUsdc` only as the liquidation-charge floor, but the open planner also uses it as a floor on the initial-margin requirement. In `OpenAccountingLib.tryBuildOpenState`, `initialMarginRequirementUsdc` is first computed as `floor(notional * initMarginBps / 10_000)` and then raised to `minBountyUsdc` when smaller. By contrast, the public docs and deployment table label `minBountyUsdc` only as the total liquidation-charge floor. An integrator or frontend that reconstructs open admission from the docs can therefore quote `notional * initMarginBps / 10_000` as the required collateral and still hit an `INSUFFICIENT_INITIAL_MARGIN` rejection once governance uses a configuration where the extra floor binds.

Preconditions

- Open admission is evaluated through `OpenAccountingLib.tryBuildOpenState`, which floors `initialMarginRequirementUsdc` to `riskParams.minBountyUsdc` (`packages/perps/src/libraries/OpenAccountingLib.sol:137-139`).
- A frontend, risk engine, or keeper relies on the published docs/deployment tables for required open collateral instead of reading planner output directly.
- The configured parameters make the floor bind, for example any deployment or future timelocked config where `floor(notional * initMarginBps / 10_000) < minBountyUsdc` for a position size the engine otherwise admits.

Impact

Off-chain integrations can understate the minimum collateral needed to open or increase a position, leading to misquotes, failed executions, and incorrect risk displays for small positions.

Remediation

Document `minBountyUsdc` consistently as both the liquidation-charge floor and the open initial-margin floor everywhere it is exposed, or split the two behaviors into separately named parameters if they are not meant to be coupled.

Low ACKNOWLEDGED

Integrators can misread immediate close cash from previews

ID: LOW-9

> **Comment**
> LOW-9 — Addressed. Close previews now separate fresh payout credited to the remaining position's PnL pledge, fresh payout credited to free settlement, and deferred payout recorded as a trader claim. immediatePayoutUsdc remains the aggregate cash paid now and includes locked collateral. payoutToFreeSettlementUsdc excludes released pre-existing collateral and is not a withdrawal-capacity quote; actual withdrawals still apply the account's collateral and reservation guards. Fix: https://github.com/Plether-Fi/plether-core/commit/d5728a72d041a33d60a908462c0bce687142ef0c

Locations:

`packages/perps/src/CfdEngineLens.sol:387-393`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:844-845`

`packages/perps/src/CfdEngineSettlementSidecar.sol:605-610`

`packages/perps/src/CfdEngine.sol:818-819`

`packages/perps/src/CfdEngine.sol:221-230`

`packages/perps/src/interfaces/ICfdEngineTypes.sol:253-257`

Close previews can report the wrong amount of immediately withdrawable settlement cash. `CfdEngineLens._previewClose` sets `immediatePayoutUsdc` to the full `freshTraderPayoutUsdc` when `freshPayoutIsImmediate` is true, and to zero otherwise. That collapses distinct settlement routes into one boolean. On a profitable partial close, the planner adds the immediate price payout back into `posMarginAfter`, and live settlement calls `settlementRecordTraderClaim(..., true)`, which pays through `creditPnlPledge` and locks the value as position margin instead of free settlement. Only `actionRebatePaidUsdc` is credited to free settlement with `settleUsdc`. In the opposite mixed case, when the price payout becomes a claim but an action rebate is payable, execution credits the rebate immediately while the preview reports zero immediate payout. The result is a public preview that can overstate or understate the cash a trader receives now, even though the underlying settlement accounting is correct.

Preconditions

- A close plan succeeds for `remainingSize > 0`, the trade has a positive `pricePayoutUsdc`, and the pool has enough unreserved cash to make that price payout immediate (`packages/perps/src/libraries/CfdEnginePlanLib.sol:844-845`, `packages/perps/src/libraries/CfdEnginePlanLib.sol:916-923`).

- Any caller or integrator reads `ICfdEngineLens.previewClose` / `simulate-Close` and interprets `immediatePayoutUsdc` according to the published `ClosePre-view` semantics (`packages/perps/src/CfdEngineLens.sol:317-400`, `pack-ages/perps/src/interfaces/ICfdEngineTypes.sol:253-257`).
- The impact framing assumes downstream code distinguishes free settle-ment from locked PnL pledge, as the engine and clearinghouse do (-`packages/perps/src/CfdEngine.sol:221-230`, `packages/perps/src/Mar-ginClearinghouse.sol:997-1006`).

Impact

Preview consumers can overestimate how much settlement becomes immediately available after a profitable partial close. A frontend or automation that treats `immediatePayoutUsdc` as free settlement can display an inflated withdrawable balance or double-count value that is already included in `remainingMargin`.

Remediation

Populate `ClosePreview.immediatePayoutUsdc` from the amount that actually credits settlement. For partial closes that should exclude immediate price payout credited to PnL pledge, and only include any separately paid action rebate. Alternatively rename and re-document the field as immediate internal value instead of settlement payout.

Low ACKNOWLEDGED

ABI consumers can miss the public self-redeem-cancel selector advertised by the vault id

ID: LOW-10

> **Comment**
> LOW-10 — Addressed. IAsyncTrancheVault now declares the existing two-argument cancelRedeemRequest(requestId, receiver) wrapper. The vault advertises the updated interface ID 0xa4fc3325 while retaining legacy ID 0x8f61af3a. The wrapper cancels the caller/controller's complete request, requires receiver == caller, and retains the unmatured, wholly unfunded cancellation restrictions. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/TrancheVault.sol:289-296`

`packages/perps/src/interfaces/IAsyncTrancheVault.sol:159-173`

`packages/perps/src/TrancheVault.sol:860-865`

`packages/perps/test/perps/spec/lp/ClaimableDepositRedeem.t.sol:257-259`

`TrancheVault` exposes `cancelRedeemRequest(uint256,address)` as a public user wrapper, and the test suite exercises that two-argument entrypoint directly. The same contract also reports support for `type(IAsyncTrancheVault).interfaceId`, so integrators are encouraged to bind against that custom interface. But `IAsyncTrancheVault` does not include the two-argument redeem-cancel wrapper. An ERC-165 based client can therefore confirm support for the custom async-vault interface and still generate a typed ABI that omits a live public selector the implementation expects users to call.

Preconditions

- The deployment uses `TrancheVault`, which advertises `type(IAsyncTrancheVault).interfaceId` in `supportsInterface` (`packages/perps/src/TrancheVault.sol:289-296`).
- An LP integration, SDK, or registry builds its callable surface from `IAsyncTrancheVault` after an ERC-165 probe.
- A controller or UI wants to use the implementation's public two-argument redeem-cancel wrapper that is exercised in the tests (`packages/perps/src/TrancheVault.sol:860-865`, `packages/perps/test/perps/spec/lp/ClaimableDepositRedeem.t.sol:257-259`).

Impact

Typed clients can ship an incomplete ABI even though the contract positively advertises the custom async-vault interface id. The result is broken UX and missing user actions for redeem cancellation until the client swaps to the concrete ABI or manually patches the selector.

Remediation

Add `cancelRedeemRequest(uint256 requestId, address receiver)` to `IAsyncTrancheVault` so the custom interface id advertised by `TrancheVault` covers the full public wrapper surface.

Low ACKNOWLEDGED

Integrator can misread risk-off cutoff and misclassify unaffected close orders

ID: LOW-11

Comment
LOW-11 — Not retained as a current defect. riskOffOrderCutoff is an inclusive global order-id boundary applied only to opens; it is not a count of invalidated orders or a guarantee that the boundary id itself is an open. Close orders are explicitly unaffected in the interface and are filtered out by consumers. A nonzero cutoff can therefore coexist with a close-only queue without invalidating any close. Existing interface documentation: https://github.com/Plether-Fi/plether-core/commit/023412d0d Source: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/interfaces/IOrderRouterEmergencyAdmin.sol#L9-L12

Locations:

`packages/perps/src/interfaces/IOrderRouterEmergencyAdmin.sol:9-12`

`packages/perps/src/OrderRouterAdmin.sol:230-243`

`packages/perps/src/router/OrderQueueBook.sol:80-86`

`packages/perps/src/OrderRouterExecutionSidecar.sol:1094-1100`

The router admin exposes `riskOffOrderCutoff` as if it were the highest invalidated open order id, but the implementation stores the global queue tail instead. `OrderRouterAdmin.pause()` sets the cutoff to `router.nextCommitId() - 1` without checking whether that tail id is an open or a close, and all on-chain consumers separately filter with `!isClose` before treating an order as risk-off. This means a close-only queue can produce a nonzero cutoff even though no open order is invalidated, and a mixed queue can return a close id above the highest invalidated open. Off-chain callers that rely on `IOrderRouterEmergencyAdmin.riskOffOrderCutoff()` as documented will misidentify which orders were actually invalidated by the pause.

Preconditions

- A risk-off pause is activated while the highest committed id is a close order, or while the queue contains close orders above the highest open order (`packages/perps/src/OrderRouterAdmin.sol:237-241`).
- An integrator, monitor, or cleanup tool interprets `riskOffOrderCutoff()` using the interface contract rather than reproducing the router's extra `!isClose` filter (`packages/perps/src/interfaces/IOrderRouterEmergencyAdmin.sol:9-12`, `packages/perps/src/router/OrderQueueBook.sol:80-86`).

Impact

Off-chain systems can report the wrong set of invalidated orders, attempt unnecessary risk-off cleanup for close orders, or infer that some open orders were invalidated when none were. This is an observable API conformance bug on the emergency admin surface.

Remediation

Either change the stored value to track the highest invalidated open order id, or update the interface and docs to describe the value as a global order-id boundary that only invalidates orders after an additional `!isClose` check.

Low ACKNOWLEDGED

Operator can treat a stale cached mark as an execution blocker on the atomic settlement route

ID: LOW-12

Comment
LOW-12 — Addressed.

The monitor no longer reports CachedMarkStale as a settlement blocker when the atomic route can refresh the mark before settlement. It continues to report mark freshness, age, settlement route, and actual oracle or dependency blockers.

The legacy blocker position remains reserved to preserve existing enum ordinals. Settlement's oracle validation is unchanged.

Fix:
https://github.com/Plether-Fi/plether-core/commit/34eaecad414174f323ad7e4009aea980c126f262

Locations:

`packages/perps/src/SettlementMonitorLens.sol:1317-1377`

`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:28-37`

`packages/perps/README.md:735-738`

`SettlementMonitorLens` marks `CachedMarkStale` in `operationalBlockerMask` whenever there are open positions and the cached mark is stale, even in the same state where it classifies the required settlement route as `AtomicOracleRefresh`. On that route the stale cached mark is not a blocker: `HousePool.settleLpEpoch()` is expected to be called through `OrderRouter.settleLpEpoch(bytes[])`, which refreshes the mark first and then settles. Because `_classifyExecutionPath()` unconditionally sets the stale-mark blocker after selecting `AtomicOracleRefresh`, offchain operators that treat `operationalBlockerMask` as a go or no-go signal can incorrectly suppress a settlement that the protocol explicitly expects them to perform via the router.

Preconditions

- There is matured LP work and at least one open position, and the cached mark is stale so the monitor selects `ExecutionPath.AtomicOracleRefresh` (`packages/perps/src/SettlementMonitorLens.sol:1327-1335`).
- An operator or integration relies on `operationalBlockerMask` as a generic permission signal instead of combining it with `requiredExecutionPath` and route-specific semantics (`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:169-198`).

Impact

Offchain settlement tooling can falsely conclude that LP epoch settlement is blocked and leave matured LP work unprocessed until a human intervenes. That is an avoidable liveness failure in an operator-facing asset-management surface.

Remediation

Only set `CachedMarkStale` when the selected route is `CachedMark`, or introduce a distinct informational bit for `AtomicOracleRefresh required because cached mark is stale` so route selection and blockers cannot contradict each other.

Low ACKNOWLEDGED

LP claimants can lose async rounding dust to other tranche holders

ID: LOW-13

Comment

LOW-13 — Accepted deterministic dust allocation; no code change. Per-controller floor rounding can leave deposit-share residue, which is burned, and funded-redemption asset residue, which returns to the pool. The residual value therefore benefits remaining tranche holders rather than the final claimant. We retain this policy instead of assigning the remainder according to claim order. Share-unit residue has exchange-rate-dependent value, so not every such case has a universal one-USDC-base-unit bound.

Disposition:
https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:729-739`

`packages/perps/src/TrancheVault.sol:1105-1129`

`packages/perps/src/TrancheVault.sol:771-785`

`packages/perps/src/TrancheVault.sol:1214-1244`

LPs can lose small but real value whenever an async epoch is split across multiple controllers and the per-controller claim calculation rounds down. On deposit claims, each controller only becomes entitled to `floor(position.assets * epoch.shares / epoch.assets)` shares. After the epoch's full asset basis has been claimed, `_consumeDepositEntitlement()` burns the leftover `epoch.shares - epoch.claimedShares` from escrow instead of assigning it to the depositing cohort. On redeem claims, each controller only receives `floor(position.shares * epoch.fundedAssets / epoch.shares)` assets, and once every controller's funding basis is marked claimed, `_maybeProcessRedeemFunding()` sends the leftover `epoch.fundedAssets - epoch.claimedAssets` back to the pool. In both directions, the rounded residue is removed from the claiming cohort and accrues to unrelated tranche holders through a higher share price or extra pool assets.

Preconditions

- A deposit or redeem epoch is shared by multiple controllers so the proportional claim formulas round down for at least one controller (`packages/perps/src/TrancheVault.sol:738-739-`, `packages/perps/src/TrancheVault.sol:780-785`, `packages/perps/src/TrancheVault.sol:1224-1227`).

- Controllers eventually consume the epoch's full asset or funding basis, which triggers the terminal dust handling paths (`packages/perps/src/TrancheVault.sol:1123-1129`, `packages/perps/src/TrancheVault.sol:1232-1244`).

Impact

The affected LP cohort does not receive the full value minted or funded for its epoch. The loss is dust-sized per epoch, but it is a real transfer of value from claimants to the rest of the tranche.

Remediation

Assign the final remainder to the last claimant in the epoch instead of burning deposit-share dust or returning redeem-asset dust to the pool.

Low ACKNOWLEDGED

Integrators can overestimate request capacity because maxRequestDeposit ignores controller

ID: LOW-14

> **Comment**
> LOW-14 — Addressed. maxRequestDeposit(address(0)) now returns zero, matching request admission's invalid-controller rejection for both tranches. Nonzero controllers continue through the existing eligibility/capacity checks. This reports new-request capacity; funded claim capacity remains a separate asynchronous surface. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IAsyncTrancheVault.sol:23-27`

`packages/perps/src/TrancheVault.sol:1298-1308`

`packages/perps/src/TrancheVault.sol:1552-1578`

The async-vault API advertises `maxRequestDeposit(controller)` as a controller-scoped capacity view, but the implementation ignores the `controller` argument entirely. `TrancheVault.maxRequestDeposit()` suppresses the parameter, returns the same global value for every address, and can return a positive amount even for `address(0)`. A real request for `address(0)` still reverts in `_requireRequestDepositPreflight()` with `TrancheVault__ZeroAddress`, so callers that trust the view as a controller-specific preflight can be told a request is acceptable when the matching `requestDeposit` call must fail.

Preconditions

- An integrator or wrapper uses `maxRequestDeposit(controller)` as the capacity check promised by the interface (`packages/perps/src/interfaces/IAsyncTrancheVault.sol:23-27`).
- The supplied controller is not actually requestable, for example `address(0)`, or the caller assumes the result reflects controller-specific admissibility (`packages/perps/src/TrancheVault.sol:1298-1308`, `packages/perps/src/TrancheVault.sol:1557-1559`).

Impact

Off-chain routers and UI preflight checks can report that a deposit request is admissible for a particular controller even though the real request path will revert. This makes the product-facing request-capacity surface semantically unreliable.

Remediation

Either make `maxRequestDeposit` enforce the same controller-level constraints as `requestDeposit` that can be checked in-view, or rename and document it as a global tranche-capacity view with no controller semantics.

Low ACKNOWLEDGED

Product consumers can miss router risk-off in protocol status

ID: LOW-15

> **Comment**
> LOW-15 — Addressed. Protocol phase now includes Paused when the Router is paused. Missing mandatory dependencies report Configuring, and Degraded takes precedence over Paused. The existing tradingActive flag continues to describe pool/bootstrap activation rather than current Router availability. Existing enum values are preserved by appending Paused. Fix: https://github.com/Plether-Fi/plether-core/commit/e640f4badec287fa4a4310bf66444a4f350a7aa5

Locations:

`packages/perps/src/interfaces/ICfdEngine.sol:242-251`

`packages/perps/src/interfaces/PerpsViewTypes.sol:228-247`

`packages/perps/src/interfaces/IProtocolViews.sol:9-15`

`packages/perps/src/PerpsPublicLens.sol:479-507`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:303-324`

`packages/perps/src/OrderRouterAdmin.sol:230-249`

The product-facing protocol status can report the protocol as active while the router is in risk-off mode and rejects new opens and increases. PerpsPublicLens._getProtocolPhase() derives phase from engine wiring, degraded mode, and HousePool.canIncreaseRisk(), and getProtocolStatus() exposes pool lifecycle flags, but it does not expose OrderRouterAdmin.paused(). Fresh open commits are still blocked by OrderLiquidationBatchLogic._validateOpenCommitAllowed() when the router admin is paused. A frontend or bot that uses getProtocolStatus() as the compact product readiness signal can keep advertising new trading during an emergency risk-off window and route users into reverting transactions.

Preconditions

- The engine is fully wired, not degraded, and the HousePool still allows risk increases, so `_getProtocolPhase()` returns `Active` (`packages/perps/src/PerpsPublicLens.sol:496-507`).
- Either the router admin is paused or the oracle policy has entered `closeOnly`, both of which block new opens in normal commit flow (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:303-316`, `packages/perps/src/OrderRouterAdmin.sol:230-249`).
- A user interface, bot, or integrator uses `getProtocolStatus()` as the protocol's high-level open-trading readiness signal (`packages/perps/src/interfaces/IProtocolViews.sol:9-15`).

Impact

Product consumers can present the market as open for new risk when the router will reject those actions. Users waste gas on reverting open orders and automation can misclassify the system's operational state.

Remediation

Include router pause and oracle `closeOnly` state in the public status surface, either by folding them into `phase` or by adding explicit booleans that product integrations can rely on for open-order readiness.

Low ACKNOWLEDGED

lp integrators can overestimate withdrawal readiness after an epoch rolls

ID: LOW-16

Comment

LOW-16 — Addressed through explicit API documentation. Withdrawal-live and capacity views check their applicable maximum mark age, but funding redemption requests with open positions outside frozen mode additionally requires a Router-bound mark published in the current epoch. A positive capacity or true status is therefore not an execution-readiness guarantee after an epoch rolls. Already-funded claims remain payable from vault escrow independently of this funding gate. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IHousePool.sol:421-445`

`packages/perps/src/interfaces/IHousePool.sol:479-483`

`packages/perps/src/HousePool.sol:773-796`

`packages/perps/src/HousePool.sol:1252-1279`

`packages/perps/src/HousePool.sol:1304-1311`

`packages/perps/src/HousePool.sol:1638-1644`

HousePool's read paths can report LP withdrawals as live even when live-market settlement cannot yet execute. `isWithdrawalLive()` and `getPendingTrancheState()` only check degraded mode and cached-mark freshness through `_withdrawalsLive`, but `settleLpEpoch()` has an extra live-market requirement when open positions exist: the authenticated engine mark's `publishTime` must be at or after the current epoch boundary. Right after an epoch changes, the cached mark can still be fresh under the configured staleness policy and these views can report nonzero withdrawal capacity, while every actual settlement attempt still reverts until a new oracle basket crosses the epoch floor. This makes the IHousePool views materially optimistic for frontends and automation that rely on them without also consulting the dedicated settlement monitor.

Preconditions

- The pool has open positions and is not in frozen-oracle mode, so LP settlement must use the Router-bound atomic path (`packages/perps/src/HousePool.sol:785-795`).
- The cached engine mark is still fresh under the normal staleness policy, so `_withdrawalsLive(...)` returns true (`packages/perps/src/HousePool.sol:1638-1644`).
- The current or refreshed mark publish time is still before the new epoch boundary, so `settleLpEpoch()` reverts with `HousePool__MarkPriceStale()` (`packages/perps/src/HousePool.sol:792-794`).

Impact

Frontends, keeper bots, and other integrations that rely on `isWithdrawalLive()` or the pending withdrawal-cap views can show or act on redeemability that does not actually exist yet. This can cause failed settlement attempts, bad automation decisions, and misleading LP status around each epoch rollover.

Remediation

Expose the epoch-floor requirement in the IHousePool liveness views, or add a dedicated view that reports whether the current epoch can actually be settled under the live-market router path.

Low ACKNOWLEDGED

Integrators can overquote redeem dust that the vault will still reject

ID: LOW-17

> **Comment**
> LOW-17 — Addressed through explicit estimate semantics. Redemption/withdrawal estimates include the active frozen-oracle fee but do not enforce request minimums, balances, cooldowns or owner-specific eligibility. A below-minimum redemption is admitted only for the owner's entire available share balance. Indicative conversion remains available for dust amounts; admission retains the minimum and full-exit exception. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IHousePool.sol:536-540`

`packages/perps/src/TrancheVault.sol:1356-1368`

`packages/perps/src/TrancheVault.sol:1589-1595`

The tranche estimate surfaces advertise redeemable value below the enforced 1 USDC request floor. `estimateRedeemAssets()` and `estimateWithdrawShares()` always return a quote from the current virtual-share math, but `_requireMinimumRedeemSize()` later reverts partial redemption requests whenever that quoted asset amount is below `POOL.minTrancheDepositUsdc()` and the caller is not redeeming their full remaining balance. `IHousePool.minTrancheDepositUsdc()` and its NatSpec say nonbinding estimate views use the same floor when deciding whether a useful request or exact-share quote remains available, but the live estimate functions do not zero or flag these dust cases. Frontends can therefore quote a positive redeemable amount that the async redeem path still rejects as `WithdrawalTooSmall`.

Preconditions

- A user asks the vault for a partial redemption quote whose current `estimateRedeemAssets(shares)` is positive but below `POOL.minTrancheDepositUsdc()` (`packages/perps/src/TrancheVault.sol:1363-1368`, `packages/perps/src/TrancheVault.sol:1589-1595`).
- The user is not redeeming their full remaining balance, so the dust-exit exception does not apply (`packages/perps/src/TrancheVault.sol:1593-1595`).
- A frontend or integrator relies on the estimate surface as an actionable request quote because the interface documents nonbinding estimate views as sharing the same minimum floor (`packages/perps/src/interfaces/IHousePool.sol:536-540`).

Impact

Users and integrations can be shown a positive redeem quote that is not actually requestable. This causes avoidable revert paths and stale UI metadata around what portion of a position can be redeemed.

Remediation

Make the estimate surface apply the same 1 USDC floor as `_requireMinimumRedeemSize()` for non-terminal partial requests, or narrow the interface documentation so the minimum-floor guarantee no longer covers these estimate functions.

Low ACKNOWLEDGED

Keeper API docs can hide protection bounty forfeiture from liquidation callers

ID: LOW-18

> **Comment**
> LOW-18 — Addressed in the keeper API documentation. Liquidation first refunds cutoff-invalid opens, then forfeits remaining live order execution bounties and unpaid position-protection trigger bounties. The Engine-planned liquidation bounty is separately credited to the liquidator's clearinghouse account. The correction documents the existing protection-bounty treatment rather than changing its economics. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IPerpsKeeper.sol:54-59`

`packages/perps/src/router/OrderBountyAccounting.sol:11-27`

`packages/perps/src/PositionProtectionBook.sol:648-667`

`packages/perps/src/interfaces/IOrderRouterAccounting.sol:24-30`

Liquidation forfeits more than queued order bounties, but `IPerpsKeeper.executeLiquidation` documents only loss of remaining live execution bounties. The interface says that on liquidation only the remaining live execution bounties are forfeited, while the router implementation first calls `_forfeitReservedOrderBountiesOnLiquidation(account)`, which adds `positionProtectionBook.forfeitOnLiquidation(account)` to the forfeited amount before transferring it to the treasury. `getAccountReservations` and the router accounting docs already state that `executionBountyUsdc` includes unpaid protection-trigger reserves, so a keeper or monitoring system that trusts the keeper-facing ABI docs can materially understate how much value liquidation strips from the account.

Preconditions

- The liquidated account has an active protection with an unpaid trigger bounty recorded in `BountyKind.ProtectionTrigger` (`packages/perps/src/PositionProtectionBook.sol:230-236`, `packages/perps/src/PositionProtectionBook.sol:418-423`).
- A keeper, user, or integrator relies on `IPerpsKeeper.executeLiquidation` documentation to estimate the funds that liquidation will forfeit (`packages/perps/src/interfaces/IPerpsKeeper.sol:54-59`).

Impact

Liquidation callers and indexers can misprice or misreport liquidation consequences because the ABI-level docs omit protection-trigger bounty forfeiture. That is incorrect public behavior on an exposed API surface and can mislead operators about account loss.

Remediation

Update the keeper-facing liquidation docs to say that liquidation forfeits both remaining order bounties and unpaid protection-trigger bounties, matching router accounting terminology.

Low ACKNOWLEDGED

Integrator can overstate trader settlement after liquidation

ID: LOW-19

Comment
LOW-19 — Addressed. Liquidation preview documentation now defines settlementRetainedUsdc as the total projected account settlement after liquidation debits and any immediate payout. immediatePayoutUsdc is already included and must not be added again. Outstanding trader claims are separate, and retained settlement is not a withdrawal-capacity quote. The calculation is preserved; the corrected contract removes the double-counting interpretation. Fix: https://github.com/Plether-Fi/plether-core/commit/e4637a63ce9a6aa7508217b0dfa028e2d5fa8d05

Locations:

`packages/perps/src/libraries/CfdEnginePlanLib.sol:1444-1447`

`packages/perps/src/CfdEngineLens.sol:579-586`

`packages/perps/src/interfaces/ICfdEngineTypes.sol:370-376`

The liquidation preview reports the same immediate payout in two fields. `CfdEnginePlanLib._applyLiquidationSettlement()` sets `delta.settlementRetainedUsdc` to the post-debit settlement balance plus `freshTraderPayoutUsdc` when the payout is immediate, and `CfdEngineLens._previewLiquidation()` also exposes that same amount again as `immediatePayoutUsdc`. However `ICfdEngineTypes.LiquidationPreview` documents `settlementRetainedUsdc` as only the existing settlement left with the trader. An integrator that follows the public interface and adds `settlementRetainedUsdc + immediatePayoutUsdc` will double count serviceable liquidation proceeds and misreport the trader's post-liquidation balance.

Preconditions

- A liquidation preview has `freshPayoutIsImmediate == true`, so `_applyLiquidationSettlement()` adds `freshTraderPayoutUsdc` into `settlementRetainedUsdc` (`packages/perps/src/libraries/CfdEnginePlanLib.sol:1433-1447`).
- An off-chain consumer reads `ICfdEngineTypes.LiquidationPreview` through the public lens and interprets `settlementRetainedUsdc` according to the interface comment while also using `immediatePayoutUsdc` as a separate field (`packages/perps/src/CfdEngineLens.sol:579-586`, `packages/perps/src/interfaces/ICfdEngineTypes.sol:370-376`).

Impact

Liquidation dashboards, risk monitors, indexers, or downstream accounting code can overstate a trader's final clearinghouse balance after liquidation. That breaks off-chain settlement accounting and any automation that relies on the preview to reconcile balances or present user-facing liquidation outcomes.

Remediation

Make the interface semantics match the implementation. Either redefine `settlementRetainedUsdc` everywhere as the final post-liquidation settlement balance including any immediate payout, or stop adding the immediate payout into that field and leave it solely in `immediatePayoutUsdc`.

Low ACKNOWLEDGED

monitor callers can see preview accounting as available while terminal deficit data is invalid

ID: LOW-20

Comment

LOW-20 — Addressed with independent availability flags. cachedPreviewAvailable now refers only to the four cached tranche principal/withdrawal preview fields. liquidityPreviewAvailable separately covers freeUsdc, withdrawalReservedUsdc and cachedPreviewTerminalDeficitUsdc, and is true only after a successful, well-formed liquidity read. A false flag means unavailable, not measured zero. Failure masks and observation completeness remain fail-closed; monitor schema/configuration/observation domains are V6. Fix: https://github.com/Plether-Fi/plether-core/commit/190ba2368f3056e1130e2e575157a0b08ac3c0ac

Locations:

`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:200-234`

`packages/perps/src/SettlementMonitorLensSidecar.sol:155-179`

A monitor consumer can trust `SettlementAccounting.cachedPreviewAvailable` even when one of the advertised cached-preview fields is known-bad. `SettlementMonitorLensSidecar._buildSettlementAccounting` marks `cachedPreviewAvailable = true` as soon as `HousePool.getPendingTrancheState()` decodes, but `cachedPreviewTerminalDeficitUsdc` comes from a separate `HousePool.getPoolLiquidityView()` read. If that liquidity-view read fails or decodes invalid booleans, the sidecar sets the `PoolAccountingPreview` dependency bit but still leaves `cachedPreviewAvailable` true and returns zero for `cachedPreviewTerminalDeficitUsdc`. The interface describes one mixed cached-state and reconcile-preview accounting bundle behind a single availability flag, so callers that gate only on `cachedPreviewAvailable` can ingest a partial preview snapshot and understate the preview terminal deficit.

Preconditions

- The pending-tranche preview call succeeds, so `_buildSettlementAccounting` sets `cachedPreviewAvailable = true` from `HousePool.getPendingTrancheState()` (`packages/perps/src/SettlementMonitorLensSidecar.sol:167-179`).
- The separate liquidity preview call fails or decodes invalid boolean flags, so `_buildSettlementAccounting` cannot populate `freeUsdc`, `withdrawalReservedUsdc`, or `cachedPreviewTerminalDeficitUsdc` from `HousePool.getPoolLiquidityView()` and sets the `PoolAccountingPreview` dependency bit instead (`packages/perps/src/SettlementMonitorLensSidecar.sol:155-165`).

- An off-chain monitor or integration treats `cachedPreviewAvailable` as the readiness gate for the cached preview bundle described by `SettlementAccounting` (`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:200-234`).

Impact

Preview consumers can ingest a mixed-source partial snapshot and read `cachedPreviewTerminalDeficitUsdc` as zero or stale-default while believing cached preview accounting is available. This can hide preview insolvency from dashboards, alerts, or automated settlement monitors until they also inspect the dependency bit.

Remediation

Only set `cachedPreviewAvailable` after both cached-preview reads succeed, or split the flag into per-source availability bits so `cachedPreviewTerminalDeficitUsdc` has its own readiness signal.

Low ACKNOWLEDGED

Monitor can block complete observations for cached settlement when the oracle view is unreadable

ID: LOW-21

Comment

LOW-21 — Full-observation fail-closed behavior retained. observationComplete attests that the complete observation, including the reported oracle section, was read successfully; it is deliberately stricter than whether one particular cached-mark settlement route can execute. An unreadable oracle can therefore leave completeObservationDigest unset even when the route-specific status does not require atomic oracle refresh. Consumers needing action readiness should use the execution-path status and its blockers rather than reinterpret a partial observation as complete. Reviewed implementation: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/SettlementMonitorLens.sol#L268-L278

Locations:

`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:9-19`

`packages/perps/src/SettlementMonitorLens.sol:239-279`

Operators can lose a complete settlement observation digest even when the next LP settlement path does not need an oracle refresh. `SettlementMonitorLens.getSettlementObservation()` always calls `SIDECAR.getOracleStatus(...)` and later requires `observation.oracle.dependencyFailureMask == 0` before setting `observationComplete` and `completeObservationDigest`. That requirement is unconditional, while `getSettlementStatus()` only treats oracle status as required when `requiredExecutionPath == AtomicOracleRefresh`, and HousePool can settle cached-mark or no-position states without reading `getLatestPoolReconcilePrice()`. If the active oracle's view ABI is unreadable or incompatible, cached-mark and no-work observations are marked incomplete even though the settlement route is otherwise fully known.

Preconditions

- `SettlementStatus.requiredExecutionPath` resolves to `CachedMark` or `NoMaturedWork`, so HousePool would not need an atomic oracle refresh (`packages/perps/src/interfaces/SettlementMonitorViewTypes.sol:9-19`, `packages/perps/src/SettlementMonitorLens.sol:1317-1335`).
- The bound oracle's `getLatestPoolReconcilePrice()` view is unreadable to the monitor, for example because the oracle ABI is malformed, truncated, or otherwise incompatible (`packages/perps/src/SettlementMonitorLens.sol:245`, `packages/perps/src/SettlementMonitorLens.sol:273-276`).
- An operator or integrator relies on `observationComplete` or `completeObservationDigest` to attest that the current settlement state is fully known before acting.

Impact

Monitoring and signing flows can be needlessly blocked on cached-settlement states. Off-chain automation loses a complete observation digest even when the next LP settlement attempt is fully determined without any oracle dependency, which is an availability defect for monitor consumers.

Remediation

Only require oracle dependency success inside `observationComplete` when `requiredExecutionPath == AtomicOracleRefresh`; cached-mark and no-work observations should ignore unrelated oracle read failures.

Low ACKNOWLEDGED

Governance cannot use the documented non-dilutive recapitalization path

ID: LOW-22

Comment

LOW-22 — Addressed. CfdEngine.recapitalizePool(amountUsdc) is owner-only and non-reentrant. It checkpoints carry before funding changes pool depth, transfers approved USDC directly from the owner to the configured pool, verifies the exact balance increase, and records the corresponding Recapitalization inflow atomically. A failed transfer, receipt check, or accounting hook reverts the entire operation. No LP shares are minted.

Funding remains available during stale marks, frozen markets, pool pause, and settlement hold. Allocation is deferred to normal reconciliation under existing senior-recovery priority; residual value without a claimant path remains unassigned. The call does not automatically clear degraded mode or other containment controls.

Fix:
https://github.com/Plether-Fi/plether-core/commit/395f85dffe7b961c34c054895ec631c24bd0cc7e

Locations:

`packages/perps/src/HousePool.sol:601-625`

`packages/perps/src/interfaces/IHousePool.sol:304-317`

The pool advertises an engine-only `Recapitalization` inflow mode, but the in-scope contracts do not expose any production path that can actually invoke it for recovery. `HousePool.recordClaimantInflow()` accepts `ClaimantInflowKind.Recapitalization` only from `ENGINE`, queues it separately from ordinary revenue, and the docs describe this as the non-dilutive way to restore senior claimant equity. However, a full in-scope code search shows no public or admin engine flow that calls `recordClaimantInflow(..., Recapitalization, ...)`; all live call sites route only `Revenue`. In practice, governance cannot perform the documented recapitalization without either adding an out-of-band engine change or using a different path such as minting new LP shares, which changes recovery economics.

Preconditions

- The protocol needs to restore claimant equity without minting new LP shares, using the documented recapitalization path (`packages/perps/src/interfaces/IHousePool.sol:304-317`).
- Governance is operating only the in-scope deployed contracts.

Impact

The documented recapitalization recovery path is unavailable in practice. Recovery tooling and operator runbooks can be misleading, and the only in-scope alternatives are economically different actions such as dilutive seeding or bootstrap assignment.

Remediation

Add an explicit engine-governed recapitalization entrypoint that transfers the cash and then calls `recordClaimantInflow(..., Recapitalization, ...)`, or remove the unsupported mode from the live contract surface and documentation.

Low ACKNOWLEDGED

Integrators can misread senior withdrawal capacity during settlement holds

ID: LOW-23

Comment
LOW-23 — Addressed in HousePool and interface documentation. getMaxSeniorWithdraw returns zero during the settlement hold, degraded mode or applicable mark staleness. It is stored-state, demand-neutral capacity capped by free cash and senior principal, not evidence of matured demand or completed reconciliation. Execution with open positions outside frozen mode also requires a current-epoch Router-bound mark. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/HousePool.sol:1218-1225`

`packages/perps/src/HousePool.sol:1638-1643`

`packages/perps/src/interfaces/IHousePool.sol:421-426`

`getMaxSeniorWithdraw()` returns zero when LP settlement is paused, but its public documentation says zero is only caused by degraded mode or stale marks. The implementation delegates to `_withdrawalsLive()`, which includes `!lpEpochSettlementPaused`, so an emergency settlement hold suppresses the reported senior funding cap even when the engine is healthy and the mark is fresh. The junior-cap docs and `isWithdrawalLive()` docs both mention the settlement hold, so the senior-cap comment is the outlier. This is a semantic mismatch for frontends and integrators that use the NatSpec as the API contract for why `getMaxSeniorWithdraw()` can suddenly drop to zero.

Preconditions

- `lpEpochSettlementPaused` is true (`packages/perps/src/HousePool.sol:142-145`, `packages/perps/src/HousePool.sol:387-404`).
- A frontend, lens, or integrator interprets a zero `getMaxSeniorWithdraw()` result using the documented causes rather than the actual implementation.
- No special deployment parameters are required.

Impact

Off-chain consumers can misclassify a settlement hold as a degraded-mode or stale-mark condition and make the wrong operational decision or surface the wrong remediation to users.

Remediation

Update the NatSpec for `getMaxSeniorWithdraw()` and the interface to state that LP settlement holds also force the reported capacity to zero, matching `_withdrawalsLive()`.

Low ACKNOWLEDGED

Pauser can brick emergency pause when no order has ever been committed

ID: LOW-24

Comment

LOW-24 — False positive; no code change required. nextCommitId is initialized to 1, so pausing before any order is committed computes currentTail = 1 - 1 = 0 without underflow. The cutoff remains zero and pause succeeds. The audited revision already includes test_Pause_EmptyQueueReaffirmsZeroRiskOffCutoff(), which calls pause(), expects RiskOffActivated(0, 0), and asserts nextCommitId == 1 and riskOffOrderCutoff == 0.

Audited initialization: https://github.com/Plether-Fi/plether-core/blob/6d6622f2871dbdfc4904c20da2e1f7662f5763b6/packages/perps/src/router/OrderValidation.sol#L12-L13
Existing regression: https://github.com/Plether-Fi/plether-core/blob/6d6622f2871dbdfc4904c20da2e1f7662f5763b6/packages/perps/test/perps/spec/admin/TimelockPause.t.sol#L795-L804

Locations:

`packages/perps/src/OrderRouterAdmin.sol:234-242`

`packages/perps/src/router/OrderValidation.sol:12-13`

`packages/perps/src/interfaces/IOrderRouterAdminHost.sol:7-9`

The first emergency pause reverts before recording any cutoff if the router has never accepted an order. `OrderRouterAdmin.pause()` computes `currentTail = router.nextCommitId() - 1`, but `nextCommitId` starts at 1 and only increments on successful commits. On a fresh deployment with zero committed orders, the subtraction underflows and the whole pause transaction reverts, so the owner or pauser cannot activate risk-off exactly when the queue is still empty.

Preconditions

- The router is in its initial state with `nextCommitId == 1`, meaning no order has been committed yet (`packages/perps/src/router/OrderValidation.sol:12-13`).
- The owner or configured pauser calls `OrderRouterAdmin.pause()` to activate emergency risk-off (`packages/perps/src/OrderRouterAdmin.sol:234-242`).

Impact

Emergency pause is unavailable on an empty queue. Governance cannot proactively freeze new risk before the first order arrives, which is an availability failure in the documented containment path.

Remediation

Handle the empty-queue case explicitly, for example by treating `nextCommitId == 1` as a tail of zero before subtracting.

Low ACKNOWLEDGED

Receipt verifiers can reject expired and config-mismatch evidence

ID: LOW-25

Comment
LOW-25 — Clarified with the receipt-context fixes. No-price terminal evidence means no execution price/oracle timestamp and no Engine execution; it does not require cached neutral-mark or gross-pool-depth context to be zero. Expiry, configuration mismatch and risk-off validation distinguish those contextual fields from execution evidence. The documentation now explicitly describes retained cleanup context, and expiry/mismatch regressions verify the authenticated receipts. Verifiers must use the terminal-reason-specific rules or Book.verifyReceipt rather than imposing a blanket zero-context rule. Clarification and receipt regressions: https://github.com/Plether-Fi/plether-core/commit/3cbdf17353419967dace164fbd432fed4e70e3f9

Locations:

`packages/perps/src/interfaces/IOrderLifecycleBook.sol:143-181`

`packages/perps/src/OrderLifecycleBook.sol:627-637`

`packages/perps/src/OrderLifecycleBook.sol:740-749`

`packages/perps/src/OrderRouterExecutionSidecar.sol:372-397`

`packages/perps/src/OrderRouterExecutionSidecar.sol:913-920`

`packages/perps/src/OrderRecoverySidecar.sol:99-106`

The lifecycle book documents `Expired` and `ConfigMismatch` receipts as terminal outcomes with no price evidence, but the execution sidecar can finalize those receipts with contextual mark data filled in. When an execution attempt finds the order expired or the pinned config hash changed, `OrderRouterExecutionSidecar` routes to `_settleNonEngine()`, which delegates the full `ItemRequest` into recovery. Recovery copies `neutralMarkPrice` and `poolDepthUsdc` from that request into the terminal receipt, while `OrderLifecycleBook` validates those reasons through `_validateNoPriceEvidence()`, a helper that ignores both fields and only checks `executionMode`, `priceSource`, `executionPrice`, `oraclePublishTime`, and `priceReachedEngine`. This means the on-chain canonical receipt shape includes extra mark context that the documented no-price-evidence rule does not describe, so off-chain verifiers that implement the stricter documented interpretation can reject authentic expired or config-mismatch receipts.

Preconditions

- An order is finalized through the non-engine expired or config-mismatch path reached from an execution attempt (`packages/perps/src/OrderRouterExecutionSidecar.sol:372-397`).

- The `ItemRequest` for that attempt carries `neutralMarkPrice` and `poolDepthUsdc`, which the execution sidecar normally preloads from the current engine mark and pool state (`packages/perps/src/OrderRouterExecutionSidecar.sol:913-920`).
- An off-chain verifier or indexer reconstructs receipt invariants from the lifecycle-book interface/comments rather than replaying the exact sidecar implementation (`packages/perps/src/interfaces/IOrderLifecycleBook.sol:143-181`).

Impact

Authentic expired or config-mismatch receipts can be rejected or misclassified by external history and monitoring tooling, breaking receipt verification and order-history reconstruction off-chain.

Remediation

Make the documented invariant and emitted receipt shape match. Either zero `neutralMarkPrice` and `poolDepthUsdc` for no-price-evidence terminal reasons, or explicitly document and validate that those contextual fields may remain populated even when `priceSource` is `None`.

Low ACKNOWLEDGED

Trader can be blocked from opening by under-funded protocol fee top-ups

ID: LOW-26

Comment

LOW-26 — Addressed while retaining partial protocol-fee funding. The open planner now projects the fee actually credited: the portion funded directly by trade cost plus a top-up capped by post-open settlement-free pool cash. It includes updated directional liabilities, the settlement buffer, trader claims and other protected reserves, including newly queued trading revenue, using shared reserve arithmetic. Solvency admission therefore no longer assumes that an unaffordable full fee leaves the pool when settlement would collect only the affordable portion. An unfunded fee remainder does not become a new treasury claim; this preserves the existing partial-funding policy rather than guaranteeing the full quoted fee. Fix: https://github.com/Plether-Fi/plether-core/commit/8feda6c83fcec0feb3229712e60d65436a42cdc6

Locations:

`packages/perps/src/libraries/CfdEnginePlanLib.sol:684-735`

`packages/perps/src/CfdEngineSettlementSidecar.sol:436-452`

`packages/perps/src/CfdEngineSettlementSidecar.sol:881-909`

Open planning can reject a position even when the live settlement would preserve the pool buffer, because the solvency preview always subtracts the full execution fee from pool assets. In `_applyOpenSettlementBufferAfterPlan()`, the planner uses `physicalAssetsDeltaUsdc = delta.tradeCostUsdc - executionFeeUsdc`, which is equivalent to assuming the protocol fee shortfall is fully topped up from HousePool cash. Live settlement does not do that. `_settleProtocolFeeTopUp()` caps the pool-funded top-up by currently free pool cash after trader claims, and may fund less than the remaining fee shortfall. When an open has negative VPI that only partially offsets the fee, and pool cash cannot fund the full remaining fee, the actual post-settlement asset change is better than the planner assumes. The order can therefore fail `SOLVENCY_EXCEEDED` in planning even though the live mutation would leave more effective assets than the planner modeled.

Preconditions

- The open has `tradeCostUsdc > 0` but `tradeCostUsdc < executionFeeUsdc`, so negative VPI partially offsets the execution fee (`packages/perps/src/libraries/CfdEnginePlanLib.sol:330-332`, `packages/perps/src/libraries/OpenAccountingLib.sol:133-135`).
- HousePool does not have enough free cash after trader claims to fund the full fee shortfall during `_settleProtocolFeeTopUp()` (`packages/perps/src/CfdEngineSettlementSidecar.sol:895-908`).

- The settlement-buffer check is tight enough that subtracting the full execution fee fails while subtracting only the actually fundable top-up would pass (`packages/perps/src/libraries/CfdEnginePlanLib.sol:710-729`).

Impact

Valid opens can be rejected during low-free-cash periods even though the live settlement path would keep the protocol within its intended solvency limits. This is an avoidable availability loss for traders and unnecessarily tightens admission during stressed conditions.

Remediation

Model the open-path physical asset delta using the actually fundable protocol-fee top-up, not the full execution fee. The planner should mirror `_settleProtocolFeeTopUp()` or explicitly cap the assumed top-up by the same free-cash calculation.

Low ACKNOWLEDGED

Anyone can force zero-share junior fee accrual to permanently skip completed hours

ID: LOW-27

Comment

LOW-27 — Accepted rounding behavior. With a nonzero junior maintenance fee, a small share supply can make the fee for completed hours round down to zero shares while the fee checkpoint still advances. Those completed hours are not carried forward, so repeated checkpoints can undercollect the fee. This behavior is accepted; no fractional-share accumulator or deferral of zero-share checkpoints is planned.

Documented in SECURITY.md, 'Accepted rounding behavior': https://github.com/Plether-Fi/plether-core/commit/f6e0f156bf5b170aa383c1df533eff9cb27ec157

Locations:

`packages/perps/src/TrancheVault.sol:411-445`

`packages/perps/src/TrancheVault.sol:308-327`

`packages/perps/src/libraries/JuniorMaintenanceFeeMathLib.sol:63-74`

Anyone who can trigger a junior-vault mint or burn at a very small supply can erase already elapsed maintenance-fee time without paying any fee shares. `_checkpointMaintenanceFee()` always advances `maintenanceFeeCheckpointBoundary` to the current completed hour, even when `JuniorMaintenanceFeeMathLib.pendingFeeShares()` rounds down to zero. Because this checkpoint runs automatically on every mint and burn in `_update()`, an attacker can wait until one or more chargeable hours have elapsed while the junior raw supply is tiny, then trigger a dust redeem funding, deposit finalization, bootstrap mint, or any other burn/mint boundary that produces `feeShares == 0`. The boundary is still moved forward, so those elapsed hours are forgiven permanently instead of accruing until supply is large enough to mint at least one share. This lets actors who control timing around a near-empty junior tranche reduce or eliminate governance-configured maintenance-fee dilution.

Preconditions

- The affected vault is the junior tranche with `maintenanceFeeAprBps != 0`, and its raw share supply is small enough that `pendingFeeShares(rawSupply, aprBps, chargeableHours)` rounds down to zero for one or more completed hours (`packages/perps/src/TrancheVault.sol:411-445`, `packages/perps/src/libraries/JuniorMaintenanceFeeMathLib.sol:63-74`).
- Any caller can cause a mint or burn boundary after those hours elapse, because `_update()` checkpoints the fee on every nonzero mint or burn (`packages/perps/src/TrancheVault.sol:308-327`).
- The impact depends on a nonzero junior maintenance fee configuration (`packages/perps/src/TrancheVault.sol:355-395`).

Impact

Completed fee-accrual hours can be wiped out without minting any fee shares. The configured junior maintenance fee under-collects, reducing dilution of junior holders and shorting the designated fee recipient whenever supply is small enough for rounding to zero.

Remediation

Only advance `maintenanceFeeCheckpointBoundary` when nonzero fee shares are minted, or accumulate sub-share fee debt in storage so zero-share checkpoints are carried forward instead of discarded.

Low ACKNOWLEDGED

Liquidation preview callers can misclassify risk-off-refunded accounts

ID: LOW-28

Comment
LOW-28 — Addressed. previewRouterLiquidation models the Router's preceding cutoff-invalid open refunds, including released committed margin and relevant order/protection bounty reservations, before projecting carry and liquidation eligibility. Regressions compare it with actual cleanup and liquidation, including refunds that restore carry solvency. The original previewLiquidation remains an Engine-only diagnostic. The Router-aware result is a state-specific projection; it does not simulate oracle ingestion policy, other accounts in a batch, callbacks or gas outcomes. Fix: https://github.com/Plether-Fi/plether-core/commit/8b7cc430649d38d821aebafdb6cfe37c79a2ead6

Locations:

`packages/perps/src/CfdEngineLens.sol:208-215`

`packages/perps/src/CfdEngineLens.sol:554-568`

`packages/perps/src/router/OrderLiquidationHandler.sol:55-63`

`packages/perps/src/OrderRouter.sol:316-335`

`packages/perps/src/MarginClearinghouse.sol:652-680`

Off-chain keepers and dashboards can classify an account as liquidatable even though the router will first refund risk-off opens and then skip liquidation. `CfdEngineLens.previewLiquidation()` and `isLiquidatableAt()` build their snapshot from the current clearinghouse reservations and only model pending execution-bounty forfeiture in memory. In the actual router path, `OrderLiquidationHandler` refunds every cutoff-invalid open before the final eligibility check, and `MarginClearinghouse.releaseInvalidatedOrderReserves()` releases the committed open margin and refundable bounty classifications back to free settlement. If that cleanup restores solvency, the batch item returns `type(uint256).max` and emits `SkippedSolvent` instead of liquidating. Callers that treat the lens result as the exact next router outcome can report the wrong liquidation status or submit an unprofitable keeper attempt.

Preconditions

- The account has one or more pending open orders at or below the active risk-off cutoff, so router cleanup will refund them before liquidation (`packages/perps/src/router/OrderLiquidationHandler.sol:55-58-`, `packages/perps/src/OrderRouter.sol:316-335`).

- Those refunds release enough committed margin or reserved-settlement bounty classifications to make the account solvent again when `engineLens.isLiquidatableAt` is re-run inside the router (`packages/perps/src/MarginClearinghouse.sol:652-680`, `packages/perps/src/router/OrderLiquidationHandler.sol:55-58`).
- A caller relies on `previewLiquidation` or `isLiquidatableAt` alone, without first simulating the same risk-off refund sequence (`packages/perps/src/CfdEngineLens.sol:186-215`).

Impact

Integrations can report the wrong liquidation status and predict a liquidation that the router will intentionally skip. This misleads permissionless keepers, monitoring systems, and users about whether an account will actually be liquidated in the next batch.

Remediation

Add a dedicated preview that mirrors the router's full liquidation sequence, including risk-off refund cleanup before the final eligibility check, or document and expose the refund-adjusted result separately so callers do not treat `previewLiquidation` as the exact batch outcome.

Low ACKNOWLEDGED

Keeper tooling can misclassify stale trigger data as same-block protection failure

ID: LOW-29

Comment

LOW-29 — Addressed. Trigger admission now has separate errors: OrderRouter__SameBlockTrigger() for a trigger in the arming execution block, and OrderRouter__TriggerPublishTimeNotAdvanced() when the oracle publication has not advanced past armedPublishTime. A later-block caller using an old publication therefore receives the publication-specific error. The block check runs first, so a call violating both conditions reports SameBlockTrigger. Both guards remain enforced.

Fix:
https://github.com/Plether-Fi/plether-core/commit/8f6cfd3d84d1289aa1209e3b08a06e9bfdf38a0d

Locations:

`packages/perps/src/interfaces/IOrderRouterErrors.sol:144-145`

`packages/perps/src/PositionProtectionBook.sol:465-466`

`packages/perps/test/perps/spec/trader/PositionProtection.t.sol:307-315`

`OrderRouter__SameBlockTrigger` is documented as a same-block guard, but protection triggering also uses it when the oracle publish time is not newer than the arming timestamp. `PositionProtectionBook._prepareActivation()` reverts with `OrderRouter__SameBlockTrigger()` for either `block.number <= protection.armedBlock` or `publishTime <= protection.armedAt`. The test suite explicitly expects the same selector for both cases. Keepers and monitoring systems cannot tell whether they should simply wait for another block or instead fetch fresher oracle data, so the router's machine-readable error surface gives the wrong remediation signal.

Preconditions

- A protection is armed and a keeper attempts `triggerPositionProtection` through the router path (`packages/perps/src/PositionProtectionBook.sol:435-479`).
- Either the call happens in the arming block or the supplied oracle publish time is not later than `armedAt` (`packages/perps/src/PositionProtectionBook.sol:465-466`).

Impact

Off-chain keepers, dashboards, and incident runbooks that key on `OrderRouter__SameBlockTrigger` cannot distinguish block-ordering failures from stale-price failures and can take the wrong retry action.

Remediation

Use a distinct custom error for stale or non-advancing trigger publish times, and reserve `OrderRouter__SameBlockTrigger` for the actual same-block condition.

Low ACKNOWLEDGED

Lens caller can receive stale post-VPI accrual in invalid open previews

ID: LOW-30

Comment

LOW-30 — Partial invalid-preview diagnostics retained. When valid is false, the invalid reason is authoritative; economics and projected fields can be only partially populated at the point planning stopped. A computed VPI charge alongside unchanged postVpiAccrued on a later rejection is not a successful post-trade state and must not be used as one. No hypothetical lifetime-accrual state is synthesized for rejected orders. This limitation is explicit in OpenPreview's contract. Reviewed contract: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/interfaces/ICfdEngineTypes.sol#L311-L314

Locations:

`packages/perps/src/CfdEngineLens.sol:252-268`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:322-365`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:382-386`

Some invalid open previews report a signed VPI trade charge or rebate but leave `postVpiAccrued` at the pre-trade value. The planner computes `openState.vpiUsdc` inside `_planOpenState()`, then can still fail later on reserve-funding checks such as `MARGIN_DRAINED_BY_FEES`, `VPI_REBATE_RESERVE_UNFUNDED`, or `LIQUIDATION_RESERVE_UNFUNDED` before it copies that value into `delta.posVpiAccruedDelta`. `_openPreviewFromPlan()` unconditionally returns `preview.vpiUsdc = delta.openState.vpiUsdc` and, for every non-numeric invalid result, also returns `preview.postVpiAccrued = snap.position.vpiAccrued + delta.posVpiAccruedDelta`. On those late-invalid paths `delta.posVpiAccruedDelta` is still zero, so the preview shows a nonzero VPI trade effect and an unchanged lifetime accrual in the same response.

Preconditions

- `planOpen()` reaches `_planOpenState()`, so `delta.openState.vpiUsdc` has already been computed (`packages/perps/src/libraries/CfdEnginePlanLib.sol:322-332`).
- The open then fails on a later pre-assignment path such as insufficient free equity, underfunded VPI reserve, or underfunded liquidation reserve before `delta.posVpiAccruedDelta` is assigned (`packages/perps/src/libraries/CfdEnginePlanLib.sol:341-365`, `packages/perps/src/libraries/CfdEnginePlanLib.sol:382-386`).
- A user or integrator consumes the invalid `previewOpen` or zero-capacity `quoteMaxOpen` response and reads both `vpiUsdc` and `postVpiAccrued` from the same preview object (`packages/perps/src/CfdEngineLens.sol:98-120`, `packages/perps/src/CfdEngineLens.sol:252-268`).

Impact

Invalid open diagnostics can contradict themselves and understate the projected lifetime VPI balance. Off-chain risk tooling and frontends that use these fields for user guidance or retry logic can misreport the account's post-trade VPI state.

Remediation

For invalid previews, either derive `postVpiAccrued` directly from `openState.vpiUsdc` once that value exists, or leave `postVpiAccrued` unset on every path where `delta.posVpiAccruedDelta` has not been populated.

Low ACKNOWLEDGED

Trader can lose a capped bounty on undersized opens before the first mark update

ID: LOW-31

Comment
LOW-31 — Dismissed from remediation; no code change. The project accepts the initial-mark admission/execution mismatch and the associated bounded bounty exposure on undersized opens before the first mark update. No change to the fallback-price or first-open policy is planned.

Disposition recorded in PR: https://github.com/Plether-Fi/plether-core/pull/122

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:392-438`

`packages/perps/src/HousePool.sol:514-529`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:494-497`

`packages/perps/src/OrderRouter.sol:285-312`

A trader can submit an open order that is below the governed minimum size before the engine has ever cached a mark, and the router can accept it using the 1e8 fallback price. When the order later executes against the real oracle price, the engine can reject it as POSITION_TOO_SMALL, but the reserved opening bounty is still paid to the keeper. This only applies during the initial no-mark state and the loss is capped by the open-order bounty configuration.

Preconditions

- The protocol is live but the engine still has no cached mark, so `engine.lastMarkPrice() == 0` and `_commitReferencePrice()` substitutes `1e8` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:430-438`). Trading can already be active because `HousePool.activateTrading()` does not require any mark initialization (`packages/perps/src/HousePool.sol:514-529`).
- A trader submits an open/increase order while the real executable oracle price is below the `1e8` fallback, so the order's true notional is below `minOpenNotionalUsdc` even though `lots * 1e8` passes the router's commit-time check (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:392-406`).
- A keeper later executes the order against the real oracle price, and the planner returns `POSITION_TOO_SMALL`, which is a terminal user-invalid failure (`packages/perps/src/libraries/CfdEnginePlanLib.sol:494-497`).

Impact

The router admits orders that violate the protocol's minimum-open-size policy. Those orders occupy FIFO capacity until executed or cleaned up, and when they execute they can terminally fail while still paying the reserved open-order bounty to the keeper, directly debiting the trader's free settlement.

Remediation

Do not use a synthetic fallback price for open-commit validation. Reject opens when no cached mark exists, or fetch a validated live mark and use that actual price for the minimum-notional and preview checks.

Low ACKNOWLEDGED

Preview callers can misread partial close failures

ID: LOW-32

> **Comment**
> LOW-32 — Addressed. Close-invalid-reason documentation now identifies PartialCloseUnderwater as a retained legacy ABI value that the current preview does not emit. Uncollectible partial-close action charges use PartialActionChargeUncollectible; residual maintenance failure uses PartialCloseUnhealthy. Enum values are preserved for compatibility. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/CfdTypes.sol:66-84`

`packages/perps/src/CfdEnginePlanTypes.sol:261-278`

`packages/perps/src/CfdEngineLens.sol:364-385`

Close preview exposes two public invalid-reason enum values for the same partial-close funding failure, but only one can ever be returned. `CfdTypes.CloseInvalidReason.PartialCloseUnderwater` is documented as the reason when a requested partial close cannot collect every assessed action charge, while the planner reports `PARTIAL_ACTION_CHARGE_UNCOLLECTIBLE` and `CfdEngineLens._previewClose()` maps that only to `PartialActionChargeUncollectible`. Clients that rely on the public enum comments or older reason mapping can treat a routine invalid partial close as an unknown or different condition.

Preconditions

- A caller uses the public close preview surface and relies on `ICfdEngineTypes.ClosePreview.invalidReason` as the authoritative rejection reason (`packages/perps/src/interfaces/ICfdEngineTypes.sol:242-293`).
- The requested partial close fails because not every assessed action charge can be collected, causing the planner to return `PARTIAL_ACTION_CHARGE_UNCOLLECTIBLE` (`packages/perps/src/libraries/CfdEnginePlanLib.sol:839-841`).
- An integrator or downstream decoder still expects the retained legacy enum member `CloseInvalidReason.PartialCloseUnderwater` to represent that class of failure (`packages/perps/src/CfdTypes.sol:74-82`).

Impact

Public preview consumers can mis-handle invalid partial closes, because the authoritative preview reason no longer uses the legacy ABI-compatible slot they were told still existed. This can break UI messaging, client-side routing, or any off-chain policy that branches on the old enum value.

Remediation

On the public `ClosePreview` surface, translate `PARTIAL_ACTION_CHARGE_UNCOLLECTIBLE` back to the retained `PartialCloseUnderwater` enum value, or make a breaking ABI revision that removes the legacy member instead of leaving it apparently supported.

Low ACKNOWLEDGED

Deployer can brick the router by binding a non-contract engine lens or house pool

ID: LOW-33

Comment

LOW-33 — Addressed. Router construction requires deployed lens and pool contracts. Both lens binding getters, engine() and engineContract(), must identify the configured Engine; the pool's ENGINE() must match, and totalAssets() must be readable. This rejects EOAs, missing interfaces, and mismatched bindings before the immutable dependencies are accepted.

These probes establish the required reported bindings, not complete contract identity or correctness. The constructor does not require Engine.pool() to be set already, preserving the supported deployment order where Engine-to-pool wiring is finalized later.

Fix:
https://github.com/Plether-Fi/plether-core/commit/c689573fa08927ba03596546952d6d3eec2dbba2

Locations:

`packages/perps/src/router/OrderOracleExecution.sol:35-41`

`packages/perps/src/router/OrderValidation.sol:33-38`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-427`

The router constructor validates the Plether oracle address thoroughly, but it only checks that `engineLens` is nonzero and does not independently verify that either `engineLens` or `housePool` is deployed code with the expected interface. `OrderOracleExecution` stores both immutably, and later public flows call them directly through typed interfaces. If deployment accidentally supplies an externally owned account or an incompatible contract, ordinary open commits revert when `OrderLiquidationBatchLogic` calls `previewOpenRevertCode`, and risk-off or liquidation paths revert when the router reads `housePool.totalAssets()`. Because this stack is non-upgradeable, the bad wiring permanently bricks that router deployment and requires a fresh redeploy.

Preconditions

- A deployment or fresh-stack rollout passes a non-contract or interface-incompatible address as `_engineLens` or `_housePool` to the router constructor (`packages/perps/src/router/OrderOracleExecution.sol:29-41`).
- Users or keepers later use open-order commit, risk-off cleanup, or liquidation entrypoints that dereference those immutable bindings (`packages/perps/src/router/OrderValidation.sol:33-38`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-427`).

Impact

A mistaken deployment can permanently disable core router functions for that stack. Opens can no longer be committed, and cleanup or liquidation flows can revert, leaving the deployment unavailable until operators replace it with a fresh router stack.

Remediation

Validate `_engineLens` and `_housePool` the same way the constructor validates the oracle: require deployed code and perform a minimal interface sanity check before accepting the addresses.

Low ACKNOWLEDGED

Integrators can overstate close carry because the evaluator reports pending carry after collection

ID: LOW-34

> **Comment**
> LOW-34 — Addressed by documenting assessed carry separately from collected cash. The assessment's carry field includes carry assessed for the action and any commitment-time collection added to the assessment; a full close can waive an uncollectible remainder. It must not be interpreted as an additional payable amount or as proof that all assessed carry was collected. No new carry-only receipt breakdown is introduced. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/CfdOrderPolicyEvaluatorBase.sol:403-476`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:771-772`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:958-959`

`packages/perps/src/OrderRouterExecutionSidecar.sol:773-782`

`packages/perps/src/OrderV3Types.sol:186-208`

`packages/perps/src/OrderV3Types.sol:221-243`

Close assessments expose a `carryUsdc` field that can be larger than the amount actually debited from the trader during execution. `planClose()` first computes `pendingCarryUsdc`, then realizes only the collectible portion into `realizedCarryUsdc` after consuming position margin and free settlement. `_buildCloseAssessment()` nevertheless stores `assessment.carryUsdc = delta.pendingCarryUsdc`, and `_includeCommitment()` adds committed carry on top of that, while the actual post-settlement checks and receipt settlement use only `realizedCarryUsdc` plus the separate action-charge paths for any unpaid remainder.

This means the public assessment surface and the executed receipt can report carry as if the full pending amount was collected even when some of it remained outstanding or was later waived through action-charge logic. The mismatch is visible to external consumers because `OrderRouterExecutionSidecar` copies `assessment.carryUsdc` directly into terminal receipt economics. Any off-chain consumer that sums `carryUsdc` as collected trader payment will overstate realized carry and misattribute the remaining amount that stayed unpaid.

Preconditions

- A close has positive pending carry, but margin-first collection realizes less than the full amount, leaving `delta.pendingCarryUsdc > delta.realizedCarryUsdc` (`packages/perps/src/libraries/CfdEnginePlanLib.sol:771-772`).

- An integrator or monitoring system reads `ExecutionAssessment.carryUsdc` or receipt `economics.carryUsdc` as the carry actually collected from the account (`packages/perps/src/CfdOrderPolicyEvaluatorBase.sol:424-476`, `packages/perps/src/OrderRouterExecutionSidecar.sol:773-782`).

Impact

Read surfaces and emitted receipt economics can over-report collected carry on close, which misleads accounting, analytics, and any automation that treats the field as realized cash flow instead of pre-collection carry due.

Remediation

Set close `carryUsdc` to the actually realized carry, or split the field into explicit `pendingCarryUsdc` and `realizedCarryUsdc` outputs so receipts cannot conflate due carry with collected carry.

Low ACKNOWLEDGED

Integrator can misread trader account equity from the public view interface

ID: LOW-35

> **Comment**
> LOW-35 — Addressed in the public view interface. For a live position, account equity means post-carry PnL pledge plus the same account's trader claim and cached-mark price PnL, floored at zero; it excludes VPI and other settlement buckets. For a flat account it is raw clearinghouse settlement. The diagnostic withdrawal boundary may need a one-atom reduction where the live guard requires strictly greater collateral. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IPerpsTraderViews.sol:10-17`

`packages/perps/src/interfaces/PerpsViewTypes.sol:22-45`

`packages/perps/src/PerpsPublicLens.sol:142-167`

The product-facing trader view interface says `getTraderAccount` returns settlement equity, but `PerpsPublicLens` returns very different semantics whenever the account has an open position. The interface comment advertises `Settlement equity` and `queued margin state`, yet the implementation sets `equityUsdc` to `max(snapshot.netEquityUsdc, 0)` for positioned accounts, where `netEquityUsdc` is cached-mark price-risk equity after projected carry and excluding generic settlement buckets. `PerpsViewTypes.TraderAccountView` also documents that field as price-risk equity, not settlement balance. An integrator coding against `IPerpsTraderViews` can therefore display or reason about this field as wallet-like settlement balance while the implementation is actually returning a risk metric.

Preconditions

- An integration relies on the public `IPerpsTraderViews` interface comments to interpret `getTraderAccount` (`packages/perps/src/interfaces/IPerpsTraderViews.sol:10-17`).
- The inspected account has an open position, so `PerpsPublicLens` returns `snapshot.netEquityUsdc` instead of the clearinghouse settlement balance (`packages/perps/src/PerpsPublicLens.sol:154-157`).

Impact

Frontends and bots can mislabel a risk-derived equity number as settlement balance, which can mislead users and downstream automation about available collateral and account state.

Remediation

Update `IPerpsTraderViews.getTraderAccount` documentation to describe the returned field as cached-mark price-risk equity, consistent with `PerpsViewTypes.TraderAccountView` and the implementation.

Low ACKNOWLEDGED

Trader or keeper can use inverted feeds to bypass the basket confidence cap

ID: LOW-36

Comment

LOW-36 — Addressed.

Confidence for inverted feeds is calculated from the transformed lower price endpoint, 1 / (price - confidence), relative to the inverted midpoint. This accounts for the larger uncertainty introduced by inversion and applies the corrected value to the basket confidence cap and adverse-price calculation.

Inverted feeds with confidence >= price are rejected because their confidence interval reaches or crosses zero. Direct-feed handling is unchanged.

Fix:
https://github.com/Plether-Fi/plether-core/commit/bf0a8a4e24824c8dea577c3c312de723d1116719

Locations:

`packages/perps/src/PletherOracle.sol:666-669`

`packages/perps/src/PletherOracle.sol:686-689`

`packages/perps/src/PletherOracle.sol:718-724`

`packages/perps/src/PletherOracle.sol:759-762`

PletherOracle underestimates confidence on inverted Pyth feeds. In `_computeBasketPriceFromFeeds`, the basket confidence contribution is always computed as `weightedPrice * conf / price` after `weightedPrice` has already been built from the inverted midpoint price (`packages/perps/src/PletherOracle.sol:666-669`). That formula is exact for direct feeds, but for an inverted feed the true upper half-width is `1 / (p - c) - 1 / p = c / (p * (p - c))`, not `c / p^2`. As soon as `conf > 0`, the oracle reports a smaller confidence than the transformed inverse price actually has. The same understated value is then used both for the hard gate at `basketMaxConfidenceRatioBps` and for adverse order/liquidation pricing (`packages/perps/src/PletherOracle.sol:686-689`, `packages/perps/src/PletherOracle.sol:718-724`, `packages/perps/src/PletherOracle.sol:759-762`).

This matters in the live basket because four components are inverted (`USD/JPY`, `USD/CAD`, `USD/SEK`, `USD/CHF`) and the release config uses a tight 10 bps aggregate cap with a 2500 bps adverse-confidence multiplier. A concrete borderline case is the 3.6% CHF leg: if the raw CHF feed is at its base level and reports `conf / price = 2.777777%`, the current code computes basket confidence of exactly 100,000 on a 100,000,000 basket and accepts the basket, but the exact inverse-space half-width is 102,857, so the configured 10 bps cap should have rejected it. The same example understates the adverse shift by 714 oracle-price units at the live 0.25x multiplier. Any trader or keeper can wait for such a wide-confidence update and then execute orders, liquidations, protection triggers, or LP mark refreshes with data that exceeds the configured transformed-confidence limit.

Preconditions

- The oracle basket contains inverted legs and uses the deployed configuration with a 10 bps `basketMaxConfidenceRatioBps` cap and 2500 bps adverse-confidence multiplier (`deployments/releases/2026-10-12-perps-arbitrum-sepolia-v1.3.1/manifest.json:307-308`).
- At least one inverted Pyth component reports a materially wide confidence interval. The live basket has four inverted legs with nonzero weights, including a 3.6% CHF leg (`deployments/releases/2026-10-12-perps-arbitrum-sepolia-v1.3.0/oracle-fork/result.json:69-83`).
- An untrusted trader or permissionless keeper calls a path that consumes the basket confidence result, such as order execution, liquidation pricing, mark refresh, protection triggering, or LP settlement (`packages/perps/src/PletherOracle.sol:192-261`, `packages/perps/src/PletherOracle.sol:271-309-`, `packages/perps/src/PletherOracle.sol:154-160`).

Impact

The protocol can accept and act on baskets whose true transformed inverse-price uncertainty is above the configured limit, and it applies a smaller adverse execution or liquidation shift than intended. This weakens an advertised oracle-integrity guard and lets users settle or liquidate against slightly too-favorable prices during high-uncertainty windows.

Remediation

For inverted feeds, derive confidence from the transformed bounds instead of reusing `weightedPrice * conf / price`. Compute the inverse-price upper half-width from the raw Pyth interval, for example `inv(price - conf) - inv(price)`, and use that transformed value in both the basket-confidence sum and the adverse-shift calculation.

Low ACKNOWLEDGED

Integrator can receive wrong side-margin totals from planner

ID: LOW-37

> **Comment**
> LOW-37 — Not confirmed. _selectedSide returns a reference to the selected in-memory side snapshot; assigning it to another memory variable does not create the detached copy assumed by the finding. Updating selected.totalMargin and selected.borrowBaseUsdc therefore updates the side inside the working snapshot. The stated stale-aggregate cause does not follow from this code, and no redundant write-back was added. Reviewed implementation: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/libraries/CfdEnginePlanLib.sol#L611-L640

Locations:

- `packages/perps/src/libraries/CfdEnginePlanLib.sol:303-306`
- `packages/perps/src/libraries/CfdEnginePlanLib.sol:327-400`
- `packages/perps/src/libraries/CfdEnginePlanLib.sol:611-637`
- `packages/perps/src/libraries/CfdEnginePlanLib.sol:692-735`
- `packages/perps/src/CfdEngine.sol:746-755`
- `packages/perps/src/CfdEngineSettlementSidecar.sol:474-484`

The planner can return overstated selected-side margin totals after pending carry is realized. In `_applyPendingCarryRealizationToSnapshot`, the account margin, custody buckets, and pool cash are updated on the snapshot, but the selected side's `totalMargin` and `borrowBaseUsdc` are modified only on a temporary `SideSnapshot` copy rather than written back to `snap.longSide` or `snap.shortSide`. `planOpen` and `planClose` then read the unchanged side snapshot and populate `sideTotalMarginBefore`, `sideTotalMarginAfterOpen`, `totalMarginBefore`, and `totalMarginAfterClose` from stale pre-carry aggregates.

This does not let a trader pass open-time solvency checks. The settlement-buffer admission check uses only max liability and pool assets, and live settlement recomputes canonical side margin from clearinghouse balances instead of trusting these planner totals. The impact is limited to incorrect planner output for offchain callers and previews that rely on these aggregate-margin fields.

Preconditions

- The account already has an open position with pending carry, so `planOpen` calls `_applyPendingCarryRealizationToSnapshot(effectiveSnap, delta.pendingCarryUsdc)` before it derives `sideTotalMarginBefore` and `sideTotalMarginAfterOpen` from the same snapshot (`packages/perps/src/libraries/CfdEnginePlanLib.sol:301-306`, `packages/perps/src/libraries/CfdEnginePlanLib.sol:327-400`).
- Any trader who can submit a same-side increase through the router can trigger it. The router-facing engine path calls `planner.planOpen` on the sidecar-built snapshot without reloading the side aggregates after carry projection (`packages/perps/src/CfdEngine.sol:731-755`).
- Impact depends on the protocol using `sideTotalMarginAfterOpen` as an admission input for settlement-buffer solvency, which the planner does in `_applyOpenSettlementBufferAfterPlan` (`packages/perps/src/libraries/CfdEnginePlanLib.sol:692-735`).

Impact

Orders can be admitted with less real side backing than the planner checked. That lets a trader increase exposure after carry has already consumed their side's pledge, weakening the settlement-buffer gate and potentially pushing the pool into an undercollateralized state that honest admission should have rejected.

Remediation

Write the carry-adjusted `totalMargin` and `borrowBaseUsdc` back to `snap.longSide` or `snap.shortSide` inside `_applyPendingCarryRealizationToSnapshot` instead of mutating a detached memory copy.

Low ACKNOWLEDGED

Owner can temporarily disable oracle-backed operations by rotating to an oracle with a non-contract Pyth endpoint

ID: LOW-38

Comment

LOW-38 — Addressed.

PletherOracle construction and router validation require the configured Pyth endpoint to contain deployed contract code. This rejects non-contract endpoints before they can become active, both during initial configuration and oracle rotation.

A rejected rotation preserves the active configuration, pending proposal, and version, allowing a valid retry. Code presence prevents the reported configuration error; selecting the correct Pyth implementation remains a governance responsibility.

Fix:
https://github.com/Plether-Fi/plether-core/commit/ce3fef7a301c2fd3ce9264d765023bc37829ba4f

Locations:

`packages/perps/src/PletherOracle.sol:108-115`

`packages/perps/src/router/OrderOracleExecution.sol:49-59`

`packages/perps/src/router/OrderHandler.sol:64-70`

`packages/perps/src/PletherOracle.sol:406-412`

The router accepts a replacement `PletherOracle` whose `pyth()` dependency is a nonzero address even if that address has no code. `PletherOracle` only rejects `pyth_ == address(0)` in its constructor, and `OrderOracleExecution._validateOracleConfig()` only checks that `oracle.pyth()` returns a nonzero address. It never verifies that the returned endpoint is a deployed contract or behaves like `IPyth`.

Because `applyOracleConfig()` only copies policy and then swaps the router's oracle pointer, a malformed oracle can be finalized successfully and fail only later on first use. After rotation, `PletherOracle.getUpdateFee`, `updatePrice`, `updateOrderExecutionPrice`, `updateLiquidationPrice`, and related helpers call `pyth.getUpdateFee`, `updatePriceFeeds`, `parsePriceFeedUpdatesUnique`, or `getPriceUnsafe`. If `pyth()` points to an externally owned account or undeployed address, those calls revert when Solidity decodes empty returndata. The result is that mark refresh, delayed-order execution, liquidations, and LP epoch settlement stop working until governance completes another oracle rotation.

Preconditions

- A PletherOracle is deployed with `pyth_` set to a nonzero address that is not an `IPyth` implementation (`packages/perps/src/PletherOracle.sol:108-115`).
- The owner finalizes an initial router deployment or a later oracle rotation that points the router at that oracle (`packages/perps/src/router/OrderHandler.sol:64-70`, `packages/perps/src/router/OrderOracleExecution.sol:49-59`).
- Recovery depends on the router admin timelock for a replacement oracle configuration (48 hours in `packages/perps/src/OrderRouterAdmin.sol:181-193`).

Impact

Oracle-dependent protocol operations become unavailable. Keepers cannot refresh the engine mark, execute delayed orders, liquidate unsafe accounts, or settle LP epochs, and the outage lasts until a new oracle rotation clears the bad dependency.

Remediation

Require `address(oracle.pyth()).code.length != 0` during router oracle validation, and preferably probe a harmless `IPyth` selector such as `getUpdateFee` before accepting the oracle.

Low ACKNOWLEDGED

Front-runner can block third-party vault share claims by dusting the receiver

ID: LOW-39

Comment

LOW-39 — Addressed by requiring receiver == controller for deposit/mint share claims, redemption cancellations, and redemption-share refunds, including zero-share acknowledgements. Eligibility no longer depends on the receiver's share balance, so unsolicited dust cannot disable the supported self-delivery path. Approved operators may still execute for the controller. The accepted compatibility trade-off is that third-party share delivery is no longer supported; no receiver opt-in mapping or additional authorization API is added. Funded USDC claim routing remains unchanged.

Fix:
https://github.com/Plether-Fi/plether-core/commit/efcf629aed56cbbf11b8cf5a717b89eb66a5fa2f

Locations:

`packages/perps/src/TrancheVault.sol:886-895`

`packages/perps/src/TrancheVault.sol:917-927`

`packages/perps/src/TrancheVault.sol:1037-1040`

`packages/perps/src/TrancheVault.sol:1062-1064`

`packages/perps/src/TrancheVault.sol:1598-1605`

Anyone who can hold a single tranche share can make the vault's third-party share-receiver paths revert. `TrancheVault` rejects a share receiver whenever `receiver != controller` and `balanceOf(receiver) != 0` (`_requireShareReceiver`). That check is reached by `claimDeposit`, `mint`-style claim flows, `cancelRedeemRequest`, and `claimRedeemRefund`. An attacker can front-run one of those transactions by transferring 1 share to the chosen receiver address first. The controller or its operator then hits `TrancheVault__ThirdPartyDepositForExistingHolder()` and the workflow reverts even though the request itself is otherwise valid. The assets are not stolen, but any automation that relies on a fixed third-party receiver can be griefed repeatedly until it rotates receivers or the dust is removed.

Preconditions

- A controller or its operator uses a third-party share receiver in `cancelRedeemRequest`, `claimRedeemRefund`, `claimDeposit`, or `mint`-style claim flows, so `receiver != controller` and execution reaches `_requireShareReceiver()` (`packages/perps/src/TrancheVault.sol:886-895`, `packages/perps/src/TrancheVault.sol:917-927`, `packages/perps/src/TrancheVault.sol:1037-1040`, `packages/perps/src/TrancheVault.sol:1062-1064`).

- The attacker can acquire at least one unlocked vault share and transfer it to the intended receiver before the victim transaction executes.
- The affected workflow expects that receiver address to stay usable, instead of falling back to `controller` or another fresh address.

Impact

Third-party receiver claim and refund workflows can be permissionlessly DoSed. Controllers do not lose principal, but their pending deposits or redeem-share refunds cannot be completed to the intended receiver until they choose a different receiver or clear the dusted balance, which can break keeper or omnibus-account automation.

Remediation

Replace the `balanceOf(receiver) != 0` ban with an explicit receiver opt-in mapping set by the receiver, and require that opt-in in the third-party share-receiver paths instead of inferring consent from a zero balance.

Low ACKNOWLEDGED

Controller can force a zero-balance receiver into LP cooldown

ID: LOW-40

Comment

LOW-40 — Addressed. Deposit/mint share claims, redemption cancellations, and redemption-share refunds now require receiver == controller, regardless of receiver balance and including zero-share acknowledgements. A controller therefore cannot use these paths to send unsolicited shares and impose a cooldown on an unrelated empty address. Operators retain execution authority but must deliver shares to the controller. This deliberately removes third-party share recipients rather than adding receiver-consent state; ordinary ERC-20 transfers and funded USDC claim routing are unchanged.

Fix:
https://github.com/Plether-Fi/plether-core/commit/efcf629aed56cbbf11b8cf5a717b89eb66a5fa2f

Locations:

`packages/perps/src/TrancheVault.sol:1037-1040`

`packages/perps/src/TrancheVault.sol:1093-1097`

`packages/perps/src/TrancheVault.sol:886-895`

`packages/perps/src/TrancheVault.sol:917-927`

`packages/perps/src/TrancheVault.sol:1598-1605`

A user with any share-delivering vault action can impose the vault's one-hour cooldown on an unrelated zero-balance address. `TrancheVault._requireShareReceiver()` only rejects third-party receivers that already hold shares, so a controller can claim deposit shares, cancel a redeem request, or claim a redeem refund to a fresh victim address. Those paths then set `lastDepositTime[receiver]` and transfer shares to that receiver. Because the cooldown is tracked per account, not per lot, if the victim receives a legitimate larger share balance before that hour expires, their whole balance inherits the forced cooldown and cannot be transferred or redeemed immediately. This lets an attacker front-run a new LP address with unsolicited dust and temporarily block that user's exit flexibility.

Preconditions

- The attacker controls a share-delivering action such as a deposit-share claim, redeem cancellation, or redeem-refund claim (`packages/perps/src/TrancheVault.sol:867-897`, `packages/perps/src/TrancheVault.sol:899-930`, `packages/perps/src/TrancheVault.sol:1006-1103`).
- The victim currently has `balanceOf(victim) == 0`, so `_requireShareReceiver` allows the unsolicited transfer (`packages/perps/src/TrancheVault.sol:1598-1605`).

- The victim receives or already plans to receive a meaningful share balance before `lastDepositTime[victim] + DEPOSIT_COOLDOWN` elapses (`packages/perps/src/TrancheVault.sol:36-`, `packages/perps/src/TrancheVault.sol:1314-1317`).

Impact

An attacker can temporarily DoS another LP's transfer and redemption ability for up to one hour by dusting a fresh address just before it is used. The victim does not lose principal, but they lose liquidity and cannot immediately exit or move the shares they later receive.

Remediation

Do not allow third-party receivers on cooldown-setting share deliveries. Require `receiver == controller` for claim, cancel, and refund paths, or add an explicit receiver opt-in mechanism before writing `lastDepositTime[receiver]`.

Low ACKNOWLEDGED

Integrator can see live status before engine wiring is complete

ID: LOW-41

> **Comment**
> LOW-41 — Addressed. Protocol status checks code availability for the Engine's pool, Router, planner, settlement sidecar, terminal NAV book and admin, and the Router admin. Missing mandatory dependencies report Configuring rather than Active. These checks detect incomplete wiring; code presence alone is not proof of arbitrary dependency correctness or complete behavioral compatibility. Fix: https://github.com/Plether-Fi/plether-core/commit/e640f4badec287fa4a4310bf66444a4f350a7aa5

Locations:

`packages/perps/src/PerpsPublicLens.sol:496-507`

`packages/perps/src/CfdEngine.sol:327-360`

`packages/perps/src/CfdEngine.sol:366-375`

`packages/perps/src/CfdEngine.sol:406-415`

`packages/perps/src/interfaces/ICfdEngine.sol:242-250`

The public status lens can report the protocol as active even while the engine is still missing mandatory execution dependencies. `PerpsPublicLens._getProtocolPhase()` only checks that `ENGINE.pool()` and `ENGINE.orderRouter()` are nonzero and that `HousePool.canIncreaseRisk()` is true. But the engine's own lifecycle description says live operation also requires the planner, settlement sidecar, and admin wiring, and those are configured later through a separate one-time `setDependencies()` call. An owner can therefore set the pool and router, finish seeding and `activateTrading()`, but forget or delay `setDependencies()`. At that point `getProtocolStatus()` returns `Active` even though execution paths still depend on zero-address or unset components and are not actually ready for live use.

Preconditions

- `setPool()` and `setOrderRouter()` have been called, but `setDependencies()` has not yet been called (`packages/perps/src/CfdEngine.sol:327-360`, `366-375`, `406-415`).
- The HousePool seed lifecycle is complete and `isTradingActive` is true, so `HousePool.canIncreaseRisk()` returns true (`packages/perps/src/HousePool.sol:510-517`).
- Any user, integrator, or monitoring system reads `PerpsPublicLens.getProtocolStatus()` and relies on `phase` to decide whether trading is live.

Impact

Off-chain consumers can treat an unusable deployment as live. Traders, front ends, and monitors may route flow to a stack that still cannot execute orders correctly because core engine dependencies are unset.

Remediation

Derive `Active` only when every required engine dependency is wired. In `PerpsPublicLens._getProtocolPhase()`, add checks that `planner`, `settlementSidecar`, and `admin` are nonzero before returning `Active`.

Low ACKNOWLEDGED

Governance can misclassify protocol-driven close failures as user faults in receipts

ID: LOW-42

Comment
LOW-42 — Not retained for the reported governance-change path. External bounded orders pin the execution configuration; execution detects a changed configuration and finalizes ConfigMismatch before entering the Engine close planner. Close planner UserInvalid classifies a failure under the applicable execution state, not an attribution of moral fault or a guarantee that market/calendar state stayed fixed. Internally generated protection closes intentionally inherit execution-time configuration. No broad reclassification of all health-sensitive close failures is made. Reviewed preflight: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/OrderRouterExecutionSidecar.sol#L896-L919

Locations:

`packages/perps/src/libraries/CfdEnginePlanLib.sol:131-143`

`packages/perps/src/CfdEngine.sol:748-751`

`packages/perps/src/OrderRouterExecutionSidecar.sol:821-845`

The router records every non-OK close planner failure as `UserInvalid`, even when the failure is caused by protocol state changing after commitment. `CfdEnginePlanLib.getExecutionFailurePolicyCategory(CloseRevertCode)` returns `UserInvalid` for all close codes, and `processOrderTyped` uses that mapping to build `CfdEngine__TypedOrderFailure`. This reaches terminal receipts through `OrderRouterExecutionSidecar._classifyTypedFailure`, so a close that fails because governance changed risk parameters after commitment, for example a larger `minBountyUsdc` making a previously valid partial close fall below the new minimum and hit `INVALID_SIZE_QUANTUM`-style admission or a stricter maintenance regime making the same reduction hit `PARTIAL_CLOSE_UNHEALTHY`, is still recorded as user invalid. No funds are stolen, but the permanent receipt evidence and downstream automation lose the distinction between trader fault and protocol-state invalidation.

Preconditions

- A close order is committed successfully under one set of engine risk parameters and remains pending until execution (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:326-345`, `packages/perps/src/CfdEngineSettlementSidecar.sol:325-342`).
- Governance or another trusted configurator changes protocol state before execution, such as the minimum liquidation charge or active maintenance threshold, so the same close now fails planner checks inside `planClose` (`packages/perps/src/libraries/CfdEnginePlanLib.sol:839-850`, `packages/perps/src/libraries/CfdEnginePlanLib.sol:894-897`).

- A keeper executes the order and the engine returns a typed close failure (`packages/perps/src/CfdEngine.sol:748-751`).

Impact

Permanent lifecycle receipts can blame the trader for failures that were actually introduced by protocol-side state changes. That degrades incident response, analytics, and any off-chain policy that treats `UserInvalid` close failures differently from protocol invalidations.

Remediation

Classify close failure codes individually, and map protocol-state-sensitive cases such as post-close health invalidation to `ProtocolStateInvalidated` instead of treating every close failure as `UserInvalid`.

Low ACKNOWLEDGED

Integrator can query wrong async vault limits through IERC7575

ID: LOW-43

> **Comment**
> LOW-43 — Addressed in IERC7575 documentation. In this asynchronous vault, maxDeposit/maxMint take the request controller and report claim capacity, not capacity for a fresh request. maxWithdraw/maxRedeem likewise report funded claim capacity for the controller, who may differ from the original owner or eventual receiver. New requests use the dedicated request-capacity views. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IERC7575.sol:38-41`

`packages/perps/src/interfaces/IERC7575.sol:54-57`

`packages/perps/src/interfaces/IERC7575.sol:70-73`

`packages/perps/src/interfaces/IERC7575.sol:87-90`

`packages/perps/src/TrancheVault.sol:1270-1295`

`packages/perps/src/interfaces/IAsyncTrancheVault.sol:12-16`

The generic `IERC7575` interface documents `maxDeposit`, `maxMint`, `maxWithdraw`, and `maxRedeem` as receiver or owner limits, but the actual async tranche vault interprets those arguments as the request controller. `TrancheVault.maxDeposit` reads `controllerDepositHead[controller]`, `maxMint` reads the same controller queue, and the withdraw/redeem variants read `controllerRedeemHead[controller]`; `IAsyncTrancheVault` even documents that these are controller limits and do not certify third-party receiver eligibility. Integrators that bind only the generic `IERC7575` ABI will query capacity for the wrong address and can show nonzero claim capacity for a receiver that the vault will still reject in `_requireShareReceiver`.

Preconditions

- An integration treats a Plether `TrancheVault` as a plain `IERC7575` vault and trusts the generic parameter names and NatSpec in `packages/perps/src/interfaces/IERC7575.sol:38-41`, `:54-57`, `:70-73`, and `:87-90`.
- The integration queries claim limits for the eventual receiver or owner address instead of the controller address that actually keys the async request state in `packages/perps/src/TrancheVault.sol:1270-1295`.

Impact

Wallets, SDKs, and indexers can present incorrect claimable capacity or submit flows for the wrong account, causing failed claims and misleading balance displays.

Remediation

Update `IERC7575` usage docs in this codebase to state that Plether's async vaults interpret these `max*` arguments as controllers, and direct generic integrations to `IAsyncTrancheVault` for the authoritative semantics.

Low ACKNOWLEDGED

Governance can set a minimum order size that partial closes ignore

ID: LOW-44

Comment

LOW-44 — Addressed. Partial-close admission now compares the requested close notional against the Router's configured minOpenNotionalUsdc, using the capped cached admission mark, instead of deriving a separate minimum from bounty parameters. Close preview uses the same configured minimum, so governance changes affect both paths consistently. Full closes remain exempt to allow complete exits from small residual positions; this exemption is intentional. The threshold is an admission-time check and does not guarantee the same notional at a later execution price. Fix: https://github.com/Plether-Fi/plether-core/commit/b3a7665b6fda74a37759697fa015a1fe83b2063b

Locations:

`packages/perps/src/interfaces/IOrderRouterAdminHost.sol:19-20`

`packages/perps/src/router/OrderHandler.sol:41-46`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:338-345`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:399-401`

Partial closes do not consistently follow the router's configured minimum order size. `applyRouterConfig` updates `minOpenNotionalUsdc`, and open/increase commits compare their notional against that value, but `_validateCloseAdmission` computes the partial-close floor from the engine liquidation settings `minBountyUsdc` and `bountyBps` instead. If governance changes one set of parameters without changing the other, the documented shared floor no longer holds. For example, lowering `minOpenNotionalUsdc` below the engine-derived floor allows a trader to open or increase by a size that cannot later be reduced in the same increment; raising it above the engine-derived floor still permits smaller partial closes than the configured order minimum. Full closes remain allowed below the floor, so the impact is limited to partial-close sizing and configuration correctness.

Preconditions

- `minOpenNotionalUsdc` has been set through the Router config path (`packages/perps/src/router/OrderHandler.sol:41-46`) to a value different from `ceil(minBountyUsdc * 10_000 / bountyBps)`, the partial-close floor computed in `packages/perps/src/router/OrderLiquidationBatchLogic.sol:338-345`.
- A trader or keeper submits a partial close, not a full close, for a position whose requested reduction falls between those two thresholds.

- The impact depends on governance using non-release parameter combinations. The release deployment currently keeps the two values equal at 1,000 USDC (`minOpenNotionalUsdc = 1_000e6` in docs, while `minBountyUsdc = 1e6` and `bountyBps = 10` imply the same floor).

Impact

Honest parameter changes can either admit positions that cannot later be reduced in the same size increments, or allow smaller partial closes than the configured minimum order size. That breaks trader risk management expectations and means the protocol's named minimum order-size control does not actually govern all order types.

Remediation

Use `minOpenNotionalUsdc` as the partial-close floor, or enforce at configuration time that `minOpenNotionalUsdc` must always equal `ceil(minBountyUsdc * 10_000 / bountyBps)` so the two controls cannot drift apart.

Low ACKNOWLEDGED

Expiring keeper can finalize receipts with a zero observed config hash

ID: LOW-45

Comment

LOW-45 — Addressed. Explicit expiry now constructs the shared cleanup context, including the observed execution-configuration hash, cached neutral mark and gross pool depth. Execute-time expiry forwards the full item context into recovery, and normal/recovery receipts copy it consistently. Tests cover explicit, bulk and execute-time expiry with authenticated receipt verification. Explicit cleanup does not ingest a new oracle price, so executionPrice and oraclePublishTime remain zero. Fix: https://github.com/Plether-Fi/plether-core/commit/3cbdf17353419967dace164fbd432fed4e70e3f9

Locations:

`packages/perps/src/OrderRecoverySidecar.sol:43-60`

`packages/perps/src/OrderRecoverySidecar.sol:63-85`

Expiry receipts can record the wrong configuration digest. `OrderRecoverySidecar.expireOrder()` builds an `ItemRequest` with only `orderId`, `action`, and `executor` set, so the explicit `router.expireOrder(orderId)` path forwards `observedConfigHash == 0` into `settleNonEngine()` even when `currentExecutionConfigHash()` is nonzero. The reservation-mismatch recovery path is also wrong: `executeOrderItemFromSidecar()` may already have read the live digest before calling `_recoverExpired`, but `recoverExpiredItem()` rebuilds the receipt from `(orderId, pending, executor)` and never writes that observed hash back. A concrete trace is an order committed under digest `H1`, governance finalizing a new config `H2`, then a keeper expiring the order after its deadline. The terminal receipt becomes `Expired` or `ExpiredReservationMismatch` with `observedConfigHash == 0` instead of `H2`. That breaks the protocol's stated receipt semantics and makes off-chain verification ambiguous, especially for internal protection attempts where `expectedConfigHash == 0` and the observed digest is the only on-chain configuration evidence.

Preconditions

- A pending order reaches its execution deadline and is finalized through `expireOrder` or mismatch recovery (`packages/perps/src/OrderRecoverySidecar.sol:43-60`, `packages/perps/src/OrderRecoverySidecar.sol:63-85`).
- Any keeper or caller triggers expiry after the deadline. No privileged role is required.
- The active execution config hash at expiry is nonzero and differs from the zero value that the receipt records. Internal protection attempts make the ambiguity worse because their expected hash is intentionally zero (deployment/docs assumption).

Impact

Terminal receipts and compact outcomes can misreport which execution configuration was active when the order expired. Indexers, auditors, and recovery tooling lose a load-bearing piece of immutable evidence, and protected-order expiry receipts lose the only observed config digest they were supposed to preserve.

Remediation

Populate `observedConfigHash` on every expiry path. Have `expireOrder()` read `currentExecutionConfigHash()` when it builds the request, and pass the already-read digest through `recoverExpiredItem()` instead of reconstructing a receipt that leaves the field zero.

Low ACKNOWLEDGED

Trader can have orders stuck until expiry when terminal book is missing

ID: LOW-46

Comment
LOW-46 — Addressed. The lifecycle book rejects fresh order commitments with TerminalNavBookUnavailable when engine.terminalNavBook() is zero. The close-admission validation path checks the same binding, and deployment assertions require the intended terminal NAV book to be connected. Existing idempotent request replays still return their original identity without creating a new order. The guard prevents orders from being admitted during incomplete wiring; it does not claim to validate every behavior of a nonzero book implementation. Fix: https://github.com/Plether-Fi/plether-core/commit/b3a7665b6fda74a37759697fa015a1fe83b2063b

Locations:

`packages/perps/src/OrderLifecycleBook.sol:192-199`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:276-278`

`packages/perps/src/CfdEngine.sol:160-176`

`packages/perps/src/CfdEngine.sol:731-755`

`packages/perps/src/CfdEngine.sol:1248-1273`

`packages/perps/src/OrderRouterExecutionSidecar.sol:592-603`

`packages/perps/src/OrderRouterExecutionSidecar.sol:1252-1278`

A trader can successfully commit an order against an engine that still has no terminal NAV book, but every execution attempt will leave that order pending until the execution window expires. `OrderLifecycleBook.currentExecutionConfigHash()` hashes `engine.terminalNavBook()` exactly as stored, so a zero address is treated as a valid pinned configuration and `OrderLiquidationBatchLogic._validateFreshRequest()` accepts it. When a keeper later executes the order, `CfdEngine.processOrderTyped()` reaches `_applyOpen()` or `_applyClose()`, both of which call `_beginTerminalCurveMutation()`. That path hard-reverts with `CfdEngine__TerminalNavBookNotSet()`. `OrderRouterExecutionSidecar` does not classify that revert as a terminal planner or policy failure, so it converts it into `PendingReason.EngineFailure` and keeps the order live. The trader's committed margin and execution bounty stay locked until a later call finally expires the order. The scratch test `$GUARD_SCRATCH/WM15MissingTerminalBook.t.sol` reproduces this flow and passes.

Preconditions

- The owner has wired the engine and router well enough for commits to proceed, but `terminalNavBook` is still unset (`packages/perps/src/CfdEngine.sol:378-400`, `packages/perps/src/OrderLifecycleBook.sol:192-199`).

- A trader submits a fresh order with the current execution-config hash, and any keeper or user tries to execute it before its deadline (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:252-278`, `packages/perps/src/OrderRouter.sol:211-217`).
- The temporary lock lasts for the configured execution window. Router config allows any nonzero window up to one hour, with a 60-second default (`packages/perps/src/router/OrderExecutionOrchestrator.sol:10-14`, `packages/perps/src/OrderRouterAdmin.sol:309-315`).

Impact

Orders that should fail immediately instead sit in the live queue as retryable engine failures. Traders cannot execute them and cannot recover their reserved margin or bounty until the deadline-based expiry path runs.

Remediation

Treat a zero terminal NAV book as an invalid execution configuration. Reject it at commit time by making `currentExecutionConfigHash()` or fresh-request validation revert when `engine.terminalNavBook()` is zero; alternatively, classify `CfdEngine__TerminalNavBookNotSet()` as a terminal configuration failure instead of a retryable pending error.

Low ACKNOWLEDGED

Keepers can lose mark context from mismatch receipts

ID: LOW-47

> **Comment**
> LOW-47 — Addressed. Mismatch/recovery receipts now preserve the supplied execution context through the shared context-copying path, including cached mark, pool depth and observed configuration. Tests cover all three expiry routes with corrupted receipt entitlement and verify the resulting authenticated receipt. Cached context remains distinct from a newly ingested execution price; cleanup does not fabricate oracle execution evidence. Fix: https://github.com/Plether-Fi/plether-core/commit/3cbdf17353419967dace164fbd432fed4e70e3f9

Locations:

`packages/perps/src/OrderRouterExecutionSidecar.sol:406-412`

`packages/perps/src/OrderRecoverySidecar.sol:43-60`

`packages/perps/src/OrderRecoverySidecar.sol:63-85`

ExpiredReservationMismatch receipts omit market context that the execution path already collected. During executeOrder or executeOrderBatch cleanup, _preOracleItem records the cached neutral mark and HousePool depth in the ItemRequest, but if the expired order then fails validateBountyFunding(), _recoverExpired() forwards only the order id and executor to OrderRecoverySidecar.recoverExpiredItem(). The recovery sidecar reconstructs the receipt without neutralMarkPrice or poolDepthUsdc, so the emitted OrderFinalized receipt records both as zero even when the Engine and pool had nonzero values at cleanup time. This does not move funds incorrectly, but it weakens the permanent receipt evidence used by off-chain monitoring and replay tooling.

Preconditions

- An order is expired and reaches the `ExpiredReservationMismatch` recovery path after `validateBountyFunding()` fails (`packages/perps/src/OrderRouterExecutionSidecar.sol:355-359`).
- Any keeper or caller triggers `executeOrder` or `expireOrder` after the deadline. No privileged role is required.
- The Engine has a nonzero cached mark and the HousePool has nonzero assets at the time of recovery, so the omitted context would have been meaningful.

Impact

The permanent receipt and compact outcome omit the neutral mark and pool-depth context for mismatch-expiry settlements. Off-chain verifiers, incident tooling, and analytics lose evidence about the market state under which the cleanup happened.

Remediation

Pass the full expiry context into `recoverExpiredItem`, not just `(orderId, executor)`. At minimum propagate `neutralMarkPrice` and `poolDepthUsdc` from the original request, and initialize them in the direct `expireOrder()` path before delegating.

Low ACKNOWLEDGED

LP receipts can report gross pool depth instead of net economic depth

ID: LOW-48

> **Comment**
> LOW-48 — Addressed by defining receipt depth explicitly. poolDepthUsdc is gross recognized HousePool.totalAssets(), before trader-claim deductions, not claim-net economic depth or freely spendable liquidity. Zero can also mean no pool context was sampled for that terminal path. Consumers must use the appropriate liability/reserve accounting instead of treating this field as net backing. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/OrderRouterExecutionSidecar.sol:919-922`

`packages/perps/src/OrderRouterExecutionSidecar.sol:938-943`

`packages/perps/src/router/OrderValidation.sol:22-38`

`packages/perps/src/libraries/PositionRiskAccountingLib.sol:18-25`

`packages/perps/ACCOUNTING_SPEC.md:57-61`

Order receipts label `poolDepthUsdc` as pool depth context, but the router records `IHousePool.totalAssets()` directly. The accounting libraries and specs define economic depth as physical assets minus outstanding trader claims, and that net depth is what open/close/VPI/carry logic actually uses. As a result, the receipt field can overstate the depth that was relevant to the settled trade whenever trader claims are nonzero, which makes receipts and off-chain replay semantics disagree with the planner inputs they are meant to describe.

Preconditions

- Outstanding trader claims are nonzero, so economic depth is `totalAssets() - totalTraderClaimBalanceUsdc` rather than raw `totalAssets()` (`packages/perps/src/libraries/PositionRiskAccountingLib.sol:21-25`).
- A Router path emits a receipt after populating `request.poolDepthUsdc` from `IHousePool.totalAssets()` (`packages/perps/src/OrderRouterExecutionSidecar.sol:919-922`, `packages/perps/src/OrderRouterExecutionSidecar.sol:938-943`, `packages/perps/src/router/OrderValidation.sol:22-38`).

Impact

Receipts and downstream monitoring can attribute a trade to a larger pool depth than the engine actually used for VPI, skew, and carry calculations, breaking semantic parity between emitted evidence and settled accounting.

Remediation

Populate receipt depth with the same net economic depth that the planners use, or rename the field to make clear that it is raw pool assets rather than pricing depth.

Low ACKNOWLEDGED

Trader can open with equity exactly at the initial-margin threshold

ID: LOW-49

Comment
LOW-49 — Addressed. Open planning now requires post-open equity to be strictly greater than the initial-margin requirement; equality returns INSUFFICIENT_INITIAL_MARGIN. The maximum-open quoter's bound and feasibility comparison were updated to the same strict rule, so it does not advertise the rejected equality boundary. The requirement retains the existing integer-USDC rounding; the change makes the comparison strict rather than introducing a new margin formula. Fix: https://github.com/Plether-Fi/plether-core/commit/b3a7665b6fda74a37759697fa015a1fe83b2063b

Locations:

`packages/perps/src/libraries/PositionRiskAccountingLib.sol:142-143`

`packages/perps/src/libraries/OpenAccountingLib.sol:137-139`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:429-433`

The engine accepts an open or increase when projected equity equals the initial-margin requirement, even though the accounting spec says the account must remain strictly above the larger of initial margin and the active maintenance requirement. `PositionRiskAccountingLib.buildExactPriceRiskState()` only marks a position liquidatable when `equity <= maintenance`, and `CfdEnginePlanLib.planOpen()` then rejects opens on `risk.liquidatable || equity < initialMarginRequirement`. When `initMarginBps > maintMarginBps`, equality at the initial threshold passes. With the Sepolia defaults, a trader can open a $100,000 position with exactly $200 of projected equity: maintenance is $100, so `liquidatable` is false, and `200 < 200` is also false. The spec at `ACCOUNTING_SPEC.md:865-866` and `README.md:501` requires that case to be rejected.

Preconditions

- The projected position's initial-margin requirement exceeds the active maintenance requirement, which is the normal configured regime (`packages/perps/src/CfdTypes.sol:103-105`, `packages/perps/src/libraries/OpenAccountingLib.sol:136-139`).
- A trader submits an open or increase whose projected post-trade equity lands exactly on `initialMarginRequirementUsdc` but remains above the maintenance threshold (`packages/perps/src/libraries/CfdEnginePlanLib.sol:424-433`).
- Deployment uses the documented strict admission rule that equity must remain above the larger threshold (`packages/perps/ACCOUNTING_SPEC.md:865-866`, `packages/perps/README.md:501`).

Impact

The protocol admits positions one atomic USDC unit below the documented collateral standard. This narrows the safety margin at the most levered boundary and makes live admission disagree with the published rule and any off-chain systems that enforce it literally.

Remediation

Make the initial-margin comparison strict on equality as well, for example by rejecting when `postOpenRiskState.equityUsdc <= int256(openState.initialMarginRequirementUsdc)` whenever the initial requirement is the active larger threshold, or by computing one required threshold as `max(initialMarginRequirementUsdc, maintenanceRequirementUsdc)` and enforcing `equity > requiredThreshold` in one comparison.

Low ACKNOWLEDGED

Reservation mismatch cleanup can strip a surviving position's protection

ID: LOW-50

Comment

LOW-50 — Addressed. Expired reservation-mismatch recovery invokes the existing failed-protection-attempt handler before deleting the child order. If the account still has a nonzero position with the recorded size and side, protection becomes Latched and retains its trigger evidence and financial limits for a fresh, zero-bounty permissionless retry. Changed or closed positions still fail protection. The shared recovery point covers explicit expiry, single execution and batch execution.

The expired child remains terminal with an ExpiredReservationMismatch receipt. Existing quarantine, foreign-ownership and refund rules remain intact, and receipt-finalization failure rolls back recovery and relatching together. Relatching does not repair disputed accounting or bypass retry checks. Regression tests inject reservation faults to verify recovery; they do not establish that public callers can create those faults.

Fix:
https://github.com/Plether-Fi/plether-core/commit/1c27078badc8ad9a943bc6ee19a4d6098ddc6d13

Locations:

`packages/perps/src/OrderRouterExecutionSidecar.sol:355-359`

`packages/perps/src/OrderRecoverySidecar.sol:63-85`

`packages/perps/src/OrderRouter.sol:183-201`

`packages/perps/src/PositionProtectionBook.sol:552-575`

A triggered protection close that expires with a bounty-reservation mismatch is resolved as a terminal protection failure even when the protected position still matches exactly. When `executeOrderItemFromSidecar()` sees `validateBountyFunding()` revert after deadline, it falls into `recoverExpiredItem()`, which calls `expireMismatchedOrderFromSidecar()`. That path deletes the queued close order and reaches `PositionProtectionBook.afterOrderTerminal(..., Failed)`, which unconditionally moves a triggered protection to `Failed` and clears the active lock. The normal relatch path in `handleFailedProtectionAttempt()` is never invoked. As a result, a trader who still has the original position loses the permissionless retry lane and their position becomes unprotected after an accounting mismatch on the zero-bounty close record.

Preconditions

- A protection-triggered close attempt is pending in `OrderLifecycleBook` and `PositionProtectionBook` (`packages/perps/src/PositionProtectionBook.sol:501-515`, `packages/perps/src/OrderLifecycleBook.sol:378-389`).

- After `executionDeadline`, `OrderLifecycleBook.validateBountyFunding(orderId)` reverts for that attempt because its zero-amount order bounty record no longer validates, and anyone calls `executeOrder` or `expireOrder` on the expired id (`packages/perps/src/OrderRouterExecutionSidecar.sol:355-359`).
- The protected position still exists with the same account, side, and size, so the ordinary failed-attempt path would have relatched it (`packages/perps/src/PositionProtectionBook.sol:605-609`).

Impact

The queued close is removed, but the protection is terminalized instead of relatched. The user loses automatic protection and cannot use `retryPositionProtectionClose` on the same protection record, leaving a still-open position unprotected until the trader notices and creates a new protection manually.

Remediation

When mismatch recovery finalizes an expired protection attempt, route it through the same exact-position relatch logic as other failed protection attempts instead of letting `afterOrderTerminal(..., Failed)` terminalize it unconditionally.

Low ACKNOWLEDGED

Integrator can understate active maintenance during FAD windows

ID: LOW-51

> **Comment**
> LOW-51 — Addressed. Invalid open previews now report the active normal/FAD maintenance requirement when the planner has constructed the projected position economics. Golden-value tests cover LONG at $0.80 and SHORT at $1.20 in both regimes. Earlier failures, including numeric-domain rejection before those economics exist, can still leave fields zero/unavailable; invalid previews do not guarantee that every diagnostic field was computed. Fix: https://github.com/Plether-Fi/plether-core/commit/3cbdf17353419967dace164fbd432fed4e70e3f9

Locations:

`packages/perps/src/CfdEngineLens.sol:258-268`

`packages/perps/src/libraries/OpenAccountingLib.sol:136-139`

`previewOpen()` reports the wrong maintenance requirement when an open is rejected before the final risk-state projection during a Friday auto-deleverage (FAD) window. `_openPreviewFromPlan()` copies `delta.openState.maintenanceMarginUsdc` into the preview before checking `delta.valid`, and `OpenAccountingLib.tryBuildOpenState()` always computes that field from `maintMarginBps`, not from the active FAD margin. In a concrete case from the test fixture, a 10,000-token open at $1.00 with 200 USDC margin during FAD is rejected because the live maintenance requirement is 300 USDC (3%), but the preview reports only 100 USDC (1%). Off-chain callers that use this view to explain rejections or size rescue margin will be shown a materially smaller requirement than the engine is actually enforcing.

Preconditions

- The engine is in a FAD window so the active maintenance ratio is `riskParams.fadMarginBps` instead of `riskParams.maintMarginBps` (`packages/perps/src/CfdEngineLens.sol:273-285`).
- A user or integrator calls `previewOpen` or `quoteMaxOpen` for an order that fails before the final valid-path risk projection, such as an order rejected with `INSUFFICIENT_INITIAL_MARGIN` during FAD.
- Deployment uses `fadMarginBps > maintMarginBps`, which is allowed and used by the protocol (`packages/perps/src/CfdEngine.sol:289-296`).

Impact

The lens returns a numerically incorrect maintenance requirement for invalid previews, which can mislead UIs, bots, or external risk checks into understating the margin needed to become non-liquidatable during FAD windows.

Remediation

Populate `preview.maintenanceMarginUsdc` from the active margin regime before any early return, or compute `OpenAccountingLib.OpenState.maintenanceMarginUsdc` with the caller-selected active margin bps instead of always using `maintMarginBps`.

Low ACKNOWLEDGED

Keeper can underfund liquidation batch item calls

ID: LOW-52

Comment
LOW-52 — Addressed. Before dispatching a liquidation item, the batch now reserves the greater of the EIP-150 forwarding allowance and its 250,000-gas return tail, plus 50,000 gas for encoding and call overhead, in addition to itemGas. The EIP-150 allowance is floor((itemGas - 1) / 63), including when itemGas grows with the account's pending-order count. An underfunded batch stops and returns the current index before attempting that item. This protects the promised forwarding budget; the finite item budget still does not guarantee successful liquidation under every dependency or state condition. Fix: https://github.com/Plether-Fi/plether-core/commit/b3a7665b6fda74a37759697fa015a1fe83b2063b

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:718-727`

`packages/perps/src/OrderRouterAdmin.sol:376-388`

`packages/perps/src/interfaces/IOrderRouterAdminHost.sol:24-26`

Liquidation batches can stop earlier than their own gas precheck indicates. `executeLiquidationBatch` computes `itemGas`, verifies only that the parent frame has `itemGas + LIQUIDATION_BATCH_TAIL_GAS`, and then calls `executeLiquidationBatchItem` with `{gas: itemGas}`. Because the EVM applies the EIP-150 63/64 forwarding cap to that external self-call, the child frame can receive less than `itemGas` even though the router's check passed. If the item needs the missing gas, it empty-reverts and the batch returns `LiquidationBatchStopped(nextIndex)`, leaving later accounts for a retry. This is a recoverable liveness and gas-accounting bug rather than a fund-loss issue.

Preconditions

- `minEngineGas` and the per-call pending-order limits are set within the documented admin range, including the allowed upper bounds (`packages/perps/src/OrderRouterAdmin.sol:376-388`).
- A keeper calls `executeLiquidationBatch` on an account whose `pendingOrderCounts(account)` makes `itemGas` large enough that EIP-150 caps the forwarded gas below the requested amount (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:718-727`).
- The impact depends on the configured values. It is reachable at the allowed maximum `minEngineGas = 5_000_000` with 20 pending orders, and at the default `minEngineGas = 600_000` with 32 pending orders (the admin limit).

Impact

A liquidation batch can deterministically stop on a busy account before attempting later candidates, despite the configuration claiming the engine gas floor is satisfied. This reduces liquidation throughput and can leave later liquidatable accounts unprocessed until a keeper resubmits a suffix batch around the pathological account.

Remediation

Apply the EIP-150 uplift when checking and assigning `itemGas`, so the parent frame reserves enough gas for the requested child budget instead of passing the raw `minEngineGas + overhead` sum.

Low ACKNOWLEDGED

Trader previews can overstate committable margin

ID: LOW-53

> **Comment**
> LOW-53 — Engine-preview scope retained. previewOpen/quoteMaxOpen model the Engine plan against the supplied state; the quote explicitly excludes Router bounty reservations and admission policy. A caller must leave free settlement for the Router bounty or simulate the complete commit. If locking that bounty leaves insufficient committed margin, commitOrder reverts atomically, so the example does not enqueue an unexecutable order or consume keeper queue capacity. The remaining limitation is a planner quote that is not a Router-admission guarantee. Reviewed scope: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/interfaces/ICfdEngineLens.sol#L79-L85

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-421`

`packages/perps/src/CfdEngineLens.sol:601-638`

`packages/perps/src/router/OrderReservationAccounting.sol:108-127`

`packages/perps/src/router/OrderCommitHandler.sol:21-35`

`packages/perps/src/libraries/MarginClearinghouseAccountingLib.sol:137-178`

The public open preview path can report an order as valid even when the router cannot commit it because the preview does not subtract the open-order execution bounty. `CfdEngineLens.previewOpen` and `quoteMaxOpen` build their snapshot from the current clearinghouse buckets only, while the router commit path locks the open bounty in reserved settlement before it reserves `marginDelta` as committed order margin. If an account has 1,000 USDC free settlement, requests `marginDelta = 1,000e6`, and the router bounty is 0.20 USDC, the preview can return a valid open from the 1,000 USDC snapshot, but `commitOrder` locks the 0.20 USDC bounty first and then reverts when only 999.80 USDC remains free for the 1,000 USDC margin reservation. The failed commit rolls back and no pending order is queued, so the impact is wasted caller gas and misleading trade-ticket sizing rather than keeper queue exhaustion.

Preconditions

- The open is admitted using `previewOpenRevertCode` against the pre-commit snapshot in `OrderLiquidationBatchLogic._validatedOpenBounty` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-416`).
- An unprivileged trader submits an open whose free settlement is sufficient before reserving the bounty but insufficient after the router locks `executionBountyUsdc` in `lockReservedSettlement` (`packages/perps/src/router/OrderReservationAccounting.sol:124-127`).

- The deployment uses a nonzero open-order execution bounty, as configured by `openOrderExecutionBountyBps` / `minOpenOrderExecutionBountyUsdc` / `maxOpenOrderExecutionBountyUsdc` (`packages/perps/src/OrderRouterBase.sol:28-33`, live defaults 1 bp with 0.01 to 0.20 USDC bounds).

Impact

Keepers and batch executors waste gas on opens that could have been rejected at commit time, and an attacker can occupy queue and cleanup capacity with orders that are deterministically unexecutable once their own bounty reservation is accounted for.

Remediation

Include the pending execution bounty in the commit-time lens snapshot, or reserve the bounty before calling the open preview and roll the reservation back if the preview rejects.

Low ACKNOWLEDGED

Senior redeemers can leave dust high-water mark by splitting withdrawals

ID: LOW-54

Comment

LOW-54 — Healthy-tranche discrepancy addressed by MED-11; residual impaired rounding accepted. HWM reduction now uses actual funded assets over pre-withdrawal principal. When H = P, withdrawals preserve exact equality H' = P'. When Senior is impaired, rounding the reduction down preserves less than one extra USDC base unit (0.000001 USDC) of protection per funded settlement pass relative to that step's exact proportional result. This can accumulate across passes and retain a small impairment. That conservative rounding is accepted; full principal funding clears the HWM.

Arithmetic fix:
https://github.com/Plether-Fi/plether-core/commit/9ab65f4175cb90805eabb434908ab4832a124d56
Accepted residual: https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/libraries/HousePoolWaterfallAccountingLib.sol:121-132`

`packages/perps/src/HousePool.sol:821-841`

`packages/perps/src/HousePool.sol:925-1008`

`packages/perps/src/TrancheVault.sol:600-626`

`packages/perps/src/TrancheVault.sol:1589-1595`

Senior redeemers can preserve dust in the senior high-water mark by splitting withdrawals across separate settlement passes. HousePoolWaterfallAccountingLib.scaleSeniorOnWithdraw subtracts floor(seniorHighWaterMark * fundedShares / preBurnSupply) from the high-water mark. HousePool correctly aggregates all senior redemption funding within one settlement pass before calling this function, but a later pass recomputes against the already rounded state and can lose another fractional unit.

The extra high-water mark is restored before junior receives revenue, so the rounding dust can be diverted from junior LPs to remaining senior LPs. The error is less than one USDC base unit per funded senior settlement pass, which makes this a dust accounting defect rather than a material value leak.

Preconditions

- Senior principal and seniorHighWaterMark are both positive, and at least one senior redemption is funded through HousePool._settleLpEpoch at packages/perps/src/HousePool.sol:821-841.

- A senior LP can split one intended redemption across multiple redeem requests because TrancheVault.requestRedeem accepts any size when it equals the holder's maxShares, even if estimateRedeemAssets(shares) is below the normal minimum, at packages/perps/src/TrancheVault.sol:600-626 and :1589-1595.
- Future positive reconcile or claimant-revenue flows occur after the split redemption, so the preserved extra HWM matters when revenue is routed back through the waterfall.

Impact

A senior LP who exits in many small pieces leaves behind more unimpaired senior entitlement than the same economic exit performed in one piece. That overstates the remaining senior claim and diverts future revenue away from junior LPs.

Remediation

Track senior high-water-mark reduction against the total funded shares for the whole frozen-pricing phase, or store enough cumulative funding state to compute one exact pro-rata HWM burn from the original pre-burn supply instead of rounding down on every sub-step.

Low ACKNOWLEDGED

Integrator can be misled by close preview accepting uncommittable client ids

ID: LOW-55

> **Comment**
> LOW-55 — Addressed. The bounded OrderRequest close preview resolves lifecycle-book client identity before fresh admission. Conflicting ids return the same conflict error as submission; reserved protocol ids are rejected for fresh external requests. Exact pending or terminal replays return their original replayOrderId and zeroed economics without re-running live configuration, deadline, queue or position checks. Consumers must inspect replayOrderId first. The lower-level admission helper remains a fresh-state diagnostic, and the legacy identity-free overload cannot resolve client identity. Fix: https://github.com/Plether-Fi/plether-core/commit/c259613ecdc52ecad48bf381f3aeb0387181a9f5

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:137-148`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:195-215`

`packages/perps/src/OrderLifecycleBook.sol:216-235`

`packages/perps/src/OrderLifecycleBook.sol:237-265`

`previewCloseAdmission()` can report success for close requests that `commitOrder()` will always reject. The preview only runs `_validateFreshRequest()` and `_validateCloseAdmission()`, but it never resolves the client intent in `OrderLifecycleBook`. As a result it does not reject already-used `clientOrderId` values, conflicting reuses, or public requests that illegally use the protocol-reserved client-id prefix. A frontend or keeper workflow that treats a successful preview as a commitability check will present or act on a close that later reverts in `_commitOrder()` or `OrderLifecycleBook.registerPending()`.

Preconditions

- An external caller or integration uses `previewCloseAdmission` as a preflight for a fresh public close (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:137-148`).
- The supplied `clientOrderId` is already bound in the account namespace or uses the reserved protocol prefix while `expectedConfigHash` is nonzero (`packages/perps/src/OrderLifecycleBook.sol:216-235`, `packages/perps/src/OrderLifecycleBook.sol:262-265`).
- The request otherwise satisfies the close-admission checks in `_validateFreshRequest` and `_validateCloseAdmission` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:252-346`).

Impact

Preview consumers can show false positives, waste keeper gas on doomed commits, or build incorrect automation around a close that the router will not accept. The issue affects commitability integrity rather than on-chain funds.

Remediation

Have `previewCloseAdmission` call `lifecycleBook.resolveClientIntent(account, request)` and reject anything except `ClientIntentResolution.Unused` before returning success.

Low ACKNOWLEDGED

Integrators can mis-handle rejected deposits because the lens says they stay pending

ID: LOW-56

> **Comment**
> LOW-56 — Not confirmed. The documented phrase "leaves Pending and becomes refundable" means exits the Pending state, not remains pending. The implementation matches that transition: pendingDepositRequest returns zero for a rejected epoch and refundableDepositRequest exposes the escrow recoverable through cancellation by the controller/operator. No behavior change is needed. Reviewed interface behavior: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/PerpsPublicLens.sol#L324-L329

Locations:

`packages/perps/src/PerpsPublicLens.sol:321-353`

`packages/perps/src/TrancheVault.sol:697-715`

The public LP request lens documents rejected deposits incorrectly. `PerpsPublicLens.getLpRequestState()` says a rejected deposit 'leaves Pending and becomes refundable', but the value it returns for `pendingDepositAssets` comes from `TrancheVault.pendingDepositRequest()`, which explicitly returns zero once the epoch is rejected. Integrators that follow the lens comment can keep treating a rejected request as both pending and refundable, even though the actual state transition is from pending to refundable-only. That can leave off-chain recovery flows waiting on a pending state that will never clear on its own.

Preconditions

- A deposit epoch has been rejected at settlement so `depositEpochQueueState[requestId].rejected` is true (`packages/perps/src/TrancheVault.sol:1423-1443`).
- An integrator or frontend relies on the `PerpsPublicLens.getLpRequestState()` NatSpec to decide whether a request is still pending or whether it should prompt the user to cancel for refund.
- No special deployment parameter is required.

Impact

Off-chain tooling can misclassify rejected deposits, delaying or breaking the user path that actually recovers the escrowed USDC.

Remediation

Update the lens documentation to say that rejected deposits are no longer pending and must be recovered through the refundable cancellation path.

Low ACKNOWLEDGED

Trader can underpay open-order keeper bounty by committing against a stale mark

ID: LOW-57

Comment

LOW-57 — Accepted limitation; Won't fix. Once a nonzero Engine mark exists, open commitment intentionally remains available even when that mark is stale. Its notional and reserved keeper bounty can therefore differ from a fresh-price calculation, potentially making execution or cleanup less attractive. Bounty floors/caps, trader-specified bounds and per-account pending-order caps limit individual commitments but do not eliminate the risk. Execution separately validates oracle data. Obtaining signed Pyth updates in this deployment requires a paid feed subscription expected to be held by keepers; a freshness gate at commitment would make opens after idle periods depend on a keeper refresh. We retain stale-mark admission to preserve liveness. The separate startup guard rejects an uninitialized zero mark. Accepted policy: https://github.com/Plether-Fi/plether-core/commit/9601993b7ba21ec8ade12aa10515fa8597274520 Startup guard: https://github.com/Plether-Fi/plether-core/commit/d7b64e164d3572f0af12504a4c86f2eea0e4de18

Locations:

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:392-427`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:430-438`

`packages/perps/src/OrderRouterExecutionSidecar.sol:523-537`

`packages/perps/test/perps/spec/trader/OrderRouterAdmission.t.sol:58-71`

Open-order bounty sizing is based on the cached engine mark at commit time, even when that mark is stale. `_validatedOpenBounty()` computes `notionalUsdc` from `_commitReferencePrice()`, which reads `engine.lastMarkPrice()` and even falls back to `1e8` when no mark exists. When the cached mark is stale, the router intentionally skips only the predictable planner rejection and still queues the order with a bounty derived from that stale or synthetic price.

Execution later pays the reserved `pending.executionBountyUsdc` exactly, not a bounty recomputed from the actual oracle execution price. If the cached mark was 1.00 and the historical execution price is 2.00, a 100,000-token order reserves a 10 USDC bounty at commit but only ever pays that 10 USDC even though the configured bounty schedule would imply 20 USDC at execution. The converse stale-high case can also reject or overcharge orders using an outdated external price. This makes keeper compensation and open-order admission depend on stale oracle state instead of the price the protocol actually trades on.

Preconditions

- The cached engine mark is stale or absent, so commit-time logic prices the order from an outdated mark or the synthetic `1e8` fallback (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-418`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:430-438`).
- A trader commits an open or increase order, and the later oracle execution price differs materially from that commit reference price.
- Execution succeeds and pays the already reserved bounty from the pending order record rather than recomputing it from the actual execution price (`packages/perps/src/OrderRouterExecutionSidecar.sol:523-537`).

Impact

Keeper compensation and some commit-time admission checks drift away from the actual oracle price used for execution. In practice this can under-incentivize execution of large orders after stale periods or incorrectly reject orders based on an outdated external price.

Remediation

Either require a fresh cached mark before sizing open-order bounty and notional gates, or recompute bounty and related price-dependent checks from the execution oracle price instead of the commit reference price.

Low ACKNOWLEDGED

Governance can make withdrawable-margin views revert for traders

ID: LOW-58

Comment

LOW-58 — Addressed. The withdrawable-margin lens now uses HousePoolAccountingLib.isMarkFresh() instead of adding maxStaleness to lastMarkTime. The age comparison avoids overflow even when the effective live or frozen freshness limit is type(uint256).max.

Existing semantics are preserved: a mark exactly at the age limit is fresh, and a future stored mark has age zero. This change does not cap governance-configured freshness durations or broaden which oracle timestamps execution accepts.

Fix:
https://github.com/Plether-Fi/plether-core/commit/26dbde811f54cb983580801f47b700c8857df1e7

Locations:

`packages/perps/src/CfdEngineAccountLens.sol:117-131`

`packages/perps/src/libraries/OracleFreshnessPolicyLib.sol:81-113`

`packages/perps/src/CfdEngineAdmin.sol:201-215`

`packages/perps/src/HousePool.sol:1646-1654`

A large freshness limit can make `CfdEngineAccountLens` revert instead of returning 0 withdrawable margin. `getWithdrawableMargin` computes staleness with `block.timestamp > lastMarkTime + maxStaleness` and `maxStaleness` comes from `OracleFreshnessPolicyLib.getPolicy(...)`. Neither `CfdEngineAdmin.proposeFreshnessConfig` nor `HousePool.proposePoolConfig` caps those staleness values, so an honest but oversized configuration such as `engineMarkStalenessLimit = type(uint256).max` makes the addition overflow as soon as an account has an open position and a nonzero cached mark. At that point UIs, bots, and integrators calling the lens cannot read withdrawable margin for affected accounts at all.

Preconditions

- `CfdEngineAccountLens.getWithdrawableMargin()` reaches the open-position staleness check (`packages/perps/src/CfdEngineAccountLens.sol:112-131`).

- Governance finalizes a very large freshness limit through `proposeFreshnessConfig` / `applyFreshnessConfig` or a very large HousePool `markStalenessLimit`, so the effective `PoolReconcile` limit selected by `OracleFreshnessPolicyLib.getPolicy` exceeds `type(uint256).max - lastMarkTime` (`packages/perps/src/CfdEngineAdmin.sol:201-215`, `packages/perps/src/HousePool.sol:1646-1654`, `packages/perps/src/libraries/OracleFreshnessPolicyLib.sol:81-113`).
- An account has an open position and the engine has a nonzero cached mark time.

Impact

Withdrawable-margin reads revert for affected traders. This does not move funds on-chain, but it breaks wallet previews, automation, and safety checks that depend on the lens to decide whether a withdrawal is possible.

Remediation

Compute freshness with subtraction, not addition. For example, reject future marks first and then compare `block.timestamp - lastMarkTime` to `maxStaleness`, or cap configured staleness values before storing them.

Low ACKNOWLEDGED

Owner can leave emergency recovery ownerless

ID: LOW-59

> **Comment**
> LOW-59 — Addressed. HousePool, OrderRouterAdmin, and EmergencyPauseCoordinator override renounceOwnership() to revert, preventing governance from permanently removing their recovery and guardian-management authority through renunciation.
>
> Two-step ownership transfers remain available, including cancellation of a pending transfer. The change prevents explicit renunciation; it does not prevent loss or compromise of the owner account or acceptance by an unsuitable successor.
>
> Fix:
> https://github.com/Plether-Fi/plether-core/commit/1151d01fd30d5fe6f6d80d604c5ddd4b90845dc6

Locations:

`lib/openzeppelin-contracts/contracts/access/Ownable.sol:76-78`

`packages/perps/src/OrderRouterAdmin.sol:15-18`

`packages/perps/src/OrderRouterAdmin.sol:154-204`

`packages/perps/src/OrderRouterAdmin.sol:223-248`

`packages/perps/src/HousePool.sol:75-76`

`packages/perps/src/HousePool.sol:319-405`

`packages/perps/src/HousePool.sol:514-530`

`packages/perps/src/EmergencyPauseCoordinator.sol:14-15`

`packages/perps/src/EmergencyPauseCoordinator.sol:104-109`

The governance owner can permanently remove its own recovery authority by calling the inherited Ownable.renounceOwnership() on OrderRouterAdmin, HousePool, or EmergencyPauseCoordinator. These contracts use Ownable2Step but do not override renounceOwnership(), so the owner can still set owner to address(0) in one transaction. After that, owner-only functions such as unpause(), unpauseLpEpochSettlement(), setPauser(), configuration finalization/cancellation, and setGuardian() cannot be called. If a pauser or guardian is still configured, a later emergency pause or LP settlement hold can become irreversible on-chain.

Preconditions

- Governance calls `renounceOwnership()` on `OrderRouterAdmin`, `HousePool`, or `EmergencyPauseCoordinator`, which is still inherited from `Ownable` (`lib/openzeppelin-contracts/contracts/access/Ownable.sol:76-78`).

- A current or future incident relies on an owner-only recovery path such as `OrderRouterAdmin.unpause()` (`packages/perps/src/OrderRouterAdmin.sol:247-248`) or `HousePool.unpause()` / `unpauseLpEpochSettlement()` (`packages/perps/src/HousePool.sol:383-405`).
- The deployment relies on these contracts' owner-only governance surfaces for normal recovery and configuration management.

Impact

Governance can irreversibly lock itself out of emergency recovery and configuration control. New trading risk, LP entry, or LP epoch settlement can become permanently unavailable after a later pause, and guardian rotation on the coordinator can become impossible.

Remediation

Override `renounceOwnership()` in every governance-bearing perps contract and make it revert, or replace `Ownable2Step` with an ownership module that removes renounce entirely.

Low ACKNOWLEDGED

Lens callers can crash max-open quotes with oversized margin inputs

ID: LOW-60

> **Comment**
> LOW-60 — Addressed. Maximum-open quotes reject margin outside the signed numeric domain through the canonical planner probe, preserving the typed NUMERIC_DOMAIN_EXCEEDED result. The search bound also caps combined existing pledge and supplied collateral at the terminal-book numeric limit before signed conversion. Boundary and full-width fuzz regressions cover oversized inputs and preserve the planner's existing funds/opposing-side error semantics. Fix: https://github.com/Plether-Fi/plether-core/commit/3cbdf17353419967dace164fbd432fed4e70e3f9

Locations:

`packages/perps/src/CfdEngineOpenQuoter.sol:179-188`

`packages/perps/src/CfdEngineOpenQuoter.sol:208-223`

`packages/perps/src/CfdEngineLens.sol:98-120`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:292-300`

`CfdEngineOpenQuoter.quote` is supposed to search with exact planner probes, but it builds an optimistic upper bound from `order.marginDelta` before the planner's own numeric-domain checks run. `_context` copies `order.marginDelta` directly, and `_upperBound` then executes `SafeCast.toInt256(ctx.snap.position.margin + ctx.marginDelta + ctx.snap.traderClaimBalanceForAccount)`. If a caller supplies `marginDelta > type(int256).max`, `previewOpen` for the same input returns the typed `NUMERIC_DOMAIN_EXCEEDED` result from `CfdEnginePlanLib.planOpen`, while `quoteMaxOpen` reverts immediately during bound construction. A Foundry check confirms the mismatch: `previewOpenRevertCode(...)` returns code 11, but `quoteMaxOpen(...)` reverts for the same account, side, price, and margin value. This violates the quote path's advertised parity with planner diagnostics and lets arbitrary callers crash integrations that expect a typed zero-capacity result from the read path.

Preconditions

- A caller invokes `quoteMaxOpen` or `CfdEngineOpenQuoter.quote` with `marginDelta > type(int256).max` (`packages/perps/src/CfdEngineOpenQuoter.sol:179-188`, `packages/perps/src/CfdEngineOpenQuoter.sol:208-223`).
- The integration expects the same typed numeric rejection that `previewOpen` returns for oversized margins (`packages/perps/src/libraries/CfdEnginePlanLib.sol:292-300`).

Impact

The max-open quote read path can revert on caller-controlled inputs that the ordinary preview path handles gracefully, breaking frontend and monitoring flows that rely on quote/preview semantic parity.

Remediation

Mirror the planner's `marginDelta > type(int256).max` guard before building quote bounds and convert it into the same typed `NUMERIC_DOMAIN_EXCEEDED` outcome instead of letting the cast revert.

Low ACKNOWLEDGED

PoolReconcile reads can enforce the order skew limit instead of the pool policy

ID: LOW-61

Comment
LOW-61 — Addressed by documenting the retained oracle policy. Live PoolReconcile component-time divergence uses orderExecutionStalenessLimit, while component age independently uses the effective Engine/HousePool mark-age limit (the smaller nonzero pool/Engine allowance). Live liquidation uses its liquidation limit; frozen reads reuse the Engine mark without comparing component times. No separate PoolReconcile divergence parameter is introduced. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IPletherOracle.sol:399-404`

`packages/perps/src/PletherOracle.sol:479-485`

`packages/perps/src/PletherOracle.sol:570-577`

`packages/perps/src/PletherOracle.sol:789-792`

Live PoolReconcile oracle reads use the order-execution publish-time divergence limit instead of the PoolReconcile staleness policy. `getLatestPoolReconcilePrice()` calls `_getLatestPriceSnapshotWithConfidence(PriceMode.PoolReconcile)`, which correctly computes `policy.maxStaleness` from the PoolReconcile freshness policy, but then passes `_policyPublishTimeDivergence(mode, policy)` into `_computeLiveBasketPrice`. Outside frozen mode `_policyPublishTimeDivergence` ignores the PoolReconcile policy and calls `_maxPublishTimeDivergence`, which maps every non-liquidation mode to `orderExecutionStalenessLimit`. If the pool/engine mark staleness limits are configured differently from the order-execution limit, LP settlement and monitoring can reject or report PoolReconcile baskets using an order policy limit rather than the pool policy described by the interface.

Preconditions

- An integrator, monitor, or operator reads `IPletherOracle.maxComponentPublishTimeDivergence()` and relies on that value to understand the active PoolReconcile current-basket divergence policy.
- The configured oracle is used through the live PoolReconcile path, such as `getLatestPoolReconcilePrice()` or Router LP settlement that validates a current `PriceMode.PoolReconcile` basket (`packages/perps/src/PletherOracle.sol:479-485`).
- The deployment has `orderExecutionStalenessLimit` and `maxComponentPublishTimeDivergence` set to materially different values.

Impact

Off-chain monitoring and configuration tooling can misreport which component-skew limit actually governs live LP-settlement oracle reads. Operators may believe a timelocked config change tightened or loosened PoolReconcile basket skew when the live check still follows `orderExecutionStalenessLimit` instead.

Remediation

Align the interface documentation with the implementation by documenting that live PoolReconcile divergence follows the applicable current-basket staleness policy, or change the implementation to use `maxComponentPublishTimeDivergence` for PoolReconcile if that was the intended external contract.

Low ACKNOWLEDGED

Contract caller can permanently lock deferred ETH refunds

ID: LOW-62

Comment

LOW-62 — Addressed. OrderRouterAdmin.claimBalanceTo(address payable recipient) and PletherOracle.claimEthRefundTo(address payable recipient) allow a beneficiary to direct its own deferred ETH to an address that accepts ETH. Each call claims only msg.sender's credit, rejects a zero recipient, clears the credit before the transfer, and reverts atomically if the transfer fails. Existing self-claim APIs remain available, and both oracle claim entry points share the reentrancy guard.

A nonpayable beneficiary contract must still be able to invoke the claim function; this does not recover credits belonging to a contract that cannot make that call.

Fix: https://github.com/Plether-Fi/plether-core/commit/2578f8a550f5f22c6f23aa764567133a76361013

Locations:

`packages/perps/src/router/OrderExecutionHandler.sol:30-42`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:785-796`

`packages/perps/src/OrderRouterAdmin.sol:256-281`

`packages/perps/src/PletherOracle.sol:349-360`

`packages/perps/src/PletherOracle.sol:833-861`

A caller that cannot receive ETH can get its refund trapped forever in the router admin or oracle. The order paths and LP-settlement path first try a 30,000-gas refund, then fall back to `OrderRouterAdmin.creditClaimableEth` when that transfer fails. The oracle does the same with its own `claimableEth` mapping. But both recovery functions, `OrderRouterAdmin.claimBalance()` and `PletherOracle.claimEthRefund()`, always send the ETH back to `msg.sender` and do not let the beneficiary redirect it to another payable address. If the beneficiary is a contract with no payable receive path, the immediate refund fails, the deferred balance is credited, and every later claim attempt fails the same way. The ETH remains stuck in the admin or oracle contract permanently.

Preconditions

- A router or oracle refund attempt to the beneficiary fails and the code falls back to deferred credit (`packages/perps/src/router/OrderExecutionHandler.sol:39-42`, `packages/perps/src/router/OrderLiquidationBatchLogic.sol:793-796`, `packages/perps/src/PletherOracle.sol:841-846-`, `packages/perps/src/PletherOracle.sol:856-861`).

- The beneficiary is a contract account that rejects ETH transfers, either because it has no payable receive/fallback or it intentionally reverts.
- That beneficiary later tries to recover the deferred refund through `OrderRouterAdmin.claimBalance()` or `PletherOracle.claimEthRefund()` (`packages/perps/src/OrderRouterAdmin.sol:271-281`, `packages/perps/src/PletherOracle.sol:349-360`).

Impact

The affected caller loses access to its ETH refund permanently. Contract-based keepers and integrators can end up with execution-fee refunds stranded inside the router admin or oracle with no on-chain way to redirect or withdraw them.

Remediation

Replace self-only claim with a beneficiary-authorized `claimTo(address payable recipient)` flow so deferred refunds can be withdrawn to a different payable address.

Low ACKNOWLEDGED

Partial LP settlement can round dust away from redeemers

ID: LOW-63

Comment

LOW-63 — Accepted partial-settlement rounding behavior; no code change. Each funding pass prices from the current principal and remaining share supply, so splitting funding can produce rounding differences or a residual share refund compared with a single pass. Unfunded shares continue to participate in tranche profit, loss, and fees; those economic changes are separate from rounding. We retain this model and accept the rounding at normal operating scales. Freezing an epoch price would require additional state and change the economics of shares waiting for funding.

Disposition: https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/HousePool.sol:973-1005`

`packages/perps/src/libraries/HousePoolRedemptionMathLib.sol:63-107`

`packages/perps/ACCOUNTING_SPEC.md:564-580`

Redeemers can lose dust when a matured redemption epoch is funded across multiple settlement calls. `HousePool._fundRedemptionPhase` prices a partial fill from the current `(pricingPrincipal, pricingSupply)` snapshot, transfers the assets, and `TrancheVault.fundRedeemEpoch` burns the funded shares. If the epoch remains queued, the next `settleLpEpoch` call builds a new snapshot from the post-burn principal and supply, so floor rounding and the ERC-4626 virtual offsets can make the total payout differ from funding the same epoch in one pass. This is a path-dependent rounding issue rather than a material redemption loss: with normal six-decimal USDC scale it leaks only dust at partial-fill boundaries.

Preconditions

- A matured redemption epoch is only partially funded in one settlement pass, so `fundedShares < remainingShares` and the epoch stays queued (`packages/perps/src/HousePool.sol:973-1005`).
- Any caller settles LP epochs while tranche withdrawal budget is positive but smaller than the epoch's full pre-burn quote.
- The rounding-sensitive impact depends on the tranche state and fee setting at settlement time; it is easiest to trigger when the epoch is near the virtual-offset boundary (`virtualAssets=1`, `virtualShares=1000`) and the post-burn repricing crosses a floor step (`packages/perps/src/HousePool.sol:938-980`).

Impact

LP redeemers can permanently lose part of a matured redemption epoch's asset value. After an honest partial funding, the remainder can become unfundable and later refundable for shares only, even though those shares had positive asset value under the original epoch price.

Remediation

Fix redemption funding to use one immutable epoch price basis for every partial fill of that epoch. Store the epoch's pre-burn pricing principal and supply when the first fill is computed, and keep using that snapshot until the epoch is fully funded or explicitly refunded.

Low ACKNOWLEDGED

Integrator can mis-handle settlement debits because `settleUsdc` documents the wrong limit

ID: LOW-64

> **Comment**
> LOW-64 — Addressed in IMarginClearinghouse. A negative settleUsdc delta is bounded by free settlement above all locked buckets, not the entire settlement balance. The Engine/settlement-sidecar-only hook changes internal accounting without transferring tokens, changing locked buckets or checkpointing carry; zero is a no-op. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IMarginClearinghouse.sol:403-410`

`packages/perps/src/MarginClearinghouse.sol:972-986`

`packages/perps/src/MarginClearinghouse.sol:1358-1368`

The public clearinghouse interface understates when `settleUsdc` will revert. `IMarginClearinghouse` says a negative delta cannot exceed the account's settlement balance, but the implementation also requires that the debit fit inside current free settlement: `_debitSettlementUsdc` reverts if `getFreeBuyingPowerUsdc(account) < amountUsdc` even when `settlementBalances[account]` is larger. An integrator, simulator, or mock built from the interface and NatSpec can therefore accept debits that the live contract rejects whenever part of the balance is locked as pledge, liquidation reserve, order margin, or reserved settlement.

Preconditions

- An account's `settlementBalances[account]` exceeds the requested debit, but some of that balance is locked in one or more margin buckets so `getFreeBuyingPowerUsdc(account) < amountUsdc` (`packages/perps/src/MarginClearinghouse.sol:466-472`, `packages/perps/src/MarginClearinghouse.sol:1358-1368`).
- An integrator, test harness, or operator relies on the interface/NatSpec contract of `settleUsdc` instead of the implementation details (`packages/perps/src/interfaces/IMarginClearinghouse.sol:403-410`).

Impact

Off-chain simulations, mocks, or operational tooling can predict a successful settlement debit and then fail against the deployed contract. That creates incorrect previews and brittle integration logic around an exposed accounting entrypoint.

Remediation

Update the interface and NatSpec to state that negative `settleUsdc` deltas are capped by free settlement, not merely by the raw settlement balance.

Low ACKNOWLEDGED

Owner can misroute ownerless pool assets to the junior seed receiver

ID: LOW-65

Comment

LOW-65 — Addressed. initializeSeedPosition() now reconciles before both senior and junior seeding. Pre-existing accounted value without an LP claimant is quarantined as unassignedAssets before junior principal is increased and new shares are priced and minted. A regression covers 100,000 USDC admitted through accountExcess() before a 10,000 USDC junior seed: the earlier 100,000 remains unassigned and the seed backs only its own deposit.

A direct USDC transfer can occur before seeding, but it remains unaccounted until accountExcess() admits it. The fix preserves explicit assignment of ownerless assets rather than allocating them implicitly to the first junior seed receiver.

Fix:
https://github.com/Plether-Fi/plether-core/commit/d9e8a3c0ca7b5f78b8267199250e4ddbf240dabe

Locations:

`packages/perps/src/HousePool.sol:711-737`

`packages/perps/src/HousePool.sol:1509-1552`

`packages/perps/src/libraries/HousePoolReconcilePlanLib.sol:76-99`

A junior seed can absorb pre-existing pool assets that had no LP owner path. `initializeSeedPosition(false, ...)` skips `_reconcile`, then immediately increments `juniorPrincipal` and mints junior shares. On the next preview or live reconcile, `planReconcile` sees nonzero `juniorPrincipal` and routes the already-accounted distributable balance into junior principal instead of quarantining it in `unassignedAssets`. Concretely, if the pool has admitted excess USDC with `accountExcess()` before the junior seed exists, the first junior seed receiver receives shares backed by their seed deposit and also acquires the earlier ownerless cash after reconcile.

Preconditions

- The pool holds canonical assets that are not yet represented by tranche principal or `unassignedAssets`, such as admitted excess after `accountExcess()` while no junior seed exists (`packages/perps/src/HousePool.sol:545-555`).
- The owner calls `initializeSeedPosition(false, amount, receiver)` before any reconcile quarantines that value (`packages/perps/src/HousePool.sol:691-740`).
- A later preview or live reconcile runs, which is routine because tranche views and LP settlement both use the reconcile plan (`packages/perps/src/HousePool.sol:1252-1293`, `packages/perps/src/HousePool.sol:1509-1552`).

Impact

Ownerless pool value is reassigned to the junior seed receiver instead of being tracked as `unassignedAssets` for explicit assignment. This breaks the pool's ownership accounting and can misallocate real USDC between seed LPs and later claimants.

Remediation

Run the same reconcile step for junior seeding that senior seeding already performs, or otherwise convert pre-existing distributable ownerless value into `unassignedAssets` before increasing `juniorPrincipal` and minting junior seed shares.

Low ACKNOWLEDGED

Integrator can misread live position margin from engine views

ID: LOW-66

> **Comment**
> LOW-66 — Addressed in the position and Engine API documentation. Live position margin is the clearinghouse-owned PnL-pledge bucket; the Engine does not maintain a separate per-position economic margin balance. Planner snapshots may project changes to that value in memory. Other settlement reservations are not part of the displayed position pledge. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/CfdTypes.sol:22-27`

`packages/perps/src/CfdEngine.sol:36-37`

`packages/perps/src/CfdEngine.sol:889-905`

`packages/perps/src/CfdEngine.sol:1339-1345`

The public position type documents `margin` as an engine-canonical field distinct from the clearinghouse custody bucket, but the engine does not store margin at all. `StoredPosition` explicitly excludes active margin, and `positions()` reconstructs `margin` by reading `clearinghouse.pnlPledgeUsdc(account)` at call time. That means the exported `CfdTypes.Position` view can change because of clearinghouse bucket mutations and claim settlement flows, not because of a position write in engine storage.

This is a misleading interface contract for off-chain consumers. Indexers, replay models, and integrators that treat `CfdTypes.Position.margin` as engine-owned state can miss live margin changes or attribute them to the wrong source, especially when comparing engine storage updates against clearinghouse events or reconstructing account state across claim settlement and margin-management flows.

Preconditions

- A downstream consumer relies on the documented meaning of CfdTypes.Position.margin from the public ABI or shared type library (packages/perps/src/CfdTypes.sol:22-27).
- The consumer reads positions(account), which derives margin from clearinghouse.pnlPledgeUsdc(account) instead of engine-owned position storage (packages/perps/src/CfdEngine.sol:889-905, packages/perps/src/CfdEngine.sol:1339-1345).

Impact

Off-chain state reconstruction can be wrong. Consumers can misattribute margin changes, miss valid state transitions, or build incorrect parity checks between engine position state and clearinghouse bucket state.

Remediation

Align the public documentation with the implementation by documenting that `positions().margin` is a live clearinghouse read, or move margin into engine-owned position storage and keep the clearinghouse bucket as a mirrored custody layer.

Low ACKNOWLEDGED

Same-window redeemer can burn a small LP's shares for no assets

ID: LOW-67

Comment

LOW-67 — Accepted full-balance dust-exit behavior; no code change. A larger request can make an epoch fundable while a small controller's individual asset allocation rounds down to zero, allowing that controller's funded shares to be burned for no asset payout. Full-balance exits are intentionally exempt from the ordinary minimum request size, so that minimum does not eliminate this case. We accept the described floor-rounding outcome and retain the existing epoch allocation policy.

Disposition: https://github.com/Plether-Fi/plether-core/commit/9a16ef4f184d3c23c241906a45daff66be0450b9

Locations:

`packages/perps/src/TrancheVault.sol:771-785`

`packages/perps/src/TrancheVault.sol:1456-1479`

`packages/perps/src/TrancheVault.sol:1589-1595`

`packages/perps/src/HousePool.sol:933-999`

A small LP can lose their full redeem request for zero assets when another controller batches into the same epoch. `requestRedeem` explicitly allows a controller to redeem its entire balance even when `estimateRedeemAssets(shares)` is below the 1 USDC minimum, because `_requireMinimumRedeemSize` only rejects undersized requests when `shares < maxShares`. During settlement, `HousePool` refunds a head epoch only when the whole batch has `fullHeadAssets == 0`. If a larger redeemer joins the same request window, the aggregate head becomes fundable, `fundRedeemEpoch` burns every funded share up front, and each controller's asset claim is later floor-rounded independently in `claimableRedeemAssets`. A dust holder whose pro-rata assets round to zero therefore has their shares burned but receives no USDC, while the lost rounding remainder is returned to `HousePool` as terminal asset dust once the epoch closes.

Preconditions

- The controller redeems its full unlocked balance, so `_requireMinimumRedeemSize` does not reject the request even though `estimateRedeemAssets(shares) < POOL.minTrancheDepositUsdc()` (`packages/perps/src/TrancheVault.sol:1311-1318`, `packages/perps/src/TrancheVault.sol:1589-1595`).

- Another controller submits a larger redeem into the same request epoch, making the aggregated head quote a positive `fullHeadAssets` value in `_fundRedemptionPhase` instead of taking the zero-payout refund branch (`packages/perps/src/HousePool.sol:933-968`).
- At claim time, the small controller's funded-share entitlement is nonzero but its pro-rata funded-asset entitlement floors to zero in `claimableRedeemAssets` (`packages/perps/src/TrancheVault.sol:771-785`).

Impact

The small controller's shares are burned and they receive 0 USDC. The lost value is redistributed to the pool through terminal redemption dust instead of being refunded to the dust holder. Each affected request is capped below the 1 USDC minimum, but the loss is permissionlessly triggerable against any dust-sized full-balance redeemer in the same epoch.

Remediation

Do not fund per-controller redemptions whose pro-rata asset entitlement rounds to zero. Either reject full-balance dust redeems up front as well, or move zero-asset funded entitlements into the refund path so the holder keeps their shares instead of having them burned for no payout.

Low ACKNOWLEDGED

Broken oracle payloads can be misreported as unavailable history

ID: LOW-68

Comment

LOW-68 — Addressed; Pyth API ambiguity acknowledged.

Only the exact four-byte PriceFeedNotFoundWithinRange() error is classified as unavailable history. Other parser failures produce PletherOracle__HistoricalParserFailure with the original error selector and return-data length.

Batch execution reports these parser failures as PendingReason.OracleFailure, preserves completed predecessors, and refunds unused funding once. Other oracle and synchronization errors still revert the transaction.

Pyth uses the same PriceFeedNotFoundWithinRange error for a missing requested feed and for no qualifying update in the supplied payload. These cases cannot be distinguished through that API; neither proves that history is globally unavailable.

Fix:
https://github.com/Plether-Fi/plether-core/commit/34eaecad414174f323ad7e4009aea980c126f262

Locations:

`packages/perps/src/PletherOracle.sol:531-557`

`packages/perps/src/OrderRouterExecutionSidecar.sol:986-993`

`packages/perps/src/OrderRouterExecutionSidecar.sol:295-297`

Malformed or incomplete historical payloads are treated the same as genuinely unavailable history. In `_resolveHistoricalOrderBasket`, the `catch` around `pyth.parsePriceFeedUpdatesUnique` swallows every parser revert, refunds the full execution funding, and returns `ok == false` when `revertOnHistoricalUnavailable` is false. The router batch path then converts that generic `ok == false` into `PendingReason.HistoricalPriceUnavailable`, so a broken oracle worker payload or missing component update is reported as an ordinary no-tick condition instead of a dependency failure. The tests confirm this behavior for a payload missing one configured feed: `updateOrderExecutionPrice(..., _request(commit, false))` returns `ok == false` rather than bubbling a parser error (`packages/perps/test/perps/spec/oracle/OracleSynchronization.t.sol:305-313`).

Preconditions

- The market is on the live historical execution path, so `_resolveHistoricalOrderBasket` is used instead of frozen current-price execution (`packages/perps/src/PletherOracle.sol:499-507`).

- A keeper or direct caller supplies malformed or incomplete historical update data that makes `parsePriceFeedUpdatesUnique` revert before the oracle's success body runs (`packages/perps/src/PletherOracle.sol:531-547`).
- The batch execution path is used, which passes `revertOnHistoricalUnavailable = false` and maps `ok == false` to `PendingReason.HistoricalPriceUnavailable` (`packages/perps/src/OrderRouterExecutionSidecar.sol:986-993-`, `packages/perps/src/OrderRouterExecutionSidecar.sol:295-297`).

Impact

Queue heads remain pending while monitors and operators see a routine historical-unavailability reason instead of a broken payload or parser failure. That delays diagnosis of oracle-worker faults and can unnecessarily prolong execution outages.

Remediation

Only treat the specific Pyth no-unique-update-in-range condition as unavailable history. For any other parser revert, bubble the revert or map it to a distinct dependency-failure path so callers can distinguish bad payloads from missing history.

Low ACKNOWLEDGED

Integrators can misread applyOpenCost return value and overestimate pledged margin

ID: LOW-69

> **Comment**
> LOW-69 — Return-value semantics retained. applyOpenCost returns netMarginChangeUsdc, the new pledge contributed by this action after funded positive cost, not the account's total post-open pledge. Existing position pledge is excluded from that delta. The return name already identifies a change; consumers needing the complete resulting balance should read pnlPledgeUsdc, as settlement does. No total-balance return or accounting change is introduced. Reviewed contract: https://github.com/Plether-Fi/plether-core/blob/9f27a7f9f1c6a7019a63a472997b33bce9184319/packages/perps/src/interfaces/IMarginClearinghouse.sol#L485-L504

Locations:

`packages/perps/src/interfaces/IMarginClearinghouse.sol:490-502`

`packages/perps/src/MarginClearinghouse.sol:1009-1064`

`packages/perps/src/libraries/MarginClearinghouseAccountingLib.sol:165-184`

`packages/perps/src/CfdEngineSettlementSidecar.sol:436-476`

`applyOpenCost()` returns the change in PnL pledge, but its public documentation says it returns the new pledge amount left after the action. The implementation forwards `plan.netMarginChangeUsdc`, which is `marginDeltaUsdc - fundedPositiveCost` and does not include any pre-existing `positionMarginUsdc`. A caller that trusts the ABI or NatSpec and treats the return value as the post-open pledged margin will mis-account existing positions. The current settlement sidecar already ignores this return value and re-reads `pnlPledgeUsdc`, which confirms the semantic mismatch.

Preconditions

- A caller or integrator consumes `applyOpenCost()` through its public ABI and trusts the documented meaning of `netMarginChangeUsdc` (`packages/perps/src/interfaces/IMarginClearinghouse.sol:490-502`).
- The account already has nonzero position margin, so the delta differs from the resulting pledged balance (`packages/perps/src/MarginClearinghouse.sol:1041-1057`).

Impact

Off-chain accounting or future on-chain integrations can over- or under-estimate post-open pledged margin and make incorrect risk or settlement decisions. The protocol itself already has to work around this by ignoring the return value and reading live margin again.

Remediation

Rename the return value to describe it as a delta, or change the function to return the resulting `pnlPledgeUsdc` balance instead of the change.

Low ACKNOWLEDGED

Keeper cannot use single-order execution to preserve progress on unavailable history

ID: LOW-70

Comment

LOW-70 — Addressed. Single-order execution now returns Pending with HistoricalPriceUnavailable when the supplied payload lacks a qualifying historical tick, preserving earlier terminal cleanup and leaving the target queued without updating the Engine mark. The Oracle refunds or defers the forwarded allocation, and the Router refunds only the excess, avoiding double refunds. The batch path already preserves this kind of progress. Unexpected parser errors and other genuine oracle failures still revert single execution; the change does not make every failure recoverable. HistoricalPriceUnavailable describes the supplied payload, not proof that no qualifying history exists elsewhere. Fix: https://github.com/Plether-Fi/plether-core/commit/107c30cfb51e89d2f6da5a7419bbcb3e8451453d

Locations:

`packages/perps/src/OrderRouterExecutionSidecar.sol:210-228`

`packages/perps/src/OrderRouterExecutionSidecar.sol:288-298`

`packages/perps/src/OrderRouterExecutionSidecar.sol:948-966`

`packages/perps/src/OrderRouterExecutionSidecar.sol:968-996`

`packages/perps/src/PletherOracle.sol:200-208`

`packages/perps/src/PletherOracle.sol:456-470`

`packages/perps/src/PletherOracle.sol:517-557`

`packages/perps/test/perps/spec/oracle/OrderRouterExecutionFreshness.t.sol:301-317`

`packages/perps/test/perps/spec/oracle/OrderRouterExecutionFreshness.t.sol:444-459`

The router exposes two execution entrypoints that treat the same oracle failure differently. In the single-order path, `_prepareSingleOracle` calls `updateOrderExecutionPrice(..., revertOnHistoricalUnavailable=true)` and lets `PletherOracle__StalePrice` bubble out (`packages/perps/src/OrderRouterExecutionSidecar.sol:948-963`). In the batch path, `_prepareBatchOracle` calls the batch oracle helper with `revertOnHistoricalUnavailable=false`; the oracle refunds the forwarded funding, returns `ok = false`, and the sidecar converts that into `PendingReason.HistoricalPriceUnavailable` while preserving the call (`packages/perps/src/OrderRouterExecutionSidecar.sol:968-996`, `packages/perps/src/PletherOracle.sol:200-208`, `242-246`, `517-557`). This divergence is externally visible: callers of `executeOrder` must handle a revert for an unavailable historical tick, while callers of `executeOrderBatch` get a structured pending result for the same condition. The tests also codify the mismatch, expecting `executeOrder` to revert but `executeOrderBatch` to return `HistoricalPriceUnavailable`.

Preconditions

- A permissionless keeper calls `executeOrder` or `executeOrderBatch` for a live delayed order that needs a unique post-commit historical tick (`packages/perps/src/OrderRouterExecutionSidecar.sol:210-228`, `288-298`).
- `PletherOracle` cannot find a unique tick in `(commitTime, commitTime + orderSettlementWindow]`, so historical execution is unavailable (`packages/perps/src/PletherOracle.sol:528-557`).
- The caller depends on the public execution API's machine-readable result surface rather than catching a full transaction revert (for example an off-chain keeper, relayer, or automation using `ExecutionResult`/`BatchResult`).

Impact

Automation and integrations cannot rely on one consistent contract for unavailable history. Single-order execution hard-reverts instead of returning a pending reason, which breaks retry logic and refund accounting assumptions that work against the batch API.

Remediation

Unify the two entrypoints so unavailable historical execution is reported the same way in both paths, preferably by returning a structured pending result from `executeOrder` instead of bubbling the oracle revert.

Low ACKNOWLEDGED

Protocol lens can understate pool assets by subtracting treasury credits that never leave the pool

ID: LOW-71

Comment

LOW-71 — Addressed. Pool accounting snapshots no longer subtract the protocol treasury's clearinghouse credit from assets already held by the pool. Treasury credit is separate custody and is still reported separately; net physical pool assets now equal canonical pool assets. Regressions vary treasury credit independently of pool custody. Strengthened invariants reconcile actual token custody and include credit/fee fault injection rather than repeating the erroneous subtraction. Accounting fix: https://github.com/Plether-Fi/plether-core/commit/f1e6b6dc8d2d8303095ab7b9e373c2c64321795f Invariant hardening: https://github.com/Plether-Fi/plether-core/commit/9f27a7f9f1c6a7019a63a472997b33bce9184319

Locations:

`packages/perps/src/CfdEngineProtocolLens.sol:64-69`

`packages/perps/src/CfdEngineProtocolLens.sol:114-126`

`packages/perps/src/MarginClearinghouse.sol:1267-1286`

`packages/perps/src/CfdEngineSettlementSidecar.sol:887-908`

The protocol accounting lens reports less pool backing than the contracts actually recognize whenever the protocol treasury has a clearinghouse balance. `CfdEngineProtocolLens` subtracts `clearinghouse.balanceUsdc(protocolTreasury)` from `pool.totalAssets()` and exposes the result as `netPhysicalAssetsUsdc`, but protocol fees are credited inside the clearinghouse without reducing HousePool cash unless `_settleProtocolFeeTopUp()` actually pays the pool out first. Cash-collected execution and liquidation fees move trader settlement directly to the treasury clearinghouse account while the corresponding non-fee proceeds are the only amounts added to HousePool accounting, so the same treasury balance is not also sitting inside `pool.totalAssets()`. Subtracting the treasury balance again makes the public engine and HousePool snapshots under-report assets and can mislead LP accounting integrations that consume `netPhysicalAssetsUsdc` as a conservative backing figure.

Preconditions

- The configured protocol treasury has a positive clearinghouse balance from cash-collected execution or liquidation fees (`packages/perps/src/MarginClearinghouse.sol:1267-1286`).

- A caller reads `getHousePoolInputSnapshot()` or `getProtocolAccountingSnapshot()` and relies on `netPhysicalAssetsUsdc` as a measure of economic pool backing (`packages/perps/src/CfdEngineProtocolLens.sol:60-69`, `packages/perps/src/CfdEngineProtocolLens.sol:109-139`).

Impact

Public accounting views can understate available pool assets and overstate apparent impairment. Off-chain monitors, dashboards, or LP integrations that trust this field can make incorrect risk or valuation decisions.

Remediation

Stop subtracting the treasury clearinghouse balance from `pool.totalAssets()` in the lens, or only subtract the portion that was explicitly funded by `HousePool.payOut()` and therefore actually left pool custody.

Low ACKNOWLEDGED

Pending depositor can receive impossible share quotes for a wiped tranche

ID: LOW-72

Comment

LOW-72 — Addressed. estimateDepositShares returns zero when projected deposit-pricing assets are zero and shares survive, including before stored principal has been reconciled. The request lens forwards that zero estimate while retaining the pending USDC amount and the existing cancellation/refund path. Empty tranches with no shares still quote normally. The guard is specific to a projected wipe: it does not use new-entry capacity, so valid accepted Senior reservations continue to show estimates when they exhaust that capacity. Other activation gates remain separate from this indicative quote.

Fix:
https://github.com/Plether-Fi/plether-core/commit/899675594d1e043383e10252a8985bac2e5b6271

Locations:

`packages/perps/src/TrancheVault.sol:1298-1308`

`packages/perps/src/TrancheVault.sol:1344-1348`

`packages/perps/src/TrancheVault.sol:1827-1838`

A pending deposit can keep showing a large positive share quote even after the tranche has reached the terminal wipe state and that request can only be cancelled for an asset refund. `estimateDepositShares()` always prices against `_depositPricingAssets()` and `accruedTotalSupply()` without checking whether new entry is disabled, while `maxRequestDeposit()` separately returns zero when `_isTerminallyWiped()` or `POOL.canAcceptTrancheDeposits()` is false. After Junior is wiped to zero principal with surviving supply, HousePool leaves pre-existing deposit epochs pending and cancellable rather than finalizing them (`entriesDeferred`); at that point `estimateDepositShares(10_000e6)` still returns a huge positive value even though the request can never activate. Any integration that uses the vault quote, or `PerpsPublicLens.getLpRequestState()` which forwards it for pending deposits, will present a settlement estimate that is no longer achievable.

Preconditions

- The tranche has nonzero share supply but zero pending principal, so ordinary entry is closed by the terminal-wipe guard (`packages/perps/src/TrancheVault.sol:1302-1308`).
- A deposit request was queued before the wipe and remains pending because HousePool defers zero-NAV activation instead of finalizing the epoch (`packages/perps/src/TrancheVault.sol:1344-1348`, `packages/perps/src/TrancheVault.sol:1827-1838`).

- A user, UI, or automation reads `estimateDepositShares()` to value that pending request or to display its expected settlement outcome.

Impact

Off-chain consumers can misstate a wiped request as still convertible into shares, even though the only reachable outcome is cancellation and refund. This breaks request-state integrity for LP dashboards, accounting jobs, and settlement monitors that rely on the vault's quote surface.

Remediation

Return zero from `estimateDepositShares()` when tranche deposits are unavailable, or expose a separate status-aware estimate that refuses to quote pending requests once activation is impossible. Preferably make the lens use that status-aware path for pending deposit estimates.

Low ACKNOWLEDGED

Trader cannot add new stop-loss protection while degraded mode is active

ID: LOW-73

Comment
LOW-73 — Addressed. Existing-position protection creation no longer rejects solely because the Engine is in degraded mode. A trader can arm protection for an existing LONG or SHORT and subsequently trigger its close while degraded. Router pause, frozen-oracle policy, freshness, position/queue checks and post-lock collateral admission remain enforced, including the strict initial-margin boundary. Degraded mode still blocks attached opening orders that would add exposure. This preserves the ability to protect an existing position without relaxing new-exposure or collateral limits.

Fix:
https://github.com/Plether-Fi/plether-core/commit/9098a042f117fe6623bfca3e496ffe1ed04940ec

Locations:

`packages/perps/src/PositionProtectionBook.sol:209-223`

`packages/perps/SECURITY.md:228`

`packages/perps/README.md:1311-1313`

When `degradedMode` is latched, `createPositionProtection` reverts even for an already-open position, so a trader who is currently unprotected cannot add a stop-loss or take-profit after the protocol enters containment. This path does not increase exposure: the position already exists, and protocol documentation says degraded mode should still permit protective transitions. The code blocks these actions anyway by hard-reverting on `ENGINE.degradedMode()` before validating or storing the protection, while `replacePositionProtection` and `retryPositionProtectionClose` remain available in the same mode.

Preconditions

- The engine's `degradedMode` latch is active (`packages/perps/src/PositionProtectionBook.sol:210-212`).
- A trader already has an open position but no active protection on the account (`packages/perps/src/PositionProtectionBook.sol:213-219`).
- The trader attempts to call `createPositionProtection` during that degraded period.

Impact

Existing positions that were opened without protection cannot add a stop-loss or take-profit during a period when protective actions are supposed to stay available. This leaves affected traders exposed until degraded mode is cleared or they manually close.

Remediation

Remove the degraded-mode rejection from `createPositionProtection` and rely on the existing post-lock risk checks, which already ensure the protected position remains safely collateralized without increasing protocol exposure.

Low | Marked invalid by client

Exercise can pay collateral again after the 90-day sweep

ID: LOW-74

MARKED INVALID BY CLIENT

Out of scope (outside of packages/perps)

Locations:

`packages/options/src/MarginEngine.sol:340-341`

`packages/options/src/MarginEngine.sol:263-265`

`packages/options/src/MarginEngine.sol:281-289`

At exactly `settlementTimestamp + 90 days`, an administrator can sweep an in-the-money series and an option holder can still call `exercise()`. The sweep adds the unclaimed exercise shares to `totalSeriesExercisedShares` and transfers them out, but it does not burn outstanding option tokens. `exercise()` remains enabled at that same timestamp and caps each payout against the original series collateral, without checking the remaining aggregate allocation. The two calls can therefore pay the same exercise allocation from the engine's shared custody.

Preconditions

- The option series is settled and in the money (`packages/options/src/MarginEngine.sol:259-268`).
- The administrator calls `sweepUnclaimedShares()` at or after the 90-day boundary, while the holder exercises at the exact boundary; the sweep opens at equality and exercise closes only after equality (`packages/options/src/MarginEngine.sol:263-265`, `packages/options/src/MarginEngine.sol:333-355`).
- The engine has enough shares in shared custody to satisfy the second transfer, such as writer residual collateral or assets backing another same-side series.

Impact

The same exercise allocation can be paid twice, consuming writer residual collateral or shares backing another series. The overlap exists only at the deadline timestamp, and the second transfer fails if the engine lacks sufficient shares.

Remediation

Make the sweep and exercise windows mutually exclusive, and cap exercise by the remaining aggregate allocation after any sweep.

Info (adjusted) | **Original Severity: High** | ACKNOWLEDGED

Trader can close positions at already-known oracle ticks against LPs

ID: HIGH-1

SEVERITY ADJUSTED BY CLIENT

I propose Informational severity. The report identifies a possible violation of post-commit ordering, but does not establish the claimed value extraction. Closing an existing position at a known favorable price can be normal trading PnL; the next tick is unknown and could favor either side.
A higher severity would require evidence that the timing window is usable and gives traders an exploitable pricing advantage at LP expense.

Comment

HIGH-1 — Acknowledged; severity adjusted to Informational. The concern is that sequencer timestamp lag may allow an oracle tick already observed by a trader to satisfy the protocol's numeric post-commit publication check. We acknowledge that this check alone does not establish that the price was unknown to the trader when the order was submitted.

The reported timing concern does not by itself establish the claimed value extraction from LPs. Closing an existing position at a favorable price can be ordinary trading PnL, and the next tick is unknown and may favor either side. A higher severity requires evidence that the timing window is practically usable and creates an exploitable pricing advantage at LP expense.

This records the existing acknowledgment and severity rationale; it does not claim that the timing concern has been remediated or that exploitation is impossible.

Locations:

`packages/perps/src/OrderRouter.sol:149-160`

`packages/perps/src/OrderRouterExecutionSidecar.sol:437-439`

`packages/perps/src/OrderRouterExecutionSidecar.sol:1022-1030`

`packages/perps/src/PletherOracle.sol:528-530`

A trader can submit an order after seeing a favorable Pyth/Hermes update and still have that tick treated as post-commit. `OrderRouter` records `commitTime` from Arbitrum `block.timestamp`, `PletherOracle` resolves the first unique Pyth basket in `(commitTime, min(commitTime + orderSettlementWindow, block.timestamp)]`, and the execution sidecar only rejects fills whose `oraclePublishTime <= commitTime`. On Arbitrum, `block.timestamp` is the sequencer clock rather than the oracle's publish-time clock, so a sequenced block can carry a timestamp that still trails an already observed Pyth update. If Alice sees a favorable tick with publish time `T`, then commits a close while the sequencer still stamps the block at `T - 1`, the later execution path will accept `T` as the first eligible post-commit tick even though Alice already knew that price when she sent the order. The protocol documentation says delayed execution is supposed to anchor fills to the actual commitment time and prevent execution at already known prices, so this is a real break of the intended MEV boundary. The strongest impact is on close and reduce orders: an existing trader can lock in a known favorable exit, realizing extra PnL or avoiding losses at LP expense. Opens are weaker because they do not pay out immediately, but they share the same boundary failure.

Preconditions

- The order is committed through `executeCommitOrderItem`, which records `commitTime` from `block.timestamp` (`packages/perps/src/OrderRouter.sol:149-160`).
- A trader commits after observing a favorable Pyth update, while Arbitrum sequencer time lags the oracle publish clock by at least one second (`.guard/research/dependencies.md:7-9`).
- The order is later executed through the normal historical path, which requests the first unique Pyth tick after `commitTime` and only rejects fills whose publish time is `<= commitTime` (`packages/perps/src/PletherOracle.sol:528-530`, `packages/perps/src/OrderRouterExecutionSidecar.sol:437-439`).

Impact

Delayed orders can be filled at prices that were already public before the trader committed. That breaks the protocol's post-commit settlement guarantee and creates a direct value-extraction path from LP capital whenever a trader can commit during sequencer timestamp skew.

Remediation

Do not use raw `block.timestamp` as the post-commit oracle boundary on Arbitrum. Add a settlement delay buffer that dominates sequencer timestamp drift, or source the commit boundary from a clock whose semantics match Pyth publish times before accepting a historical tick as post-commit.

Info

Caller can strand its own ETH refund by using an unclaimable recipient

ID: INFO-1

Locations:

```
packages/perps/src/PletherOracle.sol:154-173
packages/perps/src/PletherOracle.sol:264-309
packages/perps/src/PletherOracle.sol:346-360
packages/perps/src/PletherOracle.sol:833-861
```

A caller can permanently lose its own oracle refund by choosing a refund recipient that cannot receive or claim ETH. The payable oracle entrypoints send excess ETH to the supplied refundRecipient with a bounded call and, if that call fails, store the amount in claimableEth[refundRecipient]. claimEthRefund only lets msg.sender withdraw its own recorded balance and sends the ETH back to msg.sender with empty calldata, so a nonpayable contract recipient cannot recover a deferred refund. If the caller uses address(0), the low-level refund call succeeds and the ETH is sent to the zero address. This does not let an attacker take protocol or user funds, but it can permanently burn or strand the caller's overpaid ETH.

Preconditions

- A caller supplies a `refundRecipient` that cannot safely accept ETH immediately or cannot later call `claimEthRefund()`.
- The relevant oracle call has excess ETH to return or enters the nonreverting unavailable-history refund path.
- No alternate rescue exists because deferred balances are only withdrawable by `msg.sender` in `claimEthRefund()` (`packages/perps/src/PletherOracle.sol:349-360`).

Impact

The caller's refund can be burned or stranded permanently at the chosen recipient address.

Remediation

Reject `refundRecipient == address(0)` and consider always crediting the caller instead of an arbitrary third party, or add a caller-authorized rescue path that can redirect deferred refunds.

Info

Owner can cancel pool config without an event

ID: INFO-2

Locations:

`packages/perps/src/interfaces/IHousePool.sol:190-208`

`packages/perps/src/HousePool.sol:298-355`

`packages/perps/src/HousePool.sol:357-362`

Event-only monitors cannot observe when governance cancels a staged HousePool configuration change. proposePoolConfig emits PoolConfigProposed and finalizePoolConfig emits PoolConfigFinalized, but cancelPoolConfigProposal only clears pendingPoolConfig and poolConfigActivationTime. Frontends and alerting systems that do not poll storage can continue to display the cancelled proposal as pending.

Preconditions

- A pool configuration proposal is active after `proposePoolConfig` sets `pendingPoolConfig` and `poolConfigActivationTime` (`packages/perps/src/HousePool.sol:298-313`).
- The owner calls `cancelPoolConfigProposal`, which deletes that proposal state without emitting any cancellation event (`packages/perps/src/HousePool.sol:357-362`).

Impact

Indexers, frontends, and alerting systems that treat HousePool governance events as the source of truth can display a cancelled configuration as still pending. Users and operators can make deposit, withdrawal, or risk decisions on stale governance metadata.

Remediation

Emit a dedicated cancellation event from `cancelPoolConfigProposal` that includes the proposal being cleared and its activation timestamp.

Info ACKNOWLEDGED

Router can return a misleading oracle rotation error

ID: INFO-3

> **Comment**
> INFO-3 — Addressed. Oracle rotation now distinguishes invalid candidate wiring (OrderRouter__InvalidPletherOracle) from a candidate that fails to retain the complete copied active policy (OrderRouter__OraclePolicyMismatch). The latter is checked by comparing the full configuration after applyConfig. SDK ABIs include the new selector. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/IOrderRouterErrors.sol:43-44`

`packages/perps/src/router/OrderOracleExecution.sol:46-74`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:487-500`

Oracle rotation can fail with a custom error whose notice describes a different failure class. `OrderRouter__InvalidPletherOracle` is documented as covering invalid code, Pyth, engine, or HousePool wiring, and `_validateOracleConfig()` uses it for those checks. After those checks pass, `OrderLiquidationBatchLogic.applyOracleConfig()` copies the active policy into the candidate oracle and reuses the same selector if the candidate does not report the copied policy exactly. This does not move funds or change protocol state incorrectly, but it gives callers and automation an imprecise machine-readable reason for an admin rotation failure.

Preconditions

- The owner finalizes an oracle rotation through `applyOracleConfig` and the candidate oracle is wired well enough to pass `_validateOracleConfig` (`packages/perps/src/router/OrderOracleExecution.sol:46-74`).
- The candidate does not preserve the copied live policy exactly after `candidate.applyConfig(policy)` (`packages/perps/src/router/OrderLiquidationBatchLogic.sol:495-500`).

Impact

Admin tooling, monitors, and runbooks that treat `OrderRouter__InvalidPletherOracle` as a pure wiring failure receive the wrong machine-readable diagnosis and can take the wrong remediation path during oracle rotation incidents.

Remediation

Use a distinct custom error for policy-copy or post-copy verification failure, and reserve `OrderRouter__InvalidPletherOracle` for the address and binding checks described in the interface.

Info ACKNOWLEDGED

Integrator can misread settlement buffer checks from stale evaluator docs

ID: INFO-4

> **Comment**
> INFO-4 — Addressed. Evaluator and security documentation now state that the snapshot reads the Engine's live settlementBufferBps and the planner enforces that configured buffer. The old statement that the evaluator leaves the buffer at zero was stale. Execution continues to enforce the same configured buffer. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/CfdOrderPolicyEvaluatorBase.sol:101-104`

`packages/perps/src/CfdOrderPolicyEvaluatorBase.sol:201-250`

`packages/perps/src/CfdEngineLens.sol:601-637`

`packages/perps/src/router/OrderLiquidationBatchLogic.sol:404-416`

The policy evaluator documentation says its assessment snapshot leaves settlementBufferBps at zero, but the implementation loads the live value from the engine. CfdOrderPolicyEvaluatorBase._buildRawSnapshot assigns engine.settlementBufferBps(), and CfdEnginePlanLib.planOpen uses that field when checking protected settlement headroom. The router and lens paths also use the live buffer, so this is not an on-chain solvency mismatch, but readers of the contract documentation can reason about the wrong preview semantics.

Preconditions

- A caller relies on the documented policy-evaluator path (`assessOrder`) to preflight an open/increase order before submitting it (packages/perps/src/interfaces/ICfdOrderPolicyEvaluator.sol:48-64).
- The deployment has a nonzero settlement buffer configured in the engine (`settlementBufferBps`) so buffer enforcement changes the open result (packages/perps/src/CfdOrderPolicyEvaluatorBase.sol:246, packages/perps/src/CfdEngineLens.sol:634).

Impact

Off-chain executors, wallets, and integrators can produce wrong bounded-order UX and wrong commit admissibility decisions because the documented evaluator semantics do not match the router's actual commit-time policy. Users can be told an order should pass when the router will reject it, or vice versa if an integrator tries to compensate for the documented zero-buffer rule.

Remediation

Make the public planning surfaces use one documented solvency rule. The simplest fix is to either actually zero `settlementBufferBps` inside `CfdOrderPolicyEvaluator-Base._buildRawSnapshot` and stop using that surface for commit authorization, or update the evaluator documentation and any dependent callers so every open simulation path loads and enforces the live engine buffer identically.

Info ACKNOWLEDGED

Integrator can misclassify open-order failures in planner policy tooling

ID: INFO-5

Comment
INFO-5 — Addressed. Planner and preview documentation now describes both commit-time and execution-time failure categories: degraded mode maps to ExecutionTimeProtocolStateInvalidated, fee-drained margin to ExecutionTimeUserInvalid, other recognized failures to CommitTimeRejectable, and OK/unmapped values to None. Existing classification behavior is preserved. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/interfaces/ICfdEnginePlanner.sol:102-107`

`packages/perps/src/CfdEnginePlanner.sol:125-142`

`packages/perps/src/libraries/CfdEnginePlanLib.sol:79-105`

`ICfdEnginePlanner.getOpenFailurePolicyCategory()` is documented as a commit-time classifier that returns a commit category or `None`, but the implementation returns execution-only categories as well. For example, `DEGRADED_MODE` maps to `ExecutionTimeProtocolStateInvalidated` and `MARGIN_DRAINED_BY_FEES` maps to `ExecutionTimeUserInvalid`. Any generated docs, SDK wrapper, or off-chain policy engine that trusts the interface NatSpec can treat those values as impossible or equivalent to `None`, which breaks order-labeling and failure-handling logic around the planner's canonical policy surface.

Preconditions

- Off-chain tooling or generated documentation is built from `ICfdEnginePlanner.getOpenFailurePolicyCategory()` at `packages/perps/src/interfaces/ICfdEnginePlanner.sol:102-107` instead of the concrete implementation.
- An operator, keeper tool, or integrator encounters an open revert code such as `DEGRADED_MODE` or `MARGIN_DRAINED_BY_FEES`, which the implementation maps at `packages/perps/src/libraries/CfdEnginePlanLib.sol:96-102`.

Impact

Consumers can mis-handle canonical planner failure categories, leading to incorrect order-state labeling or bad off-chain policy decisions for open-order failures.

Remediation

Update the interface NatSpec to state that this function can return any `OpenFailurePolicyCategory`, including execution-time categories, or rename/expose a truly commit-only helper.

Info ACKNOWLEDGED

Operators can misread recovery-sidecar failures as execution-sidecar failures

ID: INFO-6

> **Comment**
> INFO-6 — Addressed. Recovery-owned InvalidSettledReason and OrderIdentityMismatch errors now use the OrderRecoverySidecar__ namespace, and the misleading execution-sidecar declaration was removed. Settled-receipt identity, reason and bounty validation remain enforced; consumers should decode the corrected selectors. Fix: https://github.com/Plether-Fi/plether-core/commit/9fd9a10b1e916d3bf9dee7004121b29b345fd991

Locations:

`packages/perps/src/OrderRecoverySidecar.sol:22-23`

`packages/perps/src/OrderRecoverySidecar.sol:148-154`

`packages/perps/src/OrderRecoverySidecar.sol:189-219`

`OrderRecoverySidecar` defines and emits errors whose ABI names still say `OrderRouterExecutionSidecar__...`. This is a copy-paste mismatch: the recovery module's own failure selectors are labeled as if they came from the execution sidecar.

That makes revert decoding and monitoring semantically wrong. Tooling, runbooks, or tests that classify failures by error name will attribute recovery-path faults to the wrong component, even though the revert originated in `OrderRecoverySidecar`.

Preconditions

- A caller or off-chain observer relies on revert names or selectors emitted by `OrderRecoverySidecar` during expiry or settled-terminal recovery flows (`packages/perps/src/OrderRecoverySidecar.sol:63-176`).

Impact

Failure attribution and alerting are misleading for recovery-path errors. This does not change state safety, but it degrades debugging, monitoring, and component-level incident response.

Remediation

Rename the recovery-sidecar errors to `OrderRecoverySidecar__...` and update call sites and tests to use the recovery module's own error namespace.

Advisory to Clients

Cecuro highly recommends scheduling a follow-up security assessment within six months of this report or immediately after any significant modifications to the codebase, whichever occurs first. This practice is essential for preserving the project's security posture and identifying potential vulnerabilities that may arise from code changes or updates.

Disclaimer

The smart contracts submitted for analysis have been reviewed using Cecuro.ai's AI security analysis system, applying industry-standard vulnerability detection patterns and best practices at the time of this report. The findings disclosed in this report reflect the AI system's assessment of the submitted source code.

This report contains no statements or warranties on the identification of all vulnerabilities or the overall security of the code. Any security review methodology may not detect all potential issues, particularly novel attack vectors, complex business logic flaws, or economic exploits. The report covers the code submitted at the specified version and may not be relevant after any modifications.

Do not consider this report as a final and sufficient assessment regarding the security, utility, or bug-free status of the code. We recommend complementing this analysis with additional independent audits and a public bug bounty program to strengthen the security posture of your smart contracts.

Smart contracts are deployed and executed on blockchain platforms. The platform, its programming language, compiler, and other related software may contain vulnerabilities beyond the scope of code-level analysis. Cecuro.ai cannot guarantee the explicit security of the analyzed smart contracts.

This report does not constitute financial, legal, or investment advice. It is not a recommendation to buy, sell, or invest in any token or project, nor an endorsement of any kind. Users should conduct their own independent due diligence.

© Cecuro, Inc 2026. All rights reserved.

cecuro

Agentic Smart Contract Auditing